

Министерство образования и науки Республики Казахстан

Некоммерческое акционерное общество
Алматинский университет энергетики и связи

ПРОЕКТИРОВАНИЕ БАЗ ДАННЫХ

Учебное пособие

АУЭС
Алматы
2016

СТ НАО

УДК
ББК
К

Рецензенты:

доктор технических наук, профессор,
профессор кафедры «Информатика» механико-математического факультета
КаззНУ им. Аль-Фараби

Шмыгалева Т.А.,

кандидат технических наук, ассистент профессора,
заведующий кафедрой «Информационная безопасность»

Сейлова Н.А.,

доктор физико-математических наук, профессор,
заведующий кафедрой «Компьютерных технологий»

Куралбаев З.К.

Рекомендовано к изданию Ученым советом Алматинского университета
энергетики и связи (протокол № от 27.12.2016 г.). Печатается по
тематическому плану выпуска ведомственной литературы АУЭС на 2015 год,
позиция 51.

Сатимова Е.Г.

Проектирование баз данных: Учебное пособие (для студентов специальностей 5B070400 – Вычислительная техника и программное обеспечение)/Е.Г.Сатимова. – Алматы: АУЭС, 2016. – 126 с.:табл. 3, ил. 22, библиогр. – 12 назв.

ISBN

Предлагаемое учебное пособие предназначено для изучающих реляционную модель баз данных, язык SQL и его процедурное расширение Transact-SQL.

Учебное пособие предназначено для студентов, обучающихся по специальности 5B070400 – Вычислительная техника и программное обеспечение.

УДК
ББК

ISBN

© АУЭС, 2016.

Сатимова Е.Г., 2016

Введение

Задачей курса «Проектирование баз данных» является не изучение особенностей той или иной СУБД, а освоение технологий создания и работы с базами данных. Проектирование информационных систем всегда начинается с определения цели проекта.

Настоящее учебное пособие написано для студентов специальности «Вычислительная техника и программное обеспечение» с учетом комментариев и предложений студентов, а также использованием реальных примеров, созданных студентами.

Хорошая база данных не получается случайно, структура ее содержимого должна тщательно прорабатываться, поэтому в Теме 1 достаточно подробно рассмотрен пример анализа предметной области или ТЗ (технического задания) организации учебного процесса с использованием системы оценки освоения дисциплин для создания учебной базы данных на основе MS Visio. Также приводится подробное описание примера проектирования базы данных. На самом деле, проектирование базы данных — это важная часть работы с БД, даже хорошая СУБД будет плохо работать с неудачно спроектированной базой данных.

Вывод требуемой информации из базы данных формулируется в виде запросов. Универсальным языком запросов к MS SQL Server является язык структурированных запросов SQL (Structured Query Language). Следует отметить, что язык SQL имеет множество диалектов, порожденных различными разработчиками.

В качестве используемой СУБД в лабораторных работах применяется система управления базами данных MS SQL Server 2008/2012, хотя в равной степени это могла быть любая из современных СУБД, так как основной упор делается на изучение языка SQL, который только на первый взгляд кажется легким (можно даже применить термин «думать» на языке SQL). В последней версии MS SQL Server используется диалект Transact SQL [1], который базируется на существующем на сегодняшний день стандарте SQL для реляционных баз данных, установленном ANSI - Американским национальным институтом стандартов - Стандарт SQL:2008 и очень близок к нему. В методических указаниях достаточно подробно рассмотрены вопросы создания скриптов самой базы данных на языке SQL, манипуляции данными от создания простых до конструирования сложных запросов по поиску информации в спроектированной базе данных на диалекте Transact SQL.

1 Концептуальное проектирование базы данных

Прежде чем реализовывать структуру базы данных (БД) в любой среде проектирования (в нашем случае это MS Visio), необходимо создать проект этой базы данных на контрольном примере.

Контрольный (или тестовый) пример представляет собой упрощенный вариант реальной задачи, просчитываемый вручную вплоть до получения конечного результата. В контрольном примере указываются требования к объему и составу данных используемой исходной информации и результатов решения, рассматриваются основные понятия проектирования, как части рабочей среды, системный анализ которой приведен ниже.

С помощью контрольного примера проверим постановку задач обработки данных и работоспособность отдельных программ и информационной системы в целом.

1.1 А нужно ли проектирование БД

Очень часто проектированию данных не уделяют должного внимания. Правильно спроектированная БД облегчает управление данными и становится ценным поставщиком информации. Плохо спроектированная БД вероятнее всего станет накопителем избыточной информации, т. е. неоправданного дублирования данных [2]. Избыточность, как правило, затрудняет выявление ошибок в данных.

Доступность современного превосходного программного обеспечения баз данных дает возможность даже новичкам создавать базы данных и приложения баз данных. К сожалению, подход «разработка без проекта» становится причиной множества провалов. Как показывает практика, многие, если не все, ошибки в плохо спроектированных системах баз данных не могут устранить даже лучшие программисты и менеджеры. При этом даже превосходная система управления базой данных (СУБД), вероятнее всего, не поможет разрешить проблемы, порожденные или усиленные плохим проектированием. Можно провести аналогию со строительством: даже лучшие каменщики и плотники не смогут построить хорошее здание по плохому проекту [3]. При этом, как вы понимаете, ни один уважающий себя человек не обратится к непрофессионалу при строительстве дома, особенно при закладке фундамента. Так и с базами данных, которые являются краеугольным камнем почти каждого бизнес-проекта.

Большинство проблем в системах управления базами данных чаще всего возникает из-за плохого проектирования. Поскольку хранилища данных (data warehouse) получают информацию от рабочих (операционных) баз данных, концепции, структура и процедуры хранилищ данных станут более обоснованными, если вы будете достаточно хорошо разбираться в структуре и реализации рабочих БД. Проектирование баз данных имеет чрезвычайно

важное практическое значение, чем и объясняется то внимание, которое ему уделено в первой главе данного учебного пособия.

В принципе, в результате создания базы данных, мы получим каким-то образом связанные между собой таблицы, которые обоснованы на анализе данных (data mining), который является важнейшим компонентом в новейших системах принятия решений. На этапе анализа концептуальных требований и информационных потребностей необходимо решить следующие задачи:

- анализ требований пользователей к БД (концептуальных требований);
- выявление имеющихся задач по обработке информации, которая должна быть представлена в БД (анализ приложений);
- выявление перспективных задач (перспективных приложений);
- документирование результатов анализа.

Чтобы исследовать различные аспекты использования СУБД, мы с вами рассмотрим достаточно сложный пример, приведенный к 4 нормальной форме (4НФ), приближенный к действительности, – ведение электронной документации деканата учебного заведения (ВУЗа), содержащую информацию об учебном процессе обучения студентов в вузах. Каждый вуз решает данную задачу по-своему, в настоящее время без использования баз данных не обходится почти никто. Наш пример не является реальным примером обучения студентов в АУЭС или другом вузе, однако очень близок к тем задачам, которые стоят в действительности перед деканатами АИЭС и других вузов. Проект учебный, он не учитывает многие нюансы и будет дополняться от темы к теме.

Рассмотрим постановку задачи. Организация учебного процесса с использованием системы оценки освоения дисциплин характеризуется следующими особенностями: необходимо учитывать списки студентов групп; перечень изучаемых предметов; преподавательский состав кафедр, обеспечивающих учебный процесс; сведения о лекционных, практических и других видах занятий в каждой группе; результаты сдачи экзаменов по каждому из проведенных занятий. Учебная программа строится в соответствии с утвержденным графиком на каждую специальность до начала учебного процесса. Учебная программа включает предлагаемые на факультете обязательные и элективные курсы. Каждому студенту предлагается учебный план на основе рабочего, исходя из того, что в процессе обучения он должен набрать заданное количество кредитов.

1.2 Описание предметной области

Если в системе обрабатывается информация о студентах, сущностью может являться студент, если обрабатывается информация об экзамене, то сущность – экзамен и т.п. Каждая сущность обладает определенным набором свойств (рассматриваем только свойства, представляющие интерес для

пользователей в рамках проводимого исследования), которые запоминаются в информационной системе.

Так, например, в качестве свойств сущности «СТУДЕНТ» можно указать фамилию, дату рождения, место рождения, в качестве свойств сущности «ЭКЗАМЕН» – предмет, дату проведения экзамена, экзаменаторов.

Совокупность сущностей, характеризующихся в информационной системе одним и тем же перечнем свойств, называется *классом сущностей* (набором объектов). Так, например, совокупность всех сущностей «СТУДЕНТ» составляет класс сущностей «СТУДЕНТ», совокупность всех сущностей «ЭКЗАМЕН» составляет класс сущностей «ЭКЗАМЕН».

Класс сущностей описывается перечнем свойств сущностей, составляющих этот класс.

Экземпляр сущности будем называть конкретную сущность (сущность с конкретными значениями соответствующих свойств).

Пример класса сущностей «СТУДЕНТ» и конкретного экземпляра сущности:

<u>Класс сущностей</u>	<u>Экземпляр сущности</u>
СТУДЕНТ	
Фамилия	Ахметов
Дата рождения	21.05.97
Место рождения	Астана

Взаимоотношения сущностей выражаются связями (Relationships).

Различают классы связей и экземпляры связей. Классы связей – это взаимоотношения между классами сущностей, а экземпляры связи – взаимоотношения между экземплярами сущностей. Класс связей может затрагивать несколько классов сущностей.

Число классов сущностей, участвующих в связи, называется *степенью связи* $n = 2, 3, \dots$. Так, например, класс сущностей «СТУДЕНТ» связан с классом сущностей «ЭКЗАМЕН» связью «сдает». Степень этой связи равна двум. В качестве примера связи степени три можно указать связь «родители» между тремя классами сущностей «МАТЬ», «ОТЕЦ», «РЕБЕНОК». При $n=2$ связь называется бинарной.

Рассмотрим классификацию бинарных связей.

Числа, которые описывают типы бинарных связей (1:1, 1:M, M:N), обозначают максимальное количество сущностей на каждой стороне связи. Эти числа называются максимальными кардинальными числами, а соответствующая пара чисел называется максимальной кардинальностью.

В зависимости от того, сколько экземпляров сущности одного класса связаны со сколькими экземплярами сущности другого класса, различают следующие типы связей:

– связь 1:1. Одиночный экземпляр сущности одного класса связан с одиночным экземпляром сущности другого класса. Примером является связь «соответствует» между классами сущностей «ФАКУЛЬТЕТ» и

«РАСПИСАНИЕ ЭКЗАМЕНОВ НА ФАКУЛЬТЕТЕ» (каждому факультету соответствует свое расписание, эта тема сама по себе сложная и рассматриваться в данном учебном пособии не будет);

– связь 1:M. Единый экземпляр сущности одного класса связан с многими экземплярами сущности другого класса. Примером является связь «обучение» между классами сущностей «ФАКУЛЬТЕТ» и «СТУДЕНТ» (на одном факультете обучается много студентов);

– связь M:N. Несколько экземпляров сущности одного класса связаны с несколькими экземплярами сущности другого класса. Примером является связь «сдают» между классами сущностей «СТУДЕНТ» и «ЭКЗАМЕН» (каждый абитуриент сдает несколько экзаменов, и каждый экзамен сдают много студентов).

1.3 Описание информационного представления предметной области

В качестве основного понятия для описания предметной области, как уже отмечалось, используется понятие сущности (объекта), характеризуемой набором определенных свойств. Для информационного описания сущности вводится понятие атрибута. *Атрибут* – поименованное свойство (характеристика) сущности.

Атрибут представляет собой информационное отображение свойства сущности и принимает конкретное значение из множества допустимых значений. Так, например, для сущности «СТУДЕНТ» атрибут «фамилия» у конкретного экземпляра сущности принимает конкретное значение «Ахметов».

Другим основным понятием для описания предметной области является понятие связи. Для представления связей между экземплярами сущностей могут использоваться атрибуты. В этом случае связь устанавливается путем включения в совокупность атрибутов сущности атрибута, однозначно идентифицирующего экземпляр сущности, находящийся в отношении с исходным экземпляром сущности.

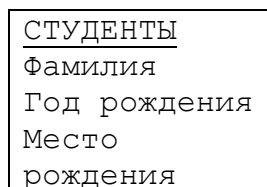
Так, если мы рассмотрим класс сущностей «ПРЕДМЕТ», представленный одной совокупностью атрибутов (название, номер), и класс сущностей «УСПЕВАЕМОСТЬ», представленный другой совокупностью атрибутов (код студента, название предмета, дата экзамена, группа студента, оценка). Для представления связи «экзамены» (тип связи 1:1) в совокупность атрибутов «УСПЕВАЕМОСТЬ» можно включить атрибут «название предмета».

1.4 Построение ER-диаграмм

Чаще всего концептуальная модель представляется в виде диаграммы сущностей – связей (entity – relationship) или ER-диаграммы [4].

Процесс построения ER-диаграммы называется ER-моделированием. При этом используются следующие классические обозначения.

Класс сущностей представляется в виде четырехугольника. В четырехугольнике записано уникальное имя класса сущности (прописными буквами, обычно во множественном числе) и имена атрибутов строчными буквами.



Связи между сущностями обозначаются стрелками, рядом со стрелками указывается имя связи, а также максимальная кардинальность связи (максимальное число сущностей, которые могут участвовать в связи). Чтобы показать, что сущность обязана участвовать в связи (каждый экземпляр должен быть связан с экземпляром другого класса), на линию связи помещают перпендикулярную черту, а чтобы показать, что сущность может (но не обязана) участвовать в связи, на линию связи помещают овал.

Отношения между группами, предметами и преподавателями с их мощностями представлены на следующей схеме (рисунок 1.1). Атрибуты объектов на схеме не указаны.

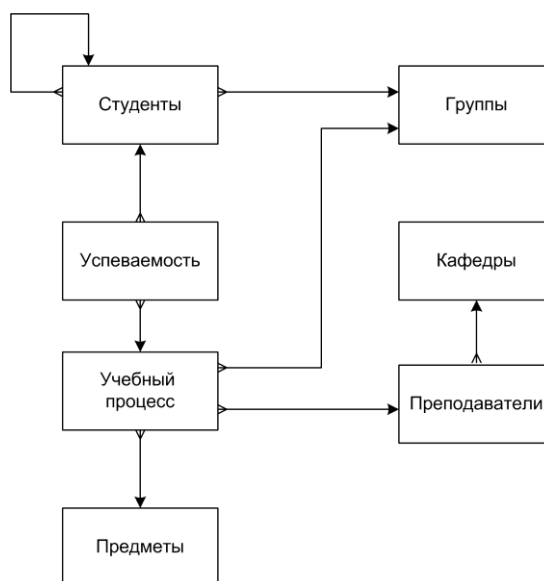


Рисунок 1.1 – Диаграмма учебного процесса

При описании сущностей обычно выделяют особые совокупности атрибутов – ключи и внешние ключи. Ключ уникально идентифицирует экземпляр сущности и, вместе с внешним ключом, используется для реализации связей. На диаграммах атрибуты, входящие в первичный ключ, подчеркиваются (обозначение РК).

Важность понятий первичного и вторичного ключей будет проиллюстрирована далее на примерах.

Приведенные выше обозначения не являются общеупотребительными. Часто производители программ, позволяющих рисовать ER-диаграммы, используют свою систему обозначений. Например, при использовании программы MS Visio в качестве основной используется так называемая реляционная нотация. В данной системе обозначений связи изображаются стрелками между сущностями и названия связей не пишутся. Название сущности выделяется цветом, поля, входящие в первичный ключ, отделяются чертой от остальных атрибутов. Обязательные атрибуты отображаются с помощью полужирного шрифта.

Кроме того, слева от атрибута указывается, входит ли данный атрибут в первичный ключ, а также является ли атрибут внешним ключом.

На практике использование различных способов записи ER-диаграмм не представляет особой сложности – беглое ознакомление с соответствующим разделом документации позволяет быстро освоить используемую систему обозначений.

1.5 Определение и моделирование сущностей и связей

При разработке концептуальной модели, прежде всего, следует определить сущности. С этой целью нужно сделать следующее:

- необходимо понять, какая информация должна храниться и обрабатываться, и можно ли это определить, как сущность;
- присвоить этой сущности имя;
- выявить атрибуты сущности и присвоить им имя;
- выявив сущности, необходимо определить, какие связи имеются между ними;
- при определении связей (естественно, рассматриваем только те связи, которые имеют отношение к решаемым задачам обработки данных) необходимо учитывать следующее:
 - то, как экземпляр одной сущности связан с экземпляром другой сущности;
 - то, как должны быть установлены связи, чтобы была возможность ответа на все запросы пользователей (исходя из их информационных потребностей).

На стадии начальной разработки БД большую часть полезной информации получают при опросе пользователей системы. Поэтому этап концептуального проектирования необходимо начать с выявления надежных источников информации. Составим список потенциальных пользователей системы.

Источники информации:

- 1) *Администраторы БД* – на основе официальных документов и отчетностей формируют структуру БД, изменяя и дополняя ее.

- 2) *Преподаватели* – предоставляют информацию.
- 3) *Студенты* – предоставляют информацию.
- 4) *Сотрудники*, не занимающиеся преподавательской деятельностью – предоставляют информацию.

В процессе анализа предметной области «ОБУЧЕНИЕ СТУДЕНТОВ» (не забывая, что это учебный пример и весь процесс глубоко не рассматривается) выделяются отдельные сущности.

Поскольку вещи одного типа хранятся в отдельных объектных множествах, можем выделить следующие сущности: «СПЕЦИАЛЬНОСТИ», «ГРУППЫ», «СТУДЕНТЫ», «КАФЕДРЫ», «ПРЕПОДАВАТЕЛИ», «ПРЕДМЕТЫ», «УЧЕБНЫЙ ПРОЦЕСС», «УСПЕВАЕМОСТЬ».

В контрольном примере рассматривается только часть бизнес-правил учебного процесса.

В соответствии с предметной областью система строится с учетом следующих особенностей. Например:

- студент не может учиться на двух специальностях и в двух группах одновременно;
- не может быть двух студентов с одинаковыми номерами зачетной книжки;
- на кафедре работает много преподавателей;
- не может быть двух кабинетов с одинаковыми номерами;
- у кафедры не может быть нескольких заведующих кафедрой;
- у группы может быть только один староста;
- по результатам промежуточной аттестации студенту выставляется дифференцированная оценка в принятой в вузе системе баллов, характеризующая качество освоения студентом знаний, умений и навыков по данной дисциплине;
- преподаватель может вести один и тот же предмет в нескольких группах или несколько разных предметов в одной группе;
- студенты сдают экзамены по предметам, которые они изучали;
- у группы может быть только один куратор;
- преподаватель может занимать только одну должность;
- преподаватель может иметь только одну ученую степень;
- не может быть двух одинаковых специальностей;
- не может быть двух одинаковых ученых степеней;
- не может быть двух одинаковых должностей;
- группа занимается только по одному учебному плану.

Может быть сформулировано множество вопросов к базе данных, например, к какой группе относится студент, на каких кафедрах работают преподаватели, какие предметы, в каких группах они ведут, какую оценку получили студенты по определенным видам занятий по определенному предмету и т.д. Чтобы ответить на широкий круг возможных вопросов к базе данных, следует рассмотреть отношения между объектными множествами, необходимо присвоить связям имена и определить тип связей.

Между объектами «ГРУППЫ» и «СТУДЕНТЫ» существует отношение «один-ко-многим», поскольку одна группа включает много студентов, а один студент входит только в одну группу. Аналогично устанавливается связь между объектами «КАФЕДРЫ» и «ПРЕПОДАВАТЕЛИ», которые также находятся в отношениях «один-ко-многим» (на одной кафедре работает много преподавателей, но каждый преподаватель работает на определенной кафедре). По каждому предмету проводится множество видов занятий в различных группах разными преподавателями. Это определяет отношения «многие-ко-многим» между множествами «ГРУППЫ» и «ПРЕДМЕТЫ», «ГРУППЫ» и «ПРЕПОДАВАТЕЛИ», «ПРЕДМЕТЫ» и «ПРЕПОДАВАТЕЛИ», «ПРЕДМЕТЫ» и «ВИДЫ ЗАНЯТИЙ».

1.6 Пример построения подробной диаграммы «сущность – связь» для предметной области «Учебный процесс»

Исходя из выше изложенного, можно выделить следующие базовые сущности нашей предметной области, учитывая, что каждая таблица должна иметь первичный ключ:

1) *Специальности (SPECIALITY)*. Атрибуты – номер, название специальности. Эта сущность необходима, т.к. студент поступает в ВУЗ на специальность, а потом распределяется по группам.

2) *Группы (GRUP)*. Атрибуты этой таблицы – номер, название, количество студентов, курс. Эта сущность отводится для хранения сведений о группах. Так как названия групп формируются в зависимости от факультета, кафедры, года поступления и будут многократно встречаться в связях с другими сущностями базы данных, то их целесообразно нумеровать и ссылаться на эти номера. Для этого вводится целочисленный атрибут Grup_ID – это ключевое поле, которое будет наращиваться на единицу при вводе в базу данных нового наименования.

3) *Студенты (STUDENTS)*. Эта сущность отводится для хранения сведений о студентах. Атрибуты – номер, фамилия, имя, отчество, дата рождения, адрес, номер группы, студент-староста. Stud_ID (идентификационный номер студента – обычно номер зачетной книжки) выбран в качестве первичного ключа (РК). Использование имени, фамилии и отчества в качестве идентификатора является неудобным решением, поскольку может появиться несколько однофамильцев. Это же правило относится к сочетанию фамилия-имя (отчества может не быть вообще). А учитывая, что идентификаторы студентов будут многократно встречаться в связях с другими сущностями базы данных, то их целесообразно нумеровать и ссылаться на эти номера.

4) *Кафедры (CHAIR)*. Атрибуты - номер, название, телефон и адрес. Эта сущность вводится для хранения сведений о кафедрах вуза. Chair_ID – это целочисленное ключевое поле, которое будет наращиваться на единицу при вводе в базу данных нового наименования автоматически.

5) *Преподаватели (TEACHER)*. Атрибуты – номер, фамилия, имя, отчество, должность, ученая степень и принадлежность к кафедре. Эта сущность отводится для хранения сведений о преподавателях. Использование имени, фамилии и отчества в качестве идентификатора является неудобным решением, а учитывая, что идентификаторы преподавателей будут многократно встречаться в связях с другими сущностями базы данных, то их целесообразно нумеровать и ссылаться на эти номера. Для этого вводится целочисленный атрибут Teach_ID – это ключевое поле, которое и будет идентификатором данной сущности.

6) *Предметы (SUBJECT)*. Атрибуты – номер, название, общее количество часов за семестр, лекционные часы, часы практикумов, лабораторные часы. Эта сущность отводится для хранения сведений о предметах. Атрибут Subj_ID – это ключевое поле, которое и будет идентификатором данной сущности. Атрибут Subj_NAME является текстовым типом данных для отображения названия предмета.

Для того, чтобы создать точную модель данных, проектировщик должен иметь полное представление о типах данных компании, объемах информации и способах ее использования. Но данные сами по себе не дают представления о проведении учебного процесса. С точки зрения БД набор данных становится значимым только тогда, когда определены бизнес-правила.

Бизнес-правила это краткое и точное повествовательное описание политики, процедур и норм в условиях предприятия. Если бизнес-правила заданы корректно, то в них определяются и сущности, и атрибуты, и связи, и связности, и мощность, и ограничения. Некоторые бизнес-правила реализуются при помощи триггеров и процедур. Бизнес-правила должны быть не только эффективными, но понятными, и к тому же они описывают на простом языке основные и точные характеристики данных с точки зрения компании.

В процессе ER-моделирования системы учебного процесса был получен следующий набор бизнес-правил для сущностей и создания связей между ними (краткое перечисление), связи сущностей базируются на бизнес-правилах, построенных на основе подробного описания операций.

Бизнес-правило 1.

Каждый студент поступает на одну специальность. На основе первого бизнес-правила мы получаем сегмент ER-модели, представленный на рисунке 1.2.

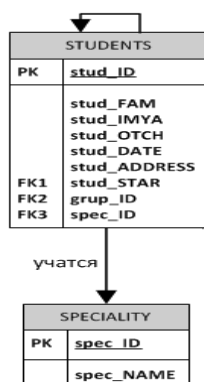


Рисунок 1.2 – ER–диаграмма бизнес-правила 1

Бизнес-правило 2.

Студент не может учиться в двух группах одновременно. Не может быть двух студентов с одинаковыми номерами зачетной книжки. Не существует групп без старост, следовательно, первый вводимый в группу студент – староста этой группы, что реализуется рекурсивной связью.

На основе бизнес-правила 2 мы получаем следующий сегмент ER-модели.

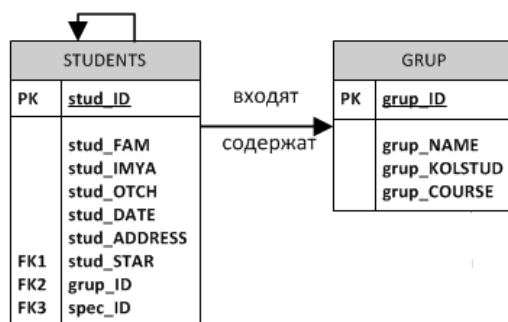


Рисунок 1.3 – ER–диаграмма бизнес-правила 2

Бизнес-правило 3.

Преподаватель не может работать на нескольких кафедрах, преподаватель числится только на одной кафедре в данный момент времени. Тем временем на кафедре работают много преподавателей.

На основе бизнес-правила 3 мы получаем следующий сегмент ER-модели

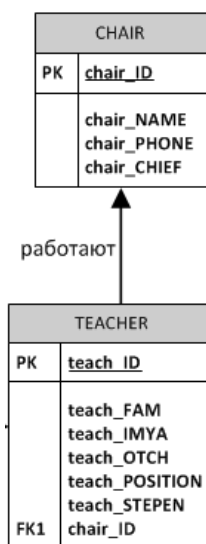


Рисунок 1.4 – ER–диаграмма бизнес-правила 3

Бизнес-правило 4.

Каждая группа (соответственно и студент в ней) изучает конкретное множество предметов, которые ведут преподаватели. Преподаватель может вести один и тот же предмет в нескольких группах и несколько разных предметов изучаются одной группой. Связь «многие-ко-многим» реализуем через ассоциативную таблицу, так как на сегодня ни одна СУБД такую связь не реализует. Сотрудники кафедры ведут занятия у нескольких групп, для этого таблицу *Учебный процесс* связываем с таблицей *Преподаватели* связью 1:M. Таким образом появляется еще одна таблица *Учебный процесс*.

Учебный процесс (STUDY). Сущность включает атрибуты - номер группы, номер предмета, номер преподавателя, форма обучения, количество часов. Атрибут Grup_ID отображает в формате integer группу, участвующую в учебном процессе. Атрибут Subj_ID отображает в формате integer предмет, относящийся к данной группе. Атрибут Teach_ID – это целочисленное поле, хранящее информацию о преподавателе, ведущем данный предмет. Атрибут Kredit_count – символьное поле, характеризующее форму обучения. Атрибут Total_Hours является целочисленным типом данных для отображения общего количества часов. Атрибут Lektion_Hours является целочисленным типом данных для отображения количества лекционных часов. Атрибут Practice_Hours является целочисленным типом данных для отображения количества часов практикума. Атрибут Labor_Hours является целочисленным типом данных для отображения количества лабораторных часов.

На основе бизнес-правила 4 мы получаем следующий сегмент ER-модели.

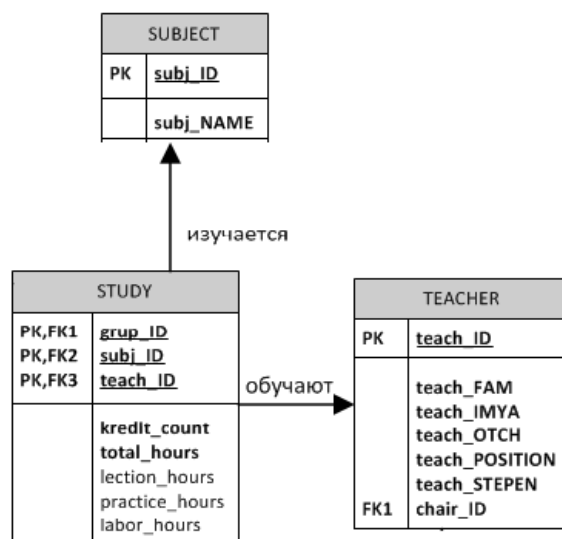


Рисунок 1.5 – ER–диаграмма бизнес-правила 4

Бизнес-правило 5.

Студенты сдают экзамены по предметам, которые они изучали. Например, если студент входит в группу 1, а этой группой в учебном процессе изучался предмет 2, то студент должен будет сдать данный предмет. Связь «многие-ко-многим» реализуется через ассоциативную таблицу *Успеваемость*.

Успеваемость (PROGRESS). Сущность включает атрибуты - код студента, код группы, код предмета, код преподавателя, дату сдачи, итоговую оценку. Данная сущность хранит данные об успеваемости каждого студента.

На основе бизнес-правила 5 мы получаем сегмент ER-Модели, представленный на рисунке 1.6.

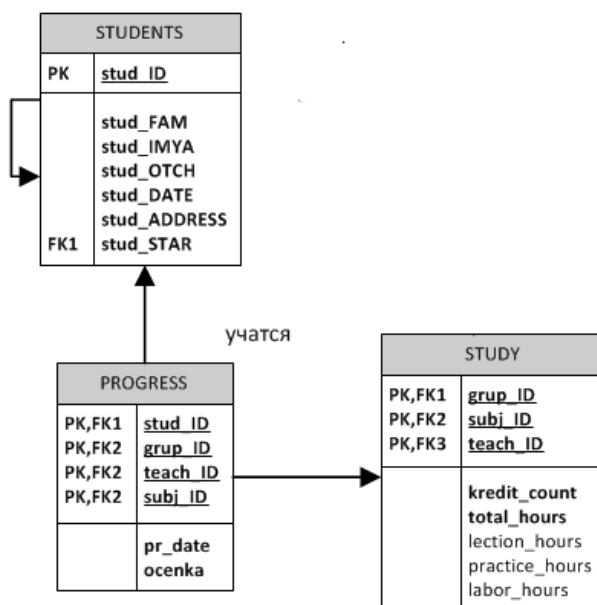


Рисунок 1.6 –ER–диаграмма бизнес-правила 5

База данных создаётся на основании схемы базы данных. Для преобразования ER–диаграммы в схему БД приведём уточнённую ER–диаграмму, содержащую атрибуты сущностей (рисунок 1.7).

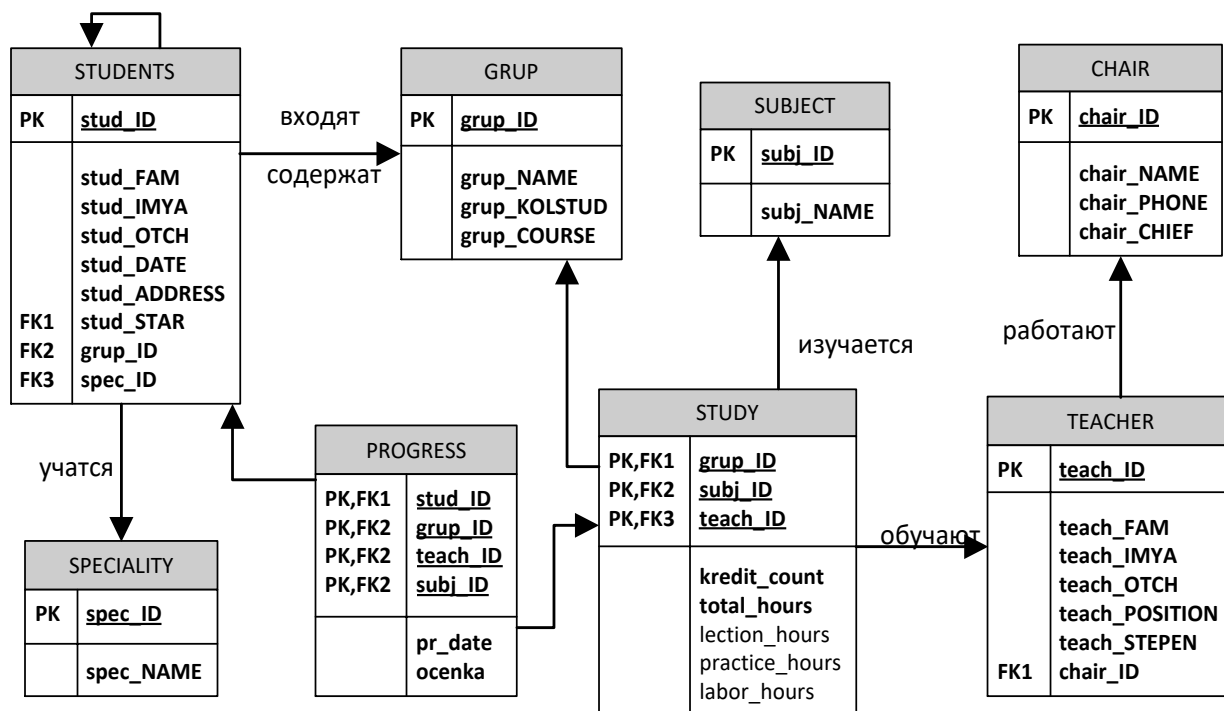


Рисунок 1.7 – Уточненная ER–диаграмма концептуальной модели учебного процесса

1.7 Преобразование концептуальной модели в реляционную модель

Концептуальная модель данных состоит из объектов, атрибутов, отношений, составных объектов. Каждая из этих конструкций должна быть преобразована в реляционные таблицы. Полученные в результате таблицы будут иметь четвертую нормальную форму. То есть дальнейшая нормализация модели не потребуется.

Преобразование объектных множеств и атрибутов. Объектное множество с атрибутами может быть преобразовано в реляционную таблицу с именем объектного множества в качестве имени таблицы и атрибутами объектного множества в качестве атрибутов таблицы. Если некоторый набор этих атрибутов может быть использован в качестве ключа таблицы, то он выбирается ключом таблицы. В противном случае, в таблицу добавляется атрибут, значения которого будут однозначно определять объекты-элементы исходного объектного множества, и который, таким образом, может служить ключом таблицы. Преобразуем объектные множества «ГРУППЫ», «КАФЕДРЫ», «ПРЕДМЕТЫ» в реляционные таблицы с соответствующими названиями.

Каждое реляционное отношение соответствует одной сущности, и в него вносятся все атрибуты сущности. Для каждого отношения необходимо определить окончательно первичный ключ и внешние ключи (если они имеются). Отношения приведены в таблицах 2-8.

Для каждого отношения указаны атрибуты с их внутренним названием, типом и длиной. Типы данных обозначаются как:

- N – числовой (numeric);
- C – символьный (char);
- D – дата (различная стандартная длина для каждой СУБД, поэтому она не указывается).

В полях примечание первичные и внешние ключи также подразумеваются обязательными полями (NOT NULL).

1.8 Создание модели базы данных Visio

Спроектируем концептуальную модель. Программа MS Visio предназначена для создания различного вида чертежей: от схем сетей до календарей, от планов офиса до блок-схем, а также структур баз данных.

Существует много типов документов Visio, но для создания практически всех документов можно воспользоваться тремя основными действиями:

- 1) Выбор и открытие шаблона.
- 2) Перетаскивание и соединение фигур.
- 3) Добавление текста (данных) в фигуры.

Для выбора и открытия шаблона необходимо выполнить следующие шаги:

- 1) Откройте программу Visio 2007.
- 2) В списке *Категории шаблонов* выберите элемент Программное обеспечение и базы данных.
- 3) В диалоговом окне ПО и БД в области *Готовые шаблоны* дважды щелкните элемент *Схема модели* базы данных.

После открытия шаблона будут открыты необходимые коллекции фигур, которые называются наборами элементов. Наборы элементов, которые открываются с шаблоном *Схема модели* базы данных, называются *Отношение сущности*. Перетащите первую фигуру (сущность, (Entity)) из набора элементов *Отношение сущности* на страницу документа и отпустите кнопку мыши.

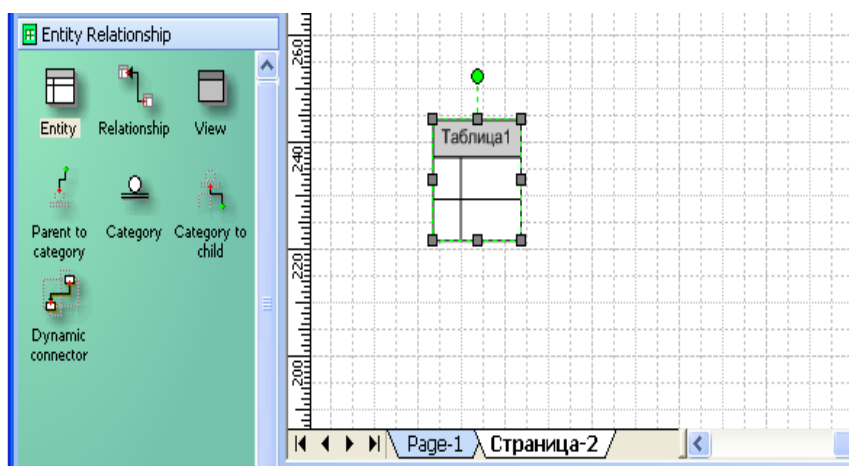


Рисунок 1.8 – Создание сущности (таблицы). Наборы элементов для работы с моделью базы данных в Visio

Фигуры Visio — это гораздо больше, чем просто изображения или символы.

Задайте свойства базы данных. Для этого щелкните по созданной фигуре (таблица 1). Соответствующее окно (свойства базы данных, рисунок 1.10) будет открыто в нижней части экрана.

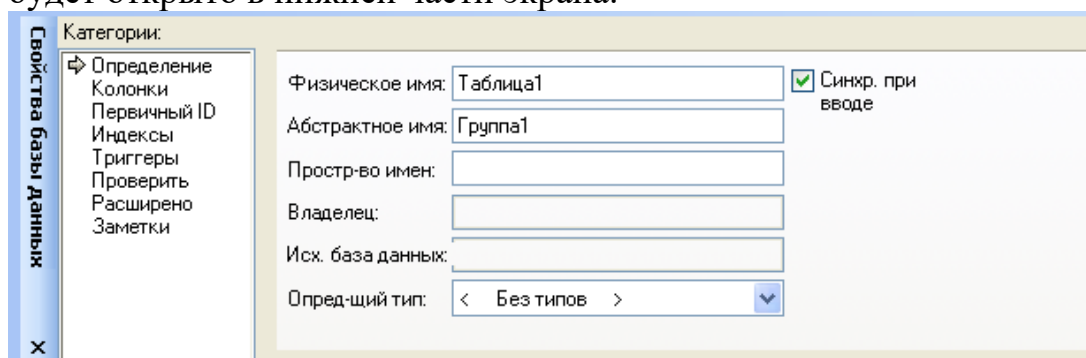


Рисунок 1.9 – Свойства базы данных

В категории *Определение* введите *физическое имя сущности* (Студенты). Затем перейдите в категорию *Колонки* и введите данные, соответствующие этой сущности.

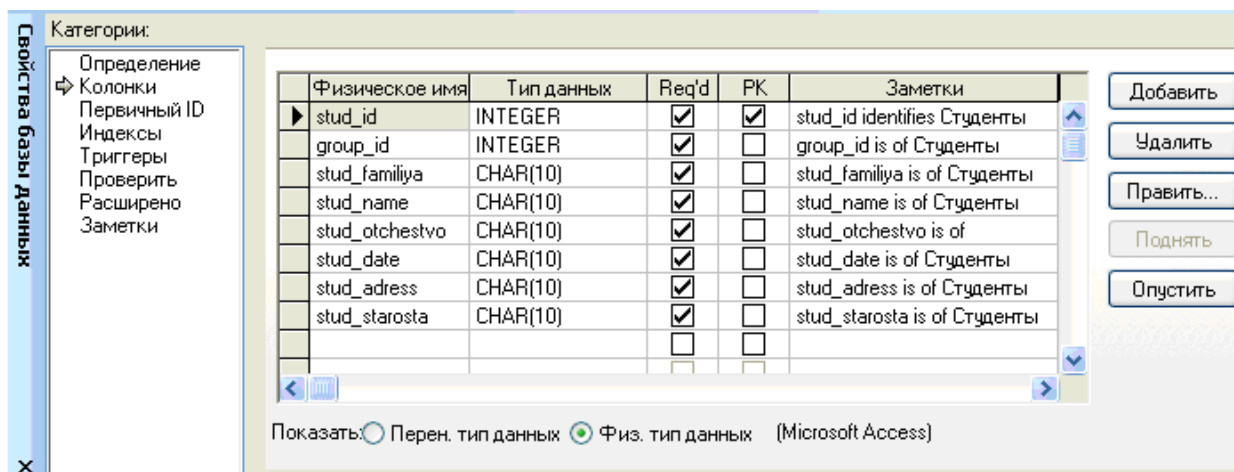


Рисунок 1.10 – Добавление атрибутов таблицы

В результате получается таблица, представленная на рисунке 1.12.

Студенты	
PK	<u>stud_id</u>
	group_id
	stud_familiya
	stud_name
	stud_otchestvo
	stud_date
	stud_adress
	stud_starosta

Рисунок 1.11 – Результат создания таблицы «Студенты»

Подобным образом необходимо создать все сущности базы данных «Учебный процесс», описанные в пункте 1.6.

После того, как создание всех сущностей завершено, необходимо создать отношения между таблицами. Для этого необходимо перетащить фигуру *Отношение* из набора элементов *Отношение сущности* (рисунок 1.13).

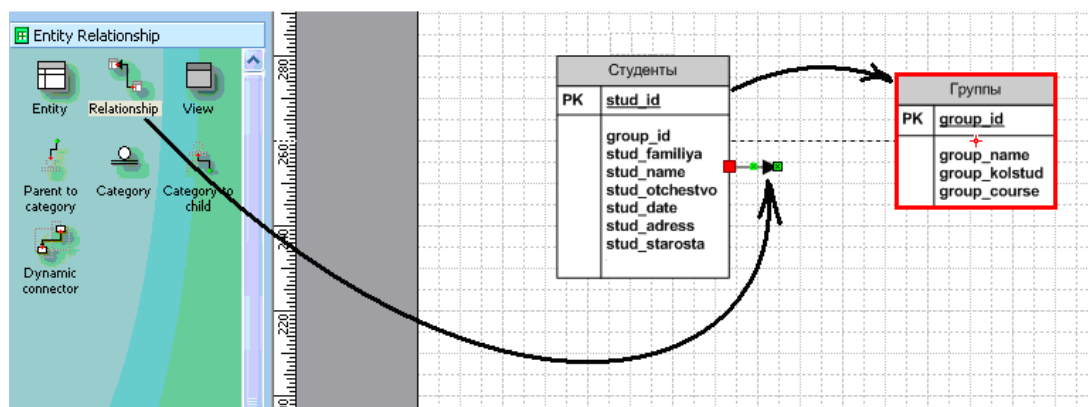


Рисунок 1.12 – Создание отношения

С помощью фигуры «отношение» необходимо соединить соответствующие таблицы, не забывая о том, что стрелка должна быть

направлена от дочерней таблицы к родительской. После создания связей между таблицами Внешние ключи расставляются автоматически по соответствующим атрибутам.

Контрольные вопросы:

- 1) Модели данных: сетевая, иерархическая и реляционная.
- 2) По реляционной модели данных:
 - а) сущность- отношение - таблица;
 - б) экземпляр сущности – кортеж - строка таблицы;
 - в) свойство – атрибут – столбец.
- 3) Что такое концептуальная модель данных?
- 4) Что понимается под реляционной схемой базы данных?
- 5) Объясните смысл терминов:
 - а) первичный ключ; внешний ключ; их особенности и назначение;
 - б) потенциальный;
 - в) простой и составной ключ.
- 6) Родительская и дочерняя таблицы, как производится связь?
- 7) Бизнес – правила, нормализация.
- 8) Нормальные формы.

2 Реализация модели базы данных в среде СУБД Microsoft SQL Server

2.1 Общие сведения о среде СУБД Microsoft SQL Server

В этой лабораторной работе мы сосредоточимся на командах ANSI SQL, которые необходимы при создании таблиц и манипулировании ими. Но прежде ознакомимся с инструментом, на котором будет реализована наша база данных.

Microsoft SQL Server – это реляционная система управления базой данных (РСУБД). В реляционных базах данных данные хранятся в таблицах. Взаимосвязанные данные могут группироваться в таблицы, кроме того, могут быть установлены также и взаимоотношения между таблицами. Отсюда и произошло название реляционные – от английского слова *relational* (родственный, связанный отношениями, взаимозависимый). Пользователи получают доступ к данным на сервере через приложения, а администраторы, выполняя задачи конфигурирования, администрирования и поддержки базы данных, производят непосредственный доступ к серверу.

SQL Server является масштабируемой базой данных, это значит, что она может хранить значительные объемы данных и поддерживать работу многих пользователей, осуществляющих одновременный доступ к базе данных.

СУБД SQL Server может быть реализована либо как клиент-серверная система, либо как автономная "настольная" система. Тип проектируемой вами системы зависит от количества пользователей, которые должны одновременно

осуществлять доступ к базе данных, и от характера работ, которые должны выполняться.

2.2 Практическая реализация

Любое изучение не будет полным без практики, поэтому физически реализуем часть спроектированной модели данных учебного процесса в среде СУБД MS SQL Server. Создадим базу данных и таблицы *Преподаватели*, *Кафедры*, *Группы*, *Студенты*, *Предметы*, *Специальности*, *Учебный план* и *Успеваемость*. В лабораторной работе для объектов рассматриваемой базы данных применяются названия на английском языке. Это связано с требованиями используемого программного обеспечения. Поэтому в приведенных ниже примерах база данных называется *Lab_Study*. Для создания любого объекта SQL Server существует несколько способов, базирующихся на выполнении определенной команды.

Размещение пользовательских баз может меняться в зависимости от версии SQL и размещения Program Files. Выяснить место расположения пользовательских баз можно из основного окна программы, выбрав из контекстного меню Свойства (Properties) любой базы в списке Databases. Также в окне свойств на вкладке TransactionLog просмотрите место расположения журнала транзакций. Команды резервного копирования и восстановления базы данных тоже выбираются в контекстном меню: строка Все задачи (All Tasks), команды *BACKUP DATABASES* и *RESTORE DATABASES*.

2.2.1 Создание базы данных.

Физически база данных располагается в одном или нескольких файлах операционной системы. В одном файле операционной системы не может содержаться несколько баз данных. В этом файле хранятся такие объекты, как таблицы и индексы. Журнал транзакций – это рабочие области, которые SQL Server применяет для записи информации до и после выполнения транзакции. Эта информация может использоваться для отмены выполненной транзакции или для восстановления базы данных, если возникнет такая необходимость. В MS SQL Server журналы транзакций хранятся в отдельном файле, а не вместе с таблицами, как было в предыдущих версиях. Для создания базы данных с помощью Transact-SQL используется команда *CREATE DATABASE*. Полный синтаксис команды приводить не будем, т.к. он занимает несколько страниц.

```
CREATE DATABASE Lab_Study
ON PRIMARY
(NAME = education_data,
FILENAME='C:\Data\education_data.mdf', size = 6,
maxsize =25, filegrowth = 1 mb)
LOG ON
```

```
(NAME = education_log,  
FILENAME='C:\Data\education_log.ldf', size = 6,  
maxsize = 20, filegrowth = 1 mb);
```

Внимание: размещение базы и журнала транзакций - 'C:\...' - может меняться в зависимости от версии SQL и размещения Program Files.

Здесь:

- *Lab_Study* - имя создаваемой базы данных;
- *ON* определяет список файлов на диске, в которых будет храниться информация базы данных;
- *PRIMARY* определяет файл, содержащий логическое начало базы данных и системных таблиц. В базе данных может быть только один первичный (PRIMARY) файл. Если этот параметр пропущен, то первичным считается первый файл в списке;
- *LOG ON* определяет список файлов на диске, в которых будет храниться журнал транзакций. Если этот параметр не определен, то размер журнала транзакций будет составлять 25% от общего размера файлов данных;
- *education_data.mdf* определяет логическое имя, которое SQL Server будет использовать для обращения к файлу;
- *education_log.ldf* определяет логическое имя, которое SQL Server будет использовать для обращения к журналу транзакций;
- *FILENAME* задает параметры файла операционной системы (имя файла, который должен находиться на сервере, где установлен SQL Server, первоначальный и максимальный размеры базы данных и приращение для увеличения размера базы данных).

2.2.2 Создание схемы EmployeeSchema.

Для создания таблиц и манипулирования ими внутри базы данных необходимо ввести понятие схемы. Схема представляет собой группу связанных между собой объектов базы данных (например, таблиц и индексов). Обычно схема является собственностью пользователя или приложения. В одной базе данных может храниться несколько схем, принадлежащих разным пользователям и приложениям. Можно представлять себе схему как логически сгруппированные таблицы или логическую базу данных, что постоянно используется при проектировании баз данных больших компаний и OLTP - БД. Схемы полезны тем, что группируют таблицы по владельцам и обеспечивают первый уровень защиты, разрешая пользователю просматривать таблицы, принадлежащие только ему. Если используемая вами РСУБД соответствует стандартам ANSI, то схема БД создается командой:

```
CREATE SCHEMA AUTHORIZATION <creator>;
```

Поэтому если создателем является Ruslan, то команда будет выглядеть так:

```
CREATE SCHEMA AUTHORIZATION Ruslan;
```

Схемы и пользователи баз данных разделены – пользователи владеют схемами.

Для обращения к объектам, обычно используется двухсоставной идентификатор *database_name.object_name* (*Education.Students*).

Если используется схема, то используется трехсоставной идентификатор *database_name.schema_name.object_name*. В пределах базы данных это имя можно сократить до *schema.object*. Т.е., если в базе существуют модули Отдел кадров, Бухгалтерия, Деканат, то таблицы могут быть разнесены по модулям (схемам).

Перед созданием таблиц необходимо определить текущую базу данных с помощью следующей команды:

```
USE lab_Study;
```

Теперь все последующие команды будут выполняться именно в этой базе данных.

Создадим две схемы для обучения студентов *StudySchema* (таблицы *Группы*, *Студенты*, *Предметы*, *Учебный план* и *Успеваемость*) и данных деканата *DekanatSchema* (таблицы *Кафедры* и *Преподаватели*):

```
CREATE SCHEMA StudySchema
GO
CREATE SCHEMA DekanatSchema
GO
```

Выделите только что введенный текст и нажмите кнопку *Выполнить*. Убедитесь, что запрос завершен без ошибок.

2.2.3 Создание таблиц.

При создании таблицы (отношения) необходимо задать имя таблицы, имена столбцов (атрибутов) и типы данных столбцов. Имена столбцов должны быть уникальными для данной конкретной таблицы. Однако одно и то же имя столбца может использоваться в различных таблицах одной и той же базы данных. Для каждого столбца следует задать тип данных.

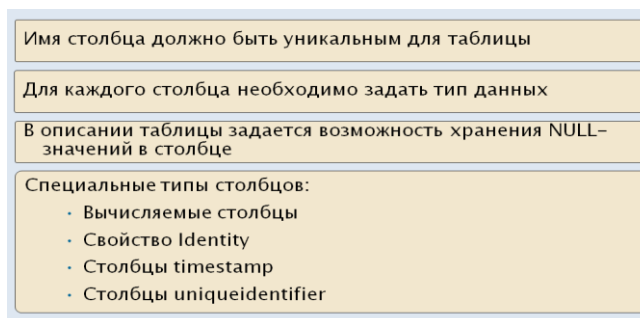


Рисунок 2.1 – Свойства отношения

Таблицу можно создать с помощью оператора *CREATE TABLE* языка SQL.

Синтаксис:

```
CREATE TABLE table_name
  ( { < column_definition > | < table_constraint > } [
    ,...n ]
  )
< column_definition > ::=
  { column_name data_type }
  [ { DEFAULT constant_expression
    | [ IDENTITY [ ( seed , increment ) ]
    ]
  } ]
  [ ROWGUIDCOL ]
  [ < column_constraint > [ ...n ] ]
< column_constraint > ::=
  [ CONSTRAINT constraint_name ]
  { [ NULL | NOT NULL ]
    | [ PRIMARY KEY | UNIQUE ]
    | REFERENCES ref_table [ ( ref_column ) ]
    [ ON DELETE { CASCADE | NO ACTION } ]
    [ ON UPDATE { CASCADE | NO ACTION } ]
  }
< table_constraint > ::=
  [ CONSTRAINT constraint_name ]
  { [ { PRIMARY KEY | UNIQUE }
    { ( column [ ,...n ] ) }
    ]
  | FOREIGN KEY
    ( column [ ,...n ] )
    REFERENCES ref_table [ ( ref_column [ ,...n ] ) ]
    [ ON DELETE { CASCADE | NO ACTION } ]
    [ ON UPDATE { CASCADE | NO ACTION } ]
  }
```

Замечания.

1) Аргументы и ограничения рассматриваются в справке оператора *CREATE TABLE*.

2) Определения столбца: при создании таблицы необходимо указать, по крайней мере, одно определение столбца.

Правила допустимости нулевого значения в рамках определения таблицы: допустимость нулевого значения столбца определяет, будет ли нулевое значение (*NULL*) принято в столбец как данные. *NULL* — это не ноль и не пробел. Это значит, что запись не была сделана или что было добавлено явное значение *NULL*, что обычно обозначает, что значение либо не известно, либо не применимо.

2.2.4 Виды ограничений ссылочной целостности.

При создании таблиц создаются ограничения, которые являются соответствующим стандарту ANSI способом обеспечения целостности данных. Каждый тип целостности данных — доменная, объектная и ссылочная — предполагает использование соответствующих типов ограничений. Ограничения обеспечивают ввод в столбцы допустимых значений и сохранение связей между таблицами.

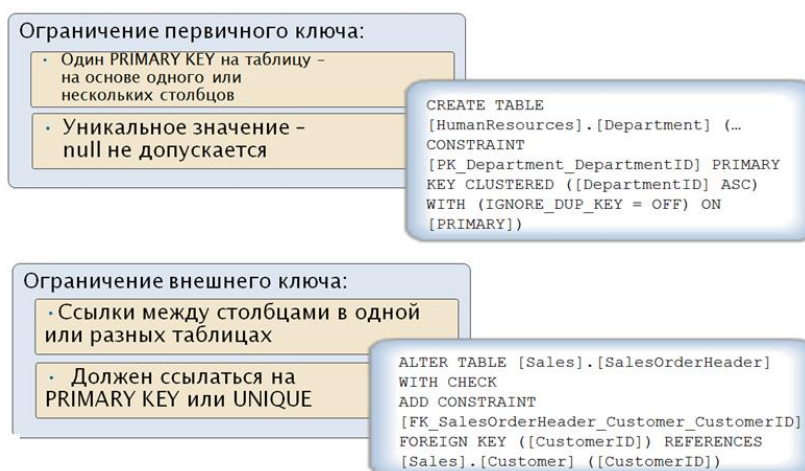


Рисунок 2.2 – Ограничения ключей

Ограничения на уровне столбца применяются к отдельному столбцу и являются частью определения столбца. Ограничения на уровне таблицы применяются к одному или нескольким столбцам таблицы. Ограничения на уровне таблицы указываются в отдельном разделе после определения всех столбцов.

Первичный ключ.

Ограничение *PRIMARY KEY* определяет один или несколько столбцов таблицы, которые образуют ключевое поле. Ключевое поле уникальным образом определяет строку таблицы и обеспечивает объектную целостность таблицы оно не может быть нулевым.

Первичный ключ - домен или несколько доменов таблицы, однозначно характеризующие запись согласно третьей нормальной форме Бойесса-Коде. Объявляется при помощи служебных слов *PRIMARY KEY*(< *FIELD1*> [, < *FIELDN*>]), где < *FIELD* > - имя поля или имена полей, перечисленные через запятую. Первичный ключ также может быть задан при создании домена таблицы. Например:

```
CREATE TABLE Student(  
    STUD_ID INTEGER PRIMARY KEY,  
    ...  
);
```

или (рекомендуемый вариант, с указанием имени *CONSTRAINT* - ограничение):

```
CREATE TABLE Student(  
    STUD_ID INTEGER NOT NULL,  
    ....  
    CONSTRAINT PK_Student PRIMARY KEY (STUD_ID)  
);
```

У таблицы может быть только один первичный ключ.

Первичный ключ также можно создать командой *ALTER TABLE*

Внешний ключ.

Внешний ключ представляет собой столбец или сочетание столбцов, которое используется для обеспечения связи между данными двух таблиц. Ограничение *FOREIGN KEY* обеспечивает ссылочную целостность и определяет ссылку на столбец с ограничением *PRIMARY KEY* или *UNIQUE* в той же или в другой таблице. Значения в столбце внешнего ключа должны отображаться в столбце ключевого поля. При наличии ссылок на значения ключевого поля эти значения не могут быть изменены или удалены.

Ограничение *FOREIGN KEY* имеет параметр *CASCADE*, с помощью которого можно распространять любое изменение значения столбца, определяющего ограничение *UNIQUE* или *PRIMARY KEY*, в любые значения внешнего ключа, ссылающиеся на это значение. Такое действие называется каскадной ссылочной целостностью.

Внешний ключ служит для связки родительской и дочерней таблиц в базе данных, когда одной записи в родительской таблице соответствует множество записей в дочерней таблице. На этом условии могут быть построены отношения сущностей в базе данных «один-к-одному» и «один-ко-многим». Внешний ключ строится по столбцам в дочерней таблице, значения которых ссылаются на значения записей в родительской таблице.

Внешний ключ также можно создать командой *ALTER TABLE*. Автоматические генераторы скриптов обычно создают описание Внешних ключей именно через *ALTER TABLE*, и располагают их после создания всех таблиц, т.к. это позволяет избежать проблем, когда Внешний ключ ссылается на таблицу, которая еще не создана скриптом.

Пример: (рекомендуемый вариант, с указанием имени constraint)

```
CREATE TABLE Student (  
    ...,  
    Grup_ID INTEGER NOT NULL,  
    ...,  
    CONSTRAINT FK_Student_Grup FOREIGN KEY (Grup_ID)  
        REFERENCES Grup(Grup_ID))
```

Конструкция *REFERENCES* <имя_мастер_таблицы> [<имя_поля>] используется для ограничения ссылочной целостности и определяет, что данный столбец является внешним ключом и указывает на ключ какой мастер-таблицы (родительской таблицы) он будет ссылаться.

Ссылку на мастер-таблицу можно делать в строке, описывающей поле, но можно использовать конструкцию *CONSTRAINT*, также используемую для работы с ограничениями ссылочной целостности, которые накладываются на таблицу или на отдельный столбец. Конструкция *CONSTRAINT* используется только в командах *CREATE TABLE* и *ALTER TABLE*.

Уникальный ключ.

Уникальный ключ - дополнительная возможность ограничения значений записей таблицы с целью профилактики занесения в нее двух и более записей, имеющих одинаковое значение указанного столбца (столбцов). В отличие от *PRIMARY KEY* количество уникальных ключей в таблице не ограничено (точнее, ограничено максимальной комбинаторной суммой вариантов комбинации имен доменов, входящих в таблицу). Уникальный ключ также называется «альтернативным», и чаще всего предназначен не для однозначной идентификации столбца, как Первичный ключ, а для указания столбца, который позволяет осуществить дополнительную идентификацию строки. Например, номер паспорта, код ИНН, номер пенсионной страховки, и т.д.

Другие ограничения.

DEFAULT	· Определяет значение столбца, если оно не задается · Только одно ограничение DEFAULT на столбец · Можно использовать некоторые системные функции
CHECK	· Ограничивает значение, которые можно задать в столбце · Можно использовать несколько CHECK для столбца · Может ссылаться на столбцы в той же таблице · Не может содержать подзапросы
UNIQUE	· Гарантирует, что каждое значение в столбце уникально · Только одно NULL-значение на столбец · Может включать один или несколько столбцов

Рисунок 2.3 – Ограничения атрибутов

Ограничение *DEFAULT* вводит значение в столбец, если оно не указано в инструкции *INSERT*. Обеспечивают доменную целостность данных.

Ограничение *CHECK* определяет диапазон значений, который пользователь может ввести в отдельном столбце с помощью инструкций *INSERT* и *UPDATE*. Проверочные ограничения устанавливаются на уровне столбца или таблицы. Ограничения *CHECK* на уровне столбца определяют диапазон значений, которые могут храниться в этом столбце. Ограничения *CHECK* уровня таблицы могут действовать в отношении нескольких столбцов

одной таблицы, что позволяет создавать перекрестные ссылки и сравнивать значения столбцов.

Ограничение *UNIQUE* указывает, что в двух строках одного столбца не могут находиться одинаковые значения. Ограничение *UNIQUE* полезно, если при наличии ключевого поля, такого как номер сотрудника, нужно гарантировать уникальность других идентификаторов, например, кода налогоплательщика для этого сотрудника. Например, обеспечим уникальность названия группы:

```
CREATE TABLE Grup (  
    Grup_ID          int identity (1,1) PRIMARY KEY,  
    Grup_Name        varchar (9) NOT NULL,  
    ..... ,  
    CONSTRAINT UK_Group UNIQUE (Grup_Name));
```

Правила допустимости нулевого значения в рамках определения таблицы: допустимость нулевого значения столбца определяет, будет ли нулевое значение (*NULL*) принято в столбец как данные. *NULL* — это не ноль и не пробел. Это значит, что запись не была сделана или что было добавлено явное значение *NULL*, что обычно обозначает, что значение либо не известно, либо не применимо.

2.2.5 Используемые типы данных.

Как и в языках программирования, в SQL существуют различные типы данных для хранения переменных:

1) Числа - для хранения числовых переменных (int, tinyint, smallint, bigint, numeric, decimal, money, smallmoney, float, real).

2) Даты - для хранения даты и времени (date, time datetime, smalldatetime).

3) Символы - для хранения символьных данных (char, nchar, varchar, nvarchar).

4) Двоичные - для хранения бинарных данных (binary, varbinary, bit).

5) Большиеобъемные - типы данных для хранения больших бинарных данных (text, ntext, image).

6) Специальные - указатели (cursor), 16-байтовое шестнадцатеричное число, которое используется для GUID (uniqueidentifier), штамп изменения строки (timestamp), версия строки (rowversion), таблицы (table).

Типы данных определяют тип информации (символы, числа или даты), которые могут содержаться в столбце, а также способ хранения этих данных. Сервер SQL Server 2008 предоставляет большой набор определенных системных типов данных, объединенных по категориям.

Системные типы данных Transact-SQL				
bigint	binary	bit	char	CLR
cursor	date	datetime	datetime2	datetimeoffset
decimal	float	hierarchyid	image	int
money	nchar	ntext	numeric	nvarchar
real	rowversion	smalldatetime	smallint	smallmoney
sql_variant	table	text	time	timestamp
tinyint	varbinary	varchar	uniqueidentifier	xml

Рисунок 2.4 – Типы данных

Замечания.

1) Для использования русских символов (не ASCII кодировки) используются типы данных с приставкой "n" (nchar, nvarchar, ntext), которые кодируют символы двумя байтами. Иначе говоря, для работы с Unicode используются типы данных с "n".

2) Для данных переменной длины используются типы данных с приставкой "var". Типы данных без приставки "var" имеют фиксированную длину области памяти, неиспользованная часть которой заполняется пробелами или нулями.

2.2.6 Креативная часть.

Для работы в заданной базе данных необходимо использовать команду:

USE имя_базы_данных;

Прежде чем создавать таблицу, рассмотрим спецификацию *identity (1,1)*.

Многие коммерческие продукты допускают использование автоинкрементируемых столбцов в таблицах, т.е. полей, значение которых формируется автоматически при добавлении новых записей. Такие столбцы широко используются в качестве первичных ключей таблицы, т.к. они автоматически обеспечивают уникальность. Типичным примером столбца такого типа является последовательный счетчик, который при вставке строки генерирует значение на единицу большее предыдущего значения (значения, полученного при вставке предыдущей строки).

Автоинкрементируемое поле определяется посредством конструкции *identity (1,1)*. При этом первый параметр свойства *identity (1)* определяет, с какого значения начнется отсчет, а второй - какой шаг будет использоваться для приращения значения. Таким образом, в нашем примере первая вставленная запись будет иметь в столбце *Spec_ID* значение 1, вторая - 2 и т.д.

Бывают случаи, когда требуется подставить вполне конкретное значение в автоинкрементируемое поле. Например, нужно перенести уже имеющиеся

данные во вновь создаваемую структуру; при этом эти данные участвуют в связи "один-ко-многим" со стороны "один". Таким образом, мы не можем допустить тут произвола. С другой стороны, нам не хочется отказываться от автоинкрементируемого поля, т.к. оно упростит обработку данных при последующей эксплуатации базы данных.

Таблицы создаются в схемах *StudySchema* (*Группы* и *Студенты*, *Предметы*, *Учебный план*, *Успеваемость*) и *DekanatSchema* (*Преподаватели*, *Кафедры*), таблица *Специальности* создается без схемы.

Создание таблицы *Специальности*:

```
CREATE TABLE Speciality (
    Spec_ID          int IDENTITY (1,1) PRIMARY KEY,
    Spec_NAME        VARCHAR (15) NOT NULL );
```

Создание таблицы *Группы*:

```
CREATE TABLE StudySchema.Grup (
    Grup_ID          int identity (1,1) PRIMARY KEY,
    Grup_NAME        varchar (9) NOT NULL,
    Grup_COURSE      int NOT NULL );
```

Так как поле *Grup_NAME*, определяющее название группы, не является первичным ключом, название группы может дублироваться, чтобы этого не происходило используется конструкция *UNIQUE*, которая используется для запрета совпадения значений в поле *Grup_NAME*, т.е значения каждого элемента столбца должно быть уникальным. Конструкция *UNIQUE* работает также как уникальный ключ

Так как слово *GROUP* является зарезервированным (часть конструкции *GROUP BY*), чтобы использовать его в качестве названия, необходимо всегда брать его в квадратные скобки [] либо не использовать это служебное слово в названиях элементов БД. Поэтому наша таблица называется *GRUP*.

Создание таблицы *Студенты* (здесь приведены варианты описания ключей):

```
CREATE TABLE StudySchema.Students (
    Stud_ID          bigint,
    Stud_FAM         char(20) NOT NULL,
    Stud_IMYA        char(10),
    Stud_OTCH        char(15),
    Stud_DATE        date NOT NULL,
    Stud_ADDRESS     char(25) NOT NULL,
    Spec_ID          int NOT NULL REFERENCES Speciality(Spec_ID),
    Grup_ID          int NOT NULL REFERENCES
                                StudySchema.Grup(Grup_ID),
    Stud_STAR        bigint,
    CONSTRAINT PK_Students PRIMARY KEY (Stud_ID),
    CONSTRAINT FK_Students_Students FOREIGN KEY (Stud_STAR)
    REFERENCES StudySchema.Students (Stud_ID) );
```

Конструкция *REFERENCES* <имя_мастер_таблицы> [<имя_поля>] используется для ограничения ссылочной целостности и определяет, что данный столбец является внешним ключом и указывает на ключ какой мастер-таблицы (родительской таблицы) он будет ссылаться.

Ссылку на мастер-таблицу можно делать в строке, описывающей поле, но можно использовать конструкцию *CONSTRAINT*, также используемую для работы с ограничениями ссылочной целостности, которые накладываются на таблицу или на отдельный столбец. Конструкция *CONSTRAINT* используется только в операторах *CREATE TABLE* и *ALTER TABLE*.

Создание таблицы Кафедры:

```
CREATE TABLE DekanatSchema.Chair (  
    Chair_ID      int    PRIMARY KEY,  
    Chair_NAME    varchar(20) NOT NULL,  
    Chair_PHONE   varchar (10) ,  
    Chair_CHIEF   varchar (15) ) ;
```

Создание таблицы Преподаватели:

```
CREATE TABLE DekanatSchema.Teacher (  
    Teach_ID      bigint not null PRIMARY KEY ,  
    Teach_FAM     varchar (20) NOT NULL,  
    Teach_IMYA    varchar (10) ,  
    Teach_OTCH    varchar (15) ,  
    Teach_POSITION varchar (18) ,  
    Teach_STEPEN  varchar (12) ,  
    Chair_ID      int NOT NULL ) ;
```

Создание таблицы Предметы:

```
CREATE TABLE StudySchema.Subjects (  
    Subj_ID       int    PRIMARY KEY ,  
    Subj_NAME     varchar(20) NOT NULL,  
    Total_Hours   int ,  
    Lektion_Hours int ,  
    Practice_Hours int ,  
    Labor_Hours   int,   ) ;
```

Создание таблицы Учебный план:

```
CREATE TABLE StudySchema.Study (  
    Grup_ID       int NOT NULL REFERENCES  
    StudySchema.Grup(Grup_ID ) ON DELETE CASCADE ,  
    Subj_ID       int NOT NULL REFERENCES  
    StudySchema.Subjects (Subj_ID) ,  
    Teach_ID      bigint NOT NULL REFERENCES  
    DekanatSchema.Teacher (Teach_ID) ,  
    Kredit_count  int ,  
    Lesson_Hours  int not null,  
    CONSTRAINT PK_Study PRIMARY KEY (Grup_ID, Subj_ID,  
    Teach_ID) ) ;
```

Создание таблицы Успеваемость:

```
CREATE TABLE StudySchema.Progress (
  Stud_ID          bigint          not null FOREIGN KEY REFERENCES
                                     StudySchema.Students (Stud_ID),
  Grup_ID          int             not null,
  Subj_ID          int             not null,
  Teach_ID         bigint          not null,
  Kredit_count     int             FOREIGN KEY REFERENCES
                                     StudySchema.Study (Kredit_count)
  Pr_DATE          datetime        null,
  OCENKA           integer         CHECK ( OCENKA in
                                     (0,1,2,3,4,5,6,7,8,9)) DEFAULT(0),
  CONSTRAINT FK_Progress_Study FOREIGN KEY (Grup_ID,
  Subj_ID,          Teach_ID ) REFERENCES StudySchema.Study
  (Grup_ID, Subj_ID, Teach_ID ),
  CONSTRAINT PK_Progress PRIMARY KEY (Stud_ID,Grup_ID,
  Subj_ID,   Teach_ID ) );
```

Сжатие данных.

В SQL Server 2008 появилась новая возможность сжатия данных для уменьшения размеров таблиц, индексов или их секций. Для этого данные типов с фиксированной длиной хранятся в формате переменной длины, и уменьшается избыточность данных.

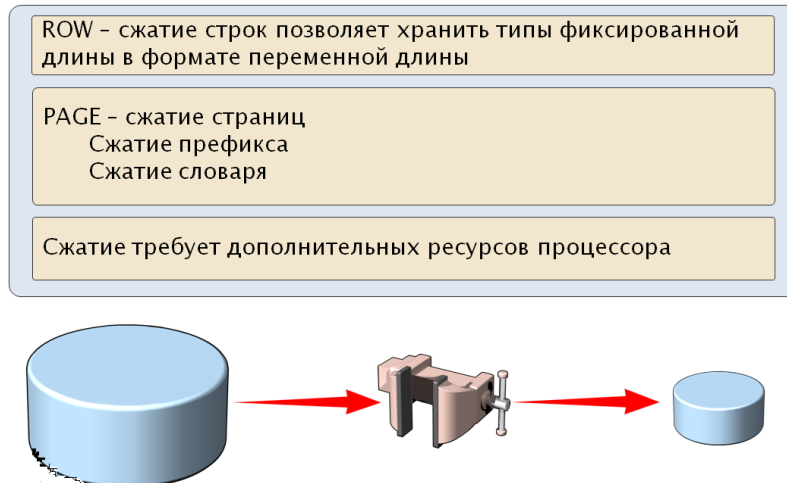


Рисунок 2.5 – Виды сжатия данных

Рассмотрим примеры создания полей таблиц для хранения подобных данных:

```
1
CREATE TABLE Tab1
  (col1 int, col2 int, sum_col AS col1+col2 );
2
CREATE TABLE Tab2
  (col1 int PRIMARY KEY,
```

```

col2 nvarchar(200) SPARSE NULL ) ;
3
CREATE TABLE Tab3
(col1 int, col2 nvarchar(150) )
WITH (DATA_COMPRESSION = ROW);

```

2.2.7 Изменение структуры таблиц.

Структуру таблиц можно изменять командой ALTER TABLE.

Добавление полей. Добавление в таблицу DekanatSchema.Chair поле DekanatSchema.Chair_Cab:

```

ALTER TABLE DekanatSchema.Chair
ADD Chair_Cab char(10) not null;

```

Удаление полей из таблицы. Удалите введенное поле из таблицы:

```

ALTER TABLE DekanatSchema.Chair
DROP COLUMN Chair_Cab ;

```

Добавьте в таблицу StudySchema.Students поле, где будет храниться информация о стипендии студентов:

```

ALTER TABLE StudySchema.Students ADD stud_stip int ;

```

Добавление ограничений. Если в таблице не были определены первичные или внешние ключи, это также можно исправить с помощью ALTER TABLE.

Предположим, что в таблице DekanatSchema.Chair не был определен первичный ключ:

```

ALTER TABLE DekanatSchema.Chair
ADD CONSTRAINT PK_Chair PRIMARY KEY (Chair_ID)

```

В таблице DekanatSchema.Teacher (*Преподаватели*) не был описан один из внешних ключей, добавим его:

```

ALTER TABLE DekanatSchema.Teacher
ADD CONSTRAINT fk_Chair_Teacher FOREIGN KEY
(Chair_ID) REFERENCES DekanatSchema.Chair (Chair_ID) ;

```

2.2.8 Удаление таблиц.

Для удаления таблиц (или объектов) предназначена команда

```

DROP <object type>.

```

```

DROP TABLE DekanatSchema.Teacher

```

Будьте осторожны с командой DROP.

Контрольные вопросы:

- 1) Что означает аббревиатура SQL?
- 2) Каковы главные отличия технологии клиент/сервер от технологии, использующей мэйнфрейм?
- 3) В рамках технологии клиент/сервер персональный компьютер является клиентом или сервером?
- 4) Какие типы данных допустимы при создании таблицы?
- 5) Как средствами меню программы MS SQL Server 2008:
 - создать базу данных 2-мя способами;
 - создать таблицы 3-мя способами;
 - заполнить и просмотреть таблицу;
 - изменить структуру таблицы;
 - создать автоматическое изменение первичного ключа (identity);
 - создать, разорвать связь между таблицами.
- 6) Как выполнить п.5 средствами языка SQL?
- 7) Как выполнить создание таблицы средствами языка SQL?
- 8) Каким образом выполнить простейшие операции вставки строк данных в таблицу средствами SQL?
- 9) Каким образом выполнить простейшие операции модификации строк таблицы средствами SQL?
- 10) Объясните смысл терминов: первичный ключ; внешний ключ; составной ключ; реляционная таблица; нормализация.
- 11) Создание первичного и вторичного ключа с помощью:
 - средствами меню программы MS SQL Server 2008;
 - средствами языка SQL.

3 Манипуляция над данными

3.1 Команды манипуляции данными

Добавление новых строк в таблицу.

Синтаксис

```
INSERT [INTO] table_name [ ( column_list ) ]  
    { VALUES ( { DEFAULT | NULL | expression } [ ,...n ] )  
    | derived_table  
    }
```

Примечания:

- 1) Чтобы заменить данные в таблице, необходимо использовать инструкцию *DELETE* для очистки существующих данных перед загрузкой новых данных с помощью *INSERT*. Чтобы изменить значения столбца в существующей строке воспользуйтесь инструкцией *UPDATE*.

2) Если вставка *column_list* пропущена, список вставки столбцов, в котором указываются все столбцы таблицы в возрастающем порядке в соответствии с их порядковыми номерами, становится неявным.

3) В *column_list* столбец таблицы можно определить только один раз. Если в *column_list* столбец отсутствует, SQL Server должен предоставить значение на основании определения столбца; в противном случае строку загрузить не удастся. SQL Server автоматически предоставляет значение для столбца в столбце:

- имеется свойство *identity*. Используется следующее значение приращения для идентификатора;
- имеется стандартное значение. Используется стандартное значение для столбца;
- неопределенное значение. Используется значение *NULL*.

При вставке явных значений в столбец идентификаторов необходимо использовать список столбцов и список *VALUES*. Если в списке *VALUES* значения не расположены аналогично столбцам таблицы или отсутствуют значения для каждого столбца таблицы, необходимо использовать *column_list* для явного указания столбца, в котором хранится каждое входящее значение.

Если для указания значения столбца используется значение *DEFAULT*, то для этого столбца вставляется стандартное значение. Если стандартного значения для столбца не существует и в столбце могут быть значения *Null*, то вставляется значение *NULL*. Значение *DEFAULT* недопустимо для столбца идентификаторов.

При вставке строк применяются следующие правила:

- если значение загружается в столбец с типом данных *char*, *varchar* или *varbinary*, добавление или усекание замыкающих пробелов (пробелов для *char* и *varchar*, нулей для *varbinary*) определяется в соответствии с приведенной ниже таблицей:

Тип данных	Стандартная операция
<i>char/binary</i>	Добавление исходного значения с замыкающими пробелами для столбцов <i>char</i> или с замыкающими нулями для столбцов <i>binary</i> к длине столбца
<i>varchar</i>	Замыкающие пробелы в значениях символов, которые вставлены в столбцы <i>varchar</i> , не усекаются. Значения не добавляются к длине столбца
<i>varbinary</i>	Замыкающие нули в двоичных значениях, которые вставлены в столбцы <i>varbinary</i> , не усекаются. Значения не добавляются к длине столбца

- если инструкция *INSERT* нарушает константу или правило, либо в ней присутствует значение, несовместимое с типом данных столбца, при

выполнении инструкции происходит сбой и отображается сообщение об ошибке;

– если *INSERT* загружает несколько строк с помощью *SELECT*, любое нарушение правила или ограничения, возникающее в результате загрузки значений, приводит к остановке полной инструкции и строки не загружаются.

Примеры кода:

1) Использование простой инструкции *INSERT*.

```
INSERT INTO DekanatSchema.Chair (Chair_ID, Chair_NAME,  
Chair_PHONE, Chair_CHIEF)  
VALUES (50763, 'Комп.технологии', '123456',  
'Куралбаев З.К. ');
```

Создание записи «преподаватели». Так как значения для всех столбцов предоставлены и перечислены в том же порядке, что и столбцы в таблице, то не нужно в параметре *column_list*. указывать имена столбцов:

```
INSERT INTO DekanatSchema.Teacher  
VALUES ('Ахметова', 'Галия ', 'Сериковна', ' ', '  
' ,50763,77512);
```

Создание нескольких записей в одной команде с использованием функции *GETDATE ()*:

```
INSERT INTO StudySchema.Progress (Stud_ID, Grup_ID, Subj_ID,  
Teach_ID, Pr_DATE, OCENKA)  
VALUES (1, 1, 2, 1, GETDATE(), 9),  
(2, 1, 1, 1, GETDATE(), 8),  
(3, 2, 2, 3, GETDATE(), 9);
```

Вставка данных в порядке, отличном от порядка столбцов таблицы

```
INSERT INTO DekanatSchema.Chair (Chair_NAME, Chair_ID,  
Chair_CHIEF, Chair_PHONE)  
VALUES ('Комп.технологии', 50763, 'Куралбаев З.К.  
' , '123456');
```

2) Оператор ввода данных позволяет ввести сразу множество строк, если их можно выбрать из некоторой другой таблицы:

```
INSERT INTO имя_таблицы [(<список столбцов>)]  
SELECT <список полей> FROM ....
```

Допустим, что у нас есть таблица со студентами и в ней указаны основные данные о студентах: их фамилии, адреса, домашние телефоны и даты рождения.

Создадим таблицу «Читатель» при помощи оператора *CREATE TABLE*:

```
CREATE TABLE READER
(ID_TICKET  bigint,
R_Fam      char(20) NOT NULL,
R_Name     char(10),
R_OTCH     char(15),
R_GRUP     int,
R_YEAR     datetime )
```

Тогда мы можем сделать всех студентов читателями нашей библиотеки одним оператором:

```
INSERT INTO READER (ID_TICKET, R_Fam , R_Name,
R_OTCH,R_GRUP, R_YEAR )
SELECT Stud_ID as ID_TICKET,STUD_Fam as R_Fam,STUD_IMYA as
R_Name, STUD_Otch as R_OTCH, Grup_Name as GRUP,
YEAR(Stud_Date) AS R_YEAR
FROM StudySchema.Students INNER JOIN StudySchema.Grup
ON StudySchema.Students.Group_Id=
StudySchema.Grup.Group_ID
```

В последнем случае, в таблицу может вставиться более одной записи. Если в таблице есть другие поля требующие заполнения, но не указанные в операторе *INSERT*, для них будет установлено значение по умолчанию, либо *NULL*, если значение по умолчанию не указано.

Так как стандарт языка SQL не предполагает наличия автоинкрементируемых полей, то соответственно не существует и единого подхода. Здесь мы покажем, как это реализуется в MS SQL Server. Оператор

```
SET IDENTITY_INSERT < имя таблицы > { ON | OFF };
```

отключает (значение *ON*) или включает (*OFF*) использование автоинкремента. Поэтому чтобы вставить строку со значением 15 в поле code, нужно написать

```
SET IDENTITY_INSERT Chair ON;
INSERT INTO Chair (Chair_ID,Chair_NAME,Chair_PHONE,
Chair_CHIEF)
VALUES (50766, 'Инж.Графики', '322223', 'Бериков А.К.');
```

3.2 Удаление строк из таблиц

Это еще одна операция, которую необходимо уметь выполнять для поддержки базы данных. Для удаления строк из таблицы используется команда *DELETE*.

Синтаксис.

```
DELETE
[ FROM ] table_name
[ WHERE < search_condition > ]
```

Замечание: аргументы и ограничения рассматриваются в справке оператора *DELETE*.

Примечания:

1) Если предложение *WHERE* не указано, *DELETE* удаляет все строки из таблицы.

2) Если указаны условия поиска, они применяется к каждой строке таблицы. Все строки, результат условий поиска для которых — *TRUE*, помечаются для удаления.

3) До проведения удалений для каждой строки в таблице проводится оценка условий поиска.

4) Все строки, помеченные для удаления, удаляются в конце выполнения инструкции *DELETE* перед проверкой ограничения целостности.

5) Сбой инструкции *DELETE* может произойти, если она нарушит ограничение *FOREIGN KEY*. Если инструкция *DELETE* удаляет несколько строк и одна из них нарушает ограничение, происходит отмена инструкции и возврат ошибки, удаления строк не происходит.

Пример кода.

В следующем примере показано удаление всех строк из таблицы *Преподаватели*:

```
DELETE FROM Table DekanatSchema.Teacher
```

В следующем примере показано удаление всех строк из таблицы *Преподаватели*, для которых в поле *<Должность_преподавателя>* значится 'Ассистент':

```
DELETE FROM DekanatSchema.Teacher
WHERE (Teach_Position= 'Ассистент');
```

Удаление записей из нескольких таблиц.

При удалении можно задавать конструкции *JOIN* связывающие несколько страниц, аналогично тому, как связываются таблицы в запросах выборки *SELECT*:

```
DELETE <Имя записи для удаления>
FROM <Имя Таблицы1>
JOIN <Имя Таблицы2>
ON <условие объединения>;
```

Удаление с подзапросом

```
DELETE FROM StudySchema.Students
WHERE Stud_ID IN
(SELECT Stud_ID
FROM StudySchema.Progress
WHERE Оценка = 2 OR Оценка IS NULL
GROUP BY Stud_ID
HAVING COUNT(*) > 2 )
```

В отличие от файловых СУБД, SQL Server не помечает записи как удаленные, а удаляет их физически, то есть восстановлению они не подлежат.

3.3 Полная очистка всех строк

TRUNCATE — в языках, подобных SQL, DML - операция очистки всех строк в таблице. Логически эквивалентна операции *DELETE* без оператора *WHERE*, однако выполняется быстрее и требует меньших ресурсов системы.

```
TRUNCATE TABLE <Имя Таблицы>
```

Последствием выполнения такой команды является полное удаление всех строк таблицы *<Имя Таблицы>*.

Отличия от оператора *DELETE*.

Основные отличия операторов *TRUNCATE* и *DELETE*, которые могут присутствовать в различных реализациях СУБД:

1) Операция *TRUNCATE* не записывает в журнал событий удаление отдельных строк. Вследствие чего не может активировать триггеры.

2) После операции *TRUNCATE* для некоторых СУБД (например, Oracle) следует неявная операция *COMMIT*. Поэтому, удаленные в таблице записи нельзя восстановить операцией *ROLLBACK*. Но существуют и СУБД, в которых операция *TRUNCATE* может участвовать в транзакциях, например, Microsoft SQL Server.

3) Операция *DELETE* блокирует каждую строку, а *TRUNCATE* - всю таблицу.

4) Операция *TRUNCATE* не возвращает какого-то осмысленного значения (обычно возвращает 0) в отличие от *DELETE*, которая возвращает число удаленных строк.

5) Операция *TRUNCATE* в некоторых СУБД (например, MySQL или Microsoft SQL Server), сбрасывает значение счетчиков (для полей с *AUTOINCREMENT / IDENTITY*).

Вообще говоря, реализация оператора *TRUNCATE* может зависеть от выбора конкретной СУБД. Поэтому в каждом случае рекомендуется смотреть документацию выбранной системы.

3.4 Изменение значения поля

Для изменения существующих значений в столбцах таблицы используется инструкция *UPDATE*.

Синтаксис:

```
UPDATE table_name
  [WITH ( < table_hint > ) ]
SET
  { column_name = { expression | DEFAULT | NULL }} [ ,...n ]
  [ WHERE < search_condition > ]
```

Замечание: аргументы и ограничения рассматриваются в справке оператора *UPDATE*.

Примечания:

- столбцы идентификаторов обновлять нельзя;
- если не указано предложение *WHERE*, обновляются все строки в таблице;
- перед обновлением строк в предложении *WHERE* проводится оценка условий поиска для каждой строки в таблице;
- если обновление строки нарушает ограничение или правило, либо настройку значения *NULL* для столбца, а также если новое значение является несовместимым типом данных, происходит отмена инструкции и возврат ошибки без обновления записей;
- все столбцы *char* имеют определенную границу по правому краю, до которой их можно заполнять;
- из данных, добавленных в *varchar*, удаляются все замыкающие пробелы, за исключением строк, содержащих только пробелы. Такие строки усекаются до одной пустой строки.

Пример.

В следующем примере показано обновление таблицы преподавателей путем установления ученой степени преподавателю с порядковым номером 2:

```
UPDATE DekanatSchema.Teacher
SET Teach_Stepen = 'Профессор'
WHERE Teach_ID = 2;
```

В следующем примере будем считать наличие трех пятерок по сессии признаком повышенной стипендии:

```
UPDATE StudySchema.Students
SET Stud_Stip= 20000
WHERE Stud_ID IN
  (SELECT Stud_ID
```

```
FROM StudySchema.Progress
WHERE Ocenka>=8
GROUP BY Stud_ID
HAVING COUNT(*)=3)
```

Контрольные вопросы:

- 1) Какие команды манипуляции данными вы знаете?
- 2) Дайте определение целостности данных.
- 3) Сформулируйте правило целостности на уровне ссылок.
- 4) Что означает определение поля NOT NULL?
- 5) Если поле определено как NULL, значит ли это, что в это поле обязательно должно быть что-нибудь введено?
- 6) Что означает определение поля identity?

4 Простые запросы к базе данных.

4.1 Извлечение информации из таблиц

В стандарте SQL есть несколько базовых команд, которые классифицируются следующим образом (не полная классификация):

```
DML:
    SELECT
    INSERT
    DELETE
    UPDATE

DDL:
    CREATE
    ALTER
    DROP

DCL:
    GRANT
    REVOKE
    DENY (для Microsoft)
```

Зная эти команды, можно достаточно просто изучить язык работы с базами данных. С командами DDL создания и управления БД кратко мы познакомились в предыдущих лабораторных. Рассмотрим базовую команду создания запросов к БД – *SELECT*.

Запрос к базе данных представляет собой операцию выборки, которая сужает диапазон считываемой информации и ограничивает ее столбцами и строками, соответствующими заданным критериям.

Для выборки данных в Transact-SQL используется инструкция *SELECT*. Большинство реальных запросов SQL предназначено для выборки не всех, а определенных строк и столбцов таблиц.

Инструкция *SELECT*.

Извлекает строки из базы данных и позволяет выбирать одну или несколько строк или столбцов в одной или нескольких таблицах. Это основной структурный компонент SQL, используемый для отправки запросов. Инструкция *SELECT* не изменяет, не вставляет и не удаляет данные.

Синтаксис:

```
SELECT select_list
[ FROM table_source ]
[ WHERE search_condition ]
[ GROUP BY group_by_expression ]
[ HAVING search_condition ]
[ ORDER BY order_expression [ ASC | DESC ] ]
```

Замечания:

Инструкция *SELECT* описывает запрос к системе. Выполнение запроса не обновляет данные. Результатом запроса является таблица идентично структурированных строк, в каждой из которых содержится одинаковый набор столбцов. Инструкция *SELECT* однозначно определяет, какие столбцы будут существовать в этой таблице результатов и какие строки ее заполняют. Инструкция *SELECT* не сообщает системе последовательность выполнения запроса; система сама выполняет запрос оптимальным в данном случае способом, используя внутренний модуль оптимизации на основе сведений о затратах. Результат должен гарантированно отвечать описанной ниже канонической стратегии выполнения. Единственное различие может заключаться в порядке расположения строк в таблице, хотя любой другой порядок может быть задан предложением *ORDER BY*.

Стратегия выполнения:

1) Создайте объединение таблиц в предложении *FROM*. При использовании явного синтаксиса *JOIN* (рассматривается в теме 5) результат *JOIN* будет явным. Если в предложении *FROM* имеется список таблиц, разделенных запятыми, это неявное объединение таблиц векторным произведением.

2) При наличии предложения *WHERE* следует применять данные условия поиска к строкам, полученным после шага 1, и сохранять только те строки, которые удовлетворяют условию.

3) Если в предложении *SELECT* нет объединений, как нет и предложения *GROUP BY*, перейдите к шагу 7.

4) Если есть предложение *GROUP BY*, разделите строки, полученные после шага 2, на несколько групп так, чтобы у всех строк в каждой группе было одинаковое значение по всем столбцам группирования. Если предложения *GROUP BY* нет, поместите все строки в одну группу.

5) К каждой группе, полученной после шага 4, примените предложение *HAVING*, если таковое указано. Останутся только те группы, которые удовлетворяют предложению *HAVING*.

6) Для каждой группы, полученной после шага 5, создайте одну строку результата путем оценки списка выбора в предложении *SELECT* в данной группе.

7) Если в предложении *SELECT* содержится ключевое слово *DISTINCT*, удалите все повторяющиеся строки, полученные в результате шага 6.

8) Если есть предложение *ORDER BY*, выполните сортировку результатов шага 7, как указано выражением порядка:

SELECT_LIST – указывает столбцы, возвращаемые запросом.

В данной лабораторной работе рассмотрим аргументы и ограничения инструкции *SELECT*.

```
SELECT [ ALL | DISTINCT ] TOP (expression) < select_list >
< select_list > ::=
{ *
| { table_name | table_alias }. *
| { column_name | expression } [ [ AS ] column_alias ]
} [ , ... n ]
```

Стандарты SQL позволяют явным образом задавать имена столбцов результирующей таблицы, для чего применяется фраза *AS*.

Аргумент *column_alias* – альтернативное имя для замены имени столбца в наборе результатов запроса. Например, для столбца «quantity» может быть указан псевдоним «Quantity», «Quantity to Date» или «Qty». Кроме того, псевдонимы используются для указания имен для результатов выражений (*AS* можно не использовать), например, следующие два выражения дают одинаковый результат:

<pre>SELECT Stud_STIP AS 'Стипендия' FROM Students</pre>	<pre>SELECT Stud_STIP 'Стипендия' FROM Students</pre>
--	---

```
SELECT AVG(Stud_STIP) AS Средняя_стипендия
FROM StudySchema.Students
```

Примечание:

– *column_alias* можно использовать в предложении *ORDER BY*, но нельзя использовать в предложениях *WHERE*, *GROUP BY* и *HAVING*.

4.2 Задание таблиц в запросе

Предложение *FROM* используется для указания таблиц из которых извлекаются строки. Предложение *FROM* в MS SQL Server является обязательным.

Синтаксис и аргументы рассматриваются в справке команды:

```
[FROM { < table_source > } [ ,...n ]
      < table_source > ::=
      table_name [ [ AS ] table_alias ]
| < joined_table >
| <derived_table> [ [ AS ] table_alias ]
  < joined_table > ::=
    < table_source > < join_type > < table_source >
    ON    <search_condition >
| <table_source> CROSS JOIN <table_source>
| <left_table_source> { CROSS | OUTER } APPLY
    <right_table_source>
| ( < joined_table > )
  < join_type > ::=
  [ INNER | { { LEFT | RIGHT } [ OUTER ] } ] JOIN ]
    left_table_source::= table_source
    right_table_source::=table_source
```

Примеры. Следующий пример предоставляет больше сведений об использовании предложения *FROM*:

```
SELECT Teach_ID, Teach_FAM, Teach_IMYA
FROM DekanatSchema.Teacher
```

Использование простого предложения *FROM*:

```
SELECT Teach_ID, Teach_FAM, Teach_IMYA
FROM DekanatSchema.Teacher
ORDER BY Teach_ID
```

В следующей инструкции *SELECT* предложение *FROM* указывает, что вернуть надо все данные, которые находятся в таблице «Students»:

```
SELECT *
FROM StudySchema.Students
```

Использование звездочки (*) в инструкции *SELECT* возвращает все столбцы. Это избавляет от необходимости специально указывать их в запросе.

Задание столбцов. При работе с инструкцией *SELECT* следует помнить, что можно отделить данные по вертикали, ограничив количество возвращаемых столбцов или по горизонтали, когда задаются подходящие строки.

Столбцы со значениями, возвращаемыми из таблиц базы данных, указываются в виде списка сразу после ключевого слова *SELECT* и отделяются друг от друга запятыми:

```
SELECT  Stud_ID, Stud_FAM, Stud_IMA, Stud_ADDRESS
FROM    StudySchema.Students
```

В этой команде:

- *SELECT* - ключевое слово;
- *Stud_ID, Stud_FAM, Stud_IMA, Stud_ADDRESS* - список столбцов из таблицы, которые выбираются запросом. Любые другие столбцы, не указанные в этом списке, игнорируются;
- *FROM* - ключевое слово, после него перечисляются таблицы - источник данных.

Выбранные строки и столбцы таблиц базы данных всегда собираются во временную таблицу. В большинстве случаев эта таблица существует ровно столько времени, сколько нужно, чтобы данные были переданы запрашивающему их клиенту.

Столбцы таблицы можно отобразить в отличном от первоначального порядка (переставить):

```
SELECT  Stud_FAM, Stud_IMA, Stud_ID, Stud_ADDRESS
FROM    StudySchema.Students
```

Возможность изменения порядка отображения столбцов таблицы базы данных соответствует характеристикам реляционной базы данных, поскольку доступ к информации реляционной базы данных не зависит от реального способа хранения информации. Можно отображать один и тот же столбец таблицы в нескольких местах. Это удобно при необходимости улучшить восприятие таблицы.

Оператор *DISTINCT* - аргумент, который обеспечивает устранение двойных значений в команде *SELECT*. Предположим, необходимо узнать, какие студенты в настоящий момент имеют хотя бы по одной оценке (сами оценки пока не нужны, нужен только список кодов студентов):

```
SELECT  Stud_ID
FROM    StudySchema.Progress
```

При выполнении этой команды, коды дублируются (некоторые студенты имеют по несколько оценок). Для получения списка без дубликатов надо ввести:

```
SELECT DISTINCT Stud_ID
FROM    StudySchema.Progress
```

DISTINCT может указываться только один раз в команде *SELECT*. Если указать несколько столбцов после ключевого слова *DISTINCT*, то в результате будет четкая комбинация всех указанных колонок, где все выбранные поля идентичны.

Если вместо *DISTINCT* указать *ALL*, то получим противоположный эффект, дублирование строк сохранится. Поскольку это тот же самый случай, когда не указывается никакой аргумент, то *ALL*, по существу, лишь пояснительный аргумент.

4.3 Сортировка вывода

Обычно требуется, чтобы данные как-то были упорядочены. Упорядочение задается с помощью ключевого слова *ORDER BY* (по умолчанию упорядочение *по возрастанию*). Синтаксис предложения *ORDER BY*:

```
ORDER BY Столбец1 | Целое_значение [ASC | DESC]
```

Например, выведем список студентов, стипендию и вычисленную половину стипендии в алфавитном порядке:

```
SELECT Stud_ID, Stud_FAM, Stud_STIP, Stud_STIP / 2
FROM StudySchema.Students
ORDER BY Stud_FAM;
```

Если после имени столбца указать имя еще одного столбца, то по значениям второго столбца будут упорядочены строки, содержащие одинаковые значения в первом столбце. Столбец, указанный в списке ключевого слова *ORDER BY*, можно заменить числом, соответствующим порядку столбца в списке (параметр *Целое_значение*):

```
SELECT Grup_ID, Stud_ID, Stud_FAM
FROM StudySchema.Students
ORDER BY 1 DESC, 3;
```

Параметры *ASC* (*ascending*) – по возрастанию, *DESC* (*descending*) – по убыванию задают порядок сортировки.

Примечания:

1) Столбцы данных типа *text* и *image* нельзя использовать в предложении *ORDER BY*.

2) Параметр *ASC* указывает, что сортировку значений в выбранных столбцах следует выполнить в возрастающем порядке, от самого нижнего до самого верхнего.

3) Параметр *DESC* указывает, что сортировку значений в выбранных столбцах следует выполнить в убывающем порядке, от самого верхнего до

самого нижнего. Нулевые значения считаются самыми нижними допустимыми значениями.

4) В предложении *ORDER BY* нет ограничения по числу элементов.

При использовании предложения *ORDER BY* с инструкцией *UNION* сортируемые столбцы должны быть именами или псевдонимами столбцов, указанными в первой инструкции *SELECT*. Например, первая из следующих инструкций *SELECT* выполнится успешно, а во время выполнения второй произойдет сбой:

```
Create t1 (col1 int, col2 int);
Create t2 (col3 int, col4 int);
SELECT * from t1
UNION
SELECT * from t2
ORDER BY col1;
```

Данная инструкция успешно выполняется, поскольку имя «col1» принадлежит первой таблице (t1)

```
SELECT * from t1
UNION
SELECT * from t2
ORDER BY col3;
```

Данная инструкция дает сбой, поскольку имя «col3» не принадлежит первой таблице (t1).

4.4 Ограничение строк таблицы

Таблицы имеют тенденцию становиться очень большими. Поскольку обычно только некоторые из них интересуют Вас в данное время, имеется возможность устанавливать критерии, чтобы определить какие строки будут выбраны. Для этого используется ключевое слово *WHERE* - предложение команды *SELECT*, позволяющее задавать условие, которое может быть верным или неверным для любой строки таблицы. Команда извлекает только те строки из таблицы, для которых это условие верно. Синтаксис предложения *WHERE*:

WHERE имя_столбца оператор_сравнения значение

Инструкция *SELECT* с предложением *WHERE* сужает результирующий набор запроса. Она может вернуть всего одну строку или даже вообще не вернуть ничего, если строки, совпадающие с указанным в инструкции критерием, не найдены.

Предположим, необходимо получить имена и фамилии всех студентов с отчеством 'Николаевич':

```
SELECT    Stud_FAM, Stud_IMYA
FROM      StudySchema.Students
WHERE     Stud_OTCH = 'Николаевич';
```

Когда в запросе имеется предложение *WHERE*, SQL Server просматривает всю таблицу построчно и исследует каждую строку, чтобы определить верно ли утверждение.

Примечание: - Строковые константы типа 'Москва' в Transact-SQL ограничиваются либо апострофами ', либо кавычками ".

Теперь попробуем построить запрос с числовым полем в предложении *WHERE*. Выберем всех студентов со стипендией 1200:

```
SELECT *
FROM    StudySchema.Students
WHERE   Stud_STIP = 12000
```

В следующем примере показано использование предложения *WHERE* для получения общего количества стипендии, приходящегося на группу с номером 5:

```
SELECT SUM(Stud_STIP) AS 'Сумма стипендии'
FROM    StudySchema.Students
WHERE   Grup_ID=5
```

4.5 Операции в условиях для отбора данных

Transact-SQL позволяет строить сложные условия отбора записей и для этого использует операторы отношения, логические операторы и специальные операторы.

Операторы отношения:

- 1) = Равно.
- 2) > Больше чем.
- 3) < Меньше чем.
- 4) >= Больше или равно.
- 5) <= Меньше или равно.
- 6) <> Не равно.

Они имеют стандартные значения для символьных значений и дат.

Символьные значения сравниваются в терминах их кодов.

Предположим, что нам надо увидеть всех студентов со стипендией выше 12000:

```
SELECT *
FROM    StudySchema.Students
WHERE   Stud_STIP > 12000
```

Логические операторы:

- 1) *AND* логическое "И".
- 2) *OR* логическое "ИЛИ".
- 3) *NOT* логическое отрицание.

Оператор *AND* сравнивает два логических значения и возвращает TRUE (истина), если оба значения истинны (т.е. равны TRUE), в остальных случаях - FALSE (ложь). Оператор *OR* возвращает TRUE, если хотя бы один из аргументов равен TRUE. Оператор *NOT* возвращает TRUE, если его аргумент равен FALSE и наоборот.

Использование логических операторов значительно увеличивает возможности команды *SELECT*.

Логический оператор *AND* (логическое И) имеет смысл, если возвращаемые в результате выполнения запроса строки должны удовлетворять обоим условиям сравнения, заданным в предложении *WHERE*:

```
SELECT *  
FROM StudySchema.Students  
WHERE Stud_IMYA = 'Алексей' AND  
Stud_ADDRESS = 'Таугуль'
```

Если в таблице имеется две строки, в которых значение в столбце *Stud_IMA* равно 'Алексей' (то есть имеются тезки), то логический оператор *AND* позволяет выбрать ту строку, в которой значение столбца *Stud_ADDRESS* равно «Таугуль» (то есть того Алексея, который живет в Таугуле).

С помощью оператора *OR* можно выбрать несколько значений из одного столбца:

```
SELECT *  
FROM StudySchema.Students  
WHERE Stud_IMYA = 'Алексей' OR  
Stud_IMYA = 'Николай'
```

Оператор *NOT* указывается перед столбцом в операциях сравнения:

```
SELECT *  
FROM StudySchema.Students  
WHERE NOT Stud_ADDRESS = 'Таугуль'
```

Этот запрос позволяет считать из таблицы *StudySchema.Students* те строки, которые содержат информацию обо всех студентах, кроме тех, кто проживает в микрорайоне Таугуль.

Для группировки выражений Transact-SQL позволяет использовать круглые скобки ().

Например, два следующих запроса дадут различные результаты:

```

SELECT stud_fam, grup_id, stud_stip
FROM students
WHERE grup_id = 1 OR (stud_fam LIKE '%A%'
AND stud_stip >4500);

SELECT stud_fam, grup_id, stud_stip
FROM students
WHERE (grup_id = 1 OR stud_fam LIKE '%A%')
AND stud_stip >4500;

```

Распределительный закон гласит, что $A \text{ AND } (B \text{ OR } C)$ - то же самое, что и $(A \text{ AND } B) \text{ OR } (A \text{ AND } C)$. Опытным путем было установлено, что запрос вида «...*WHERE* column1 = 1 *AND* (column2 = 'A' *OR* column2 = 'B')» выполняется несколько быстрее, чем «...*WHERE* (column1 = 1 *AND* column2 = 'A') *OR* (column1 = 1 *AND* column2 = 'B')». Некоторые БД сами умеют оптимизировать запросы такого типа, но лучше перестраховаться.

4.6 Специальные операторы: IN, BETWEEN, LIKE, IS NULL

Оператор *IN* определяет список значений, в который должно входить значение поля. Например, если необходимо найти всех студентов с именем 'Алексей' и 'Николай', можно использовать такой запрос:

Набор значений для оператора *IN* заключается в круглые скобки, значения разделяются запятыми.

```

SELECT *
FROM StudySchema.Students
WHERE Stud_IMYA IN ('Алексей' , 'Николай')

```

Оператор *BETWEEN* похож на оператор *IN*. В отличие от списка допустимых значений, *BETWEEN* определяет диапазон значений. В запросе необходимо указать слово *BETWEEN*, затем начальное значение, ключевое слово *AND* и конечное значение. Первое значение должно быть меньше второго. Следующий запрос выберет студентов с оценками между 3 и 5:

```

SELECT *
FROM StudySchema.Progress
WHERE Ocenka BETWEEN 3 AND 5

```

Оператор *LIKE* применим только к символьным полям, с которыми он используется, чтобы находить подстроки. То есть, он ищет поле символа, чтобы видеть совпадает ли с условием часть его строки. В качестве условия он использует специальные символы:

- символ подчеркивания замещает любой одиночный символ. Например, 'к_т' будет соответствовать 'кот' и 'кит', но не 'крот';
- знак процента % замещает последовательность любого числа символов. Например, '%м%p' будет соответствовать 'компьютер' и 'омар'.

Пример. Выбрать студентов, чьи имена начинаются с буквы 'О':

```
SELECT *
FROM StudySchema.Students
WHERE Stud_IMYA LIKE 'O%'
```

Оператор *LIKE* удобен при поиске значений - можно использовать ту часть значения, которую помните.

Часто необходимо различать строки, содержащие значения *NULL* в каком-либо столбце. Так как *NULL* указывает на отсутствие значения, для сравнения с *NULL* существует специальный оператор - *IS NULL*. Выберем из нашей базы студентов с *NULL* значениями в столбце *Stud_STIP*:

```
SELECT *
FROM StudySchema.Students
WHERE Stud_STIP IS NULL
```

4.7 Использование стандартных функций

4.7.1 Математические функции.

Замечание: В качестве параметров функции будем указывать соответствующий им тип данных.

- *ABC (numeric)* – модуль числа;
- *ACOS/ASIN/ATAN (Float)* – арккосинус, арксинус, арктангенс в радианах;
- *COS/SIN/TAN/COT (Float)* – косинус, синус, тангенс, котангенс;
- *CEILING (Numeric)* – наименьшее целое, большее или равное параметру в скобках;
- *DEGREES (Numeric)* – преобразует радианы в градусы;
- *EXP(Float)* – экспонента, e^x ;
- *FLOOR(Numeric)* – наименьшее целое меньше или равное выражению numeric;
- *LOG(Float)* – натуральный логарифм $\ln$;
- *LOG10(Float)* – десятичный логарифм $\log_{10}$;
- *PI ()* – число пи;
- *POWER (Numeric,y)* – возводит выражение Numeric в степень y;
- *RADIANS (Numeric)* – преобразует градусы в радианы;
- *RAND ()* – генерирует случайное число типа данных Float, расположенное между нулем и единицей;

- *ROUND (Numeric, Длина)* – округляет выражение Numeric до заданной длины (количество знаков после запятой);
- *SIGN (Numeric)* – выводит знак числа +/- или ноль;
- *SQUARE (Float)* – вычисляет квадрат числа Float;
- *SQRT (Float)* – вычисляет квадратный корень числа Float;
- *%* – модуль от деления числа1 на число2.

Примеры использования математических функций:

```
SELECT ABC(-10); результат 10
SELECT SQRT (16); результат 4;
SELECT ROUND (125.85,0); результат 126
SELECT POWER (2,4); результат 16
SELECT 38 % 5; результат 3
```

4.7.2 Строковые функции.

Строковые функции позволяют производить операции с одной или несколькими строками:

- 'Строка1'+'Строка2' присоединяет Строку1 к Строке2;
- *ASCII(Char)* возвращает ASCII код с самого левого символа выражения Char;
- *CHAR(Int)* выводит символ, соответствующий ASCII коду в выражении Int;
- *CHARINDEX (Образец, Выражение)* выводит позицию Образца выражения, то есть где находится Образец в Выражении;
- *DIFFERENCE (Выражение1, Выражение2)* сравнивает два выражения, выводит числа от 0 до 4: 0 – выражения абсолютно различны; 4 – выражения абсолютно идентичны. Оба выражения типа данных Char;
- *LEFT (Char, Int)* выводит из строки Char Int символов слева;
- *RIGHT (Char, Int)* выводит из строки Char Int символов справа;
- *LTRIM(Char)* удаляет из строки Char пробелы слева;
- *RTRIM(Char)* удаляет из строки Char пробелы справа;
- *WCHAR(Int)* выводит выражение Int в формате Unicode;
- *REPLACE (Строка1, Строка2, Строка3)* меняет в Строке1 все элементы Строка2 на элементы Строка3;
- *REPLICATE(Char, Int)* повторяет строку Char Int раз;
- *REVERSE(Char)* производит инверсию строки Char, то есть располагает символы в обратном порядке;
- *SPACE(Int)* выводит Int пробелов;
- *STR(Float)* переводит число Float в строку;
- *STUFF (Выражение1, Начало, Длина, Выражение2)* удаляет из Выражения1 начиная с позиции символа Начало количество символов, равное параметру Длина, вместо них подставляет Выражение2;
- *SUBSTRING (Выражение, Начало, Длина)* из Выражения выводится строка заданной Длины начиная с позиции Начало;

- *UNICODE(Char)* выводит код в формате Unicode первого символа в строке Char;
- *LOWERC(Char)* переводит строку Char в маленькие буквы;
- *UPPER(Char)* переводит строку Char в заглавные буквы.

Примеры применения строковых функций:

```
SELECT ASCII('G'); - результат 71,
SELECT LOWER('ABC'); - результат abc,
SELECT Right('ABCDE',3'); - результат CDE,
SELECT REVERSE('МИР'); - результат РИМ.
```

Замечание. Во всех строковых функциях значения выражения типа Char заключаются в одинарные кавычки.

4.7.3 Функции дат.

В некоторых функциях дат используется так называемая часть дат, которая кодируется специальными символами:

- *dd* – число дней (от 1 до 31);
- *dy* – день года (число от 1 до 366);
- *hh* – значение часа (0-23);
- *ms* – значение миллисекунд (от 0 до 999);
- *ss* – значение секунд (0-59);
- *mi* – значение минут (0-59);
- *qq* – значение квартала (1-4);
- *mm* – значение месяцев (1-12);
- *wk, ww* – значение номеров недель в году;
- *dw* – значение дней недели, неделя начинается с воскресенья (1-7);
- *yy, yyy* – значение лет (1753 -999).

Функции дат предназначены для работы с датами или времени. Существуют несколько следующие функции дат:

- *DATEADD (часть, число, date)* добавляет к дате date часть даты увеличенное на число;
- *DATEDIFF (часть, date1, date2)* выводит количество частей даты между date1 и date2;
- *DATENAME (часть, date)* выводит символьное значение частей даты к заданной дате (название дней недели);
- *DATEPART (часть, date)* выводит числовое значение части даты из заданной даты (номер месяца);
- *DAY(date)* выводит количество дней в заданной дате;
- *MONTH (date)* выводит количество месяцев в заданной дате;
- *YEAR(date)* выводит количество лет в заданной дате;
- *GETDATE ()* выводит текущую дату, установленную на компьютере.

Замечание: даты выводятся в Американском формате: месяц/день/год.

Примеры функции работ с датами:

```
SELECT DATEADD(dd,5,11/20/15); - результат Nov/25/2015;  
SELECT DATEDIFF(dd,11/20/15, 11/25/15); - результат 5 дней;  
SELECT DATENAME(mm,11/20/15); - результат November;  
SELECT DATENAME(DW,11/20/15); - результат день недели (1  
вар.);  
SELECT DATENAME(WEEKDAY,GETDATE()); - результат день недели  
(2 вар.);  
SELECT DATEPART(mm, 11/20/15); - результат 11.
```

Пусть сегодня 23/01/2015, и мы хотим узнать, какой день будет через неделю. Мы можем написать:

```
SELECT DATEADD(day, 7, current_timestamp)
```

а можем и так:

```
SELECT DATEADD(ww, 1, current_timestamp)
```

Замечание. В выражениях оператора *SELECT* можно использовать операции сравнения. В результате будет либо истина TRUE, либо ложь FALSE. Можно использовать следующие операторы: =, <, >, >=, <=, <>, !<(не меньше), !>(не больше), !=(не равно). Приоритет операции задается круглыми скобками.

4.7.4 Конвертирование типов.

Язык SQL – интерпретируемый язык, он не типизирован и автоматически конвертирует данные из одного формата в другой, но есть ситуации (особенно в Transact-SQL) когда форматы нужно преобразовывать явно.

Функции для преобразования данных:

- *CAST (выражение AS data_type)* позволяет форматировать данные при преобразовании их к символьному типу и указывать формат при обратном преобразовании.

- *CONVERT (data_type [(length)], выражение [, style])* позволяет форматировать данные (например, темпоральные данные типа *datetime*) при преобразовании их к символьному типу и указывать формат при обратном преобразовании, *style* для указания стиля вывода даты.

Пример. Вывести среднюю стипендию с предваряющим текстом 'средняя стипендия = '.

Попытка выполнить запрос

```
SELECT 'Средняя стипендия = ' + AVG(STUD_Stip)
```

```
FROM StudySchema.Students;
```

приведет к сообщению об ошибке, но следующий запрос даст требуемый результат:

```
SELECT 'Средняя стипендия = ' + CAST(AVG(STUD_Stip)
AS CHAR(15))
FROM StudySchema.Students;
```

Мы использовали выражение явного преобразования типов *CAST* для приведения среднего значения стипендии к строковому представлению.

Если нас интересует результат с заданной точностью, скажем, до двух десятичных знаков? Применение выражения *CAST* к среднему значению ничего не даст.

```
SELECT CAST(AVG(STUD_Stip *1.0) AS NUMERIC(6,2))
FROM StudySchema.Students;
```

Аналогичные преобразования типа можно выполнить с помощью функции *CONVERT*:

```
SELECT CONVERT(NUMERIC(6,2), AVG(STUD_Stip *1.0))
FROM StudySchema.Students;
```

4.7.5 Системные функции.

Системные функции предназначены для получения информации о базе данных и ее содержимом. В SQL сервере существуют следующие системные функции:

- *COL_LENGTH* (таблица, поле) выводит ширину поля;
- *DATALENGTH* (выражение) выводит длину выражения;
- *GETANSINULL* (имя БД) выводит допустимо или недопустимо использовать в БД значение NULL;
- *IDENTINCR* (таблица) выводит шаг увеличения поля счетчика в таблице;
- *IDENT_SEED* (таблица) выводит начальное значение счетчиков в таблице;
- *ISDAT E*(выражение) выводит единицу, если выражение является датой и ноль, если не является;
- *ISNUMERIC* (выражение) выводит единицу, если выражение является числовым и ноль, если не числовым.

4.7.6 Работа с NULL

При работе с неизвестными значениями (*NULL*)

- *COALESCE*(*expression1,expression2,...*) возвращает первое не-*NULL* выражение из списка,

- *NULLIF* (*expression1*,*expression2*) возвращает *NULL*, если два выражения равны,
- *ISNULL* (*expression1*, *expression*) возвращает *expression2* в том случае, если *expression1* 'IS NULL'.

Использование оператора *COALESCE*. Возвращает первое выражение из списка аргументов, не равное *NULL*. Данный запрос выводит всех студентов, их стипендию. Если поле со стипендией будет пустым, т.е. иметь значение *NULL*, то оно заменится на другой аргумент :

```
SELECT Stud_FAM as 'Фамилия', Stud_Ima as 'Имя',
       COALESCE (cast(stud_stip as char(10)), 'Платник') as
       'Стипендия'
FROM StudySchema.Students
```

4.8 Обобщение данных с помощью агрегатных функций

Агрегатные функции позволяют вычислять итоговые значения по полям таблицы:

- *AVG(поле)* – выводит среднее значение поля;
- *COUNT(*)* – выводит количество записей в таблице;
- *COUNT(поле)* – выводит количество всех значений поля;
- *MAX(поле)* – выводит максимальное значение поля;
- *MIN(поле)* – выводит минимальное значение поля;
- *STDEV(поле)* – выводит средне квадратичное отклонение всех значений поля;
- *STDEVP(поле)* – выводит среднеквадратичное отклонение различных значений поля;
- *SUM(поле)* – суммирует все значений поля;
- *TOP n [Percent]* – выводит n первых записей из таблицы, либо n% записей из таблицы;
- *VAR(поле)* – выводит дисперсию всех значений поля;
- *VARP(поле)* – выводит дисперсию всех различных значений поля.

В общем случае агрегирующая функция задается следующим образом:

```
aggregate_function ([all | distinct] выражение)
```

Функции *SUM* и *AVG* применимы только к числовым полям - *int* (целое), *smallint* (малое целое), *tinyint* (очень малое целое), *decimal* (десятичное), *numeric* (числовой), *float* (плавающий), *money* (денежный). С функциями *COUNT*, *MAX*, *MIN* могут использоваться числовые или символьные поля. При применении к символьным полям *MAX*, *MIN* сравнивают значения в алфавитном порядке. Агрегатные функции при своей работе игнорируют значения *NULL*.

Чтобы найти среднюю стипендию студентов, можно ввести запрос:

```
SELECT  AVG (Stud_STIP)
        FROM StudySchema.Students;
```

Или

```
SELECT  AVG(ocenka) , MIN(ocenka) , MAX(ocenka)
        FROM StudySchema.Progress;
```

Функция *COUNT* несколько отличается от остальных. Она подсчитывает число значений в данной колонке или число строк в таблице. Например, подсчитаем количество студентов, сдавших учебные предметы:

```
SELECT COUNT( DISTINCT Stud_ID)
        FROM StudySchema.Progress
```

Обратите внимание, что в приведенном выше примере используется ключевое слово *DISTINCT* - это означает что подсчитываться будет количество уникальных значений в колонке *Stud_ID* таблицы *Progress*. Если опустить его, результат изменится. Опция *DISTINCT* (различные), которая может использоваться в функциях *SUM*, *AVG* и *COUNT*, позволяет исключить дублирующие значения и вести подсчет только различных значений указанного поля данных. Эту опцию нельзя использовать в функциях *MAX*, *MIN* и *COUNT(*)*. Для функций *SUM*, *AVG* и *COUNT* по умолчанию предполагается опция *all* (все), которая указывает на выполнение операций по всем значениям, включая дублирующиеся.

Чтобы подсчитать общее число строк в таблице, используйте функцию *COUNT* со звездочкой вместо имени поля:

```
SELECT COUNT (*)
        FROM StudySchema.Students
```

Предложение *GROUP BY* позволяет задавать Вам подмножество значений, для которых применяется агрегатная функция. Это дает возможность объединять поля и агрегатные функции в одном предложении *SELECT*. Практически мы получаем в результате запроса, использующего *GROUP BY*, новую таблицу. Предположим, что надо найти номера студентов с минимальной оценкой из ведомости успеваемости:

```
SELECT  Stud_ID, MIN(ocenka )
        FROM StudySchema.Progress
        GROUP BY Stud_ID
```

Расширением языка Transact-SQL является возможность вложенного агрегирования, то есть использование векторного агрегирования внутри

скалярного. Например, в следующем запросе вычисляется средняя стипендия для каждой группы:

```
SELECT AVG(stud_stip)
FROM StudySchema.Students
GROUP BY Grup_ID
```

Другой дополнительной возможностью языка Transact-SQL по отношению к SQL является группировка по выражению, которое не содержит агрегирующих функций. В стандартном языке SQL группировку можно проводить только по названиям столбцов. Например, следующий запрос допустим в языке Transact-SQL:

```
SELECT AVG(total_sales), total_sales * price
FROM titles
GROUP BY total_sales * price
```

Чтобы названия столбцов были осмысленными, измените предыдущую инструкцию следующим образом (примените *псевдонимы* для столбцов):

```
SELECT Stud_ID, MIN(ocenka) AS Мин_оценка
FROM StudySchema.Progress
GROUP BY Stud_ID
```

GROUP BY применяет агрегатные функции к группам записей. Условие формирования группы - одинаковое значение поля (в данном случае Stud_ID). *GROUP BY* можно использовать с несколькими полями. Усложним предыдущий запрос:

```
SELECT Stud_ID, Pr_Date, MAX(ocenka )
FROM StudySchema.Progress
GROUP BY Stud_ID, Pr_DATE
```

То есть мы выбираем строки с номерами студентов и максимальные оценки, полученные ими на каждую дату. Дни, в которые не было оценок, не будут показаны.

В следующем примере возвращается информация о количестве студентов в группах:

```
SELECT Grup_ID, COUNT(*) AS 'Количество'
FROM StudySchema.Students
GROUP BY Grup_ID
```

Ключевое слово *HAVING* используется в операторе *SELECT* вместе с ключевым словом *GROUP BY*, чтобы указать, какие из групп должны быть представлены в выводе.

Задаёт условие поиска, удовлетворяющее данную группу. Условие поиска может использовать статистические и нестатистические выражения. В нестатистических выражениях можно использовать только столбцы, отмеченные в предложении *GROUP BY* как столбцы группирования. Причина в том, что сгруппированные столбцы имеют общие для всей группы свойства. Аналогичным образом, статистические выражения имеют общее для всей группы свойство. Условие поиска предложения *HAVING* выражает предикат над свойствами группы.

Типы данных *image* и *text* нельзя использовать в предложении *HAVING*.

Для *GROUP BY* ключевое слово *HAVING* играет ту же роль, что и *WHERE* для *ORDER BY*. Предположим, что мы хотим получить максимальную оценку каждого студента, которая больше 4. Для достижения такого эффекта применяется предложение *HAVING*, которое определяет критерий, используемый для удаления групп из результата запроса, как это делает предложение *WHERE* для отдельных строк:

```
SELECT Stud_ID, Pr_DATE, MAX (ocenka )
FROM StudySchema.Progress
GROUP BY Stud_ID, Pr_DATE
HAVING MAX (ocenka ) > 4
```

Агрегатные функции позволяют не просто выбирать определенные значения из базы, но и производить их обобщение и анализ.

В следующем примере возвращается список всех групп, в которых больше 20 студентов:

```
SELECT Grup_ID, COUNT(*) AS 'Количество'
FROM StudySchema.Students
GROUP BY Grup_ID
HAVING (COUNT(*) )>20
```

4.9 Ограничение числа строк, возвращаемых в результирующем наборе запроса

Для ограничения числа строк, возвращаемых в результирующем наборе запроса до заданного числа или процентного значения в SQL Server, используется предложение *TOP*.

Если предложение *TOP* используется в сочетании с предложением *ORDER BY*, то результирующий набор ограничен первыми *N* строками отсортированного результата. В противном случае возвращаются первые *N*

случайных строк. Это предложение позволяет указать число строк, возвращаемых инструкцией *SELECT* или обработанных инструкциями *INSERT*, *UPDATE*, *MERGE* и *DELETE*.

Синтаксис.

```
[ TOP (expression) [ PERCENT ] [ WITH TIES ] ]
```

Аргументы:

- *Expression* - числовое выражение, определяющее количество возвращаемых строк. При указании *PERCENT* аргумент *expression* неявно преобразуется в тип *float*; в противном случае он преобразуется в тип *bigint*;
- *PERCENT* - указывает на то, что запрос возвращает только первые *expression* процентов строк из результирующего набора. Дробные значения округляются до следующего целого числа;
- *WITH TIES* - указывает, что результирующий набор включает какие-либо дополнительные строки, которые соответствуют значениям в столбце *ORDER BY* или столбцах в последней возвращенной строке. Это может привести к тому, что вернется строк больше, чем указано в значении *expression*. Например, если *expression* имеет значение 5, но еще 2 строки соответствуют значениям в столбцах *ORDER BY* в строке 5, то результирующий набор будет содержать 7 строк.

Пример. Использование ключевого слова *TOP* со значением константы. Выдать списки самых молодых студентов 1 и 2 групп. Если написать запрос следующим образом

```
SELECT TOP (1) Stud_ID, Stud_FAM, Stud_IMYA, YEAR(Stud_DATE)
FROM StudySchema.Students WHERE Grup_ID= 1
UNION ALL
SELECT TOP (1) Stud_ID, Stud_FAM, Stud_IMYA, YEAR(Stud_DATE)
FROM StudySchema.Students WHERE Grup_ID= 2
ORDER BY YEAR(Stud_DATE) ASC;
```

то возвращаются непредвиденные результаты, поскольку инструкция *TOP* логически выполняется раньше, чем предложение *ORDER BY*, которое сортирует результаты оператора (в данном случае *UNION ALL*).

Следующий пример показывает, как правильно написать запрос, который позволит получить нужный результат.

```
SELECT a.Stud_ID, a.Stud_FAM, a.Stud_IMYA, YEAR(a.DDDD)
FROM (SELECT TOP (1) Stud_ID, Stud_FAM, Stud_IMYA,
YEAR(Stud_DATE) DDDD
FROM StudySchema.Students
WHERE Grup_ID= 1
ORDER BY YEAR(Stud_DATE) ASC) AS a
UNION ALL
SELECT b.Stud_ID, b.Stud_FAM, b.Stud_IMYA, YEAR(b.DDDD)
```

```

FROM (SELECT TOP (1) Stud_ID, Stud_FAM, Stud_IMYA,
YEAR(Stud_DATE) DDDD
FROM StudySchema.Students
WHERE Grup_ID= 2
ORDER BY YEAR(Stud_DATE) ASC) AS b;

```

Использование *TOP* и *ORDER BY* во вложенной операции выбора гарантирует, что результаты предложения *ORDER BY* применяются к инструкции *TOP*, а не к сортировке результата операции *UNION*.

Пример. Указание процентов. В следующем примере *PERCENT* указывает число студентов, возвращаемых в результирующем наборе запроса. Если в таблице 290 студентов, то, поскольку 5% от 290 является дробным числом, значение округляется до следующего целого числа.

```

SELECT TOP (5) PERCENT Stud_ID, Stud_FAM, Stud_IMYA
FROM Students Students
ORDER BY Stud_ID DESC;

```

Пример. Использование ключевого слова *WITH TIES* для включения строк, соответствующих значениям в последней строке. Следующий пример извлекает первые 10 студентов с наибольшей стипендией и возвращает их в порядке убывания размера стипендии. Указание параметра *WITH TIES* позволяет быть уверенным в том, что все студенты со стипендией, равной минимальной возвращенной стипендии (последняя строка), включены в результирующий набор, даже если это приведет к превышению ограничения в 10 процентов студентов.

```

SELECT TOP (3) WITH TIES Stud_ID, Stud_FAM, Stud_IMYA,
Stud_STIP
FROM StudySchema.Students
ORDER BY Stud_STIP DESC;

```

4.10 Построение вычисляемых полей и выборка записей по дате

В общем случае для создания *вычисляемого (производного) поля* в списке *SELECT* следует указать некоторое выражение языка SQL. В этих выражениях применяются арифметические операции сложения, вычитания, умножения и деления, а также встроенные функции языка SQL. Можно указать имя любого столбца (поля) таблицы или запроса, но использовать имя столбца только той таблицы или запроса, которые указаны в списке предложения *FROM* соответствующей инструкции. При построении сложных выражений могут понадобиться скобки.

В операторе *SELECT* можно применять арифметические выражения, используя арифметические операторы:

Operator	Description
----------	-------------

+	Add
-	Subtract
*	Multiply
/	Divide

```
SELECT stud_name, stud_stip, 4*(stud_stip-100), 4*tuition - 100
FROM StudySchema.Students
```

В данном примере если stud_stip = 5500, то пересчитав на калькуляторе, в первом случае получим (приоритет операций имеет значение)

$$5500 - 100 = 5400$$

$$5400 * 4 = 21600,$$

а во втором

$$4 * 5500 = 22000$$

$$22000 - 100 = 21900$$

Пример. Получить список студентов с указанием фамилии и инициалов студентов и годовой стипендией:

```
SELECT Grup_id, Stud_FAM + ' ' + Left(Stud_IMYA,1) + '.' +
Left(Stud_OTCH, 1) + '.' AS 'Ф.И.О', STUD_STIP*12 AS
YEAR_STIP
FROM StudySchema.Students
```

В запросе использована встроенная функция *Left*, позволяющая вырезать в текстовой переменной один символ слева в данном случае.

При обращении к базе данных для выборки каких-то записей по дате нужно написать символьную строку приблизительно такого вида:

```
SELECT *
FROM SomeTable
WHERE Time <= '02/29/2000'
```

Пример. Вывод текущей даты:

```
SELECT GetDate()
```

В этом примере была использована встроенная функция времени для получения текущей даты.

Пример. Использование функции *DatePart*:

```
SELECT DatePart(year, Stud_date)
FROM StudySchema.Students
```

В этом примере используется функция *DatePart* для выделения года из даты.

Пример. Получить список студентов с указанием года и месяца рождения:

```
SELECT Stud_FAM, Year(Stud_DATE) AS Год,  
       Month (Stud_DATE) AS Месяц  
FROM   StudySchema.Students
```

В запросе использованы встроенные функции *Year* и *Month* для выделения года и месяца из даты.

Пример. Выбрать для каждого преподавателя его два инициала с точкой и фамилию, например, W.J.SMITH.

```
SELECT left(Teach_IMYA,1) + '.' + ' ' + left(Teach_OTCH,1) +  
'.' + ' ' + Teach_FAM  
FROM DekanatSchema.Teacher
```

4.11 UNPIVOT

Конструкции *PIVOT* и *UNPIVOT* появились в последних версиях стандарта SQL и были реализованы SQL Server, начиная с версии 2005. Первая из них позволяет значения в столбце вытянуть в строку, а вторая поможет нам выполнить обратную операцию:

```
SELECT col  
FROM  
      (SELECT col1, col2  
       FROM T) p  
UNPIVOT  
      (col FOR xxx IN (col1, col2)  
      )AS unpvt
```

Значения из столбцов *col1* и *col2* собираются в одном столбце *col* вспомогательной таблицы *unpvt*. Однако есть одна особенность в использовании операторов *PIVOT* и *UNPIVOT* - они не учитывают *NULL*-значения. Результат последнего запроса будет таким:

```
col  
1  
1  
1  
3  
2
```

Это препятствие на пути к решению нашей задачи можно преодолеть, если заменить *NULL*-значение на входе оператора *UNPIVOT* псевдозначением,

т.е. значением, которого заведомо не может быть в исходных данных, а потом выполнить обратное преобразование:

```
SELECT  NULLIF (col,777)
FROM
    (SELECT COALESCE(col1,777) col1, COALESCE(col2,777) col2
    FROM T) p
UNPIVOT
    (col FOR xxx IN
        (col1, col2)
    )AS unpvt
```

Здесь *COALESCE(colx,777)* заменяет *NULL*-значения в столбце *colx* на 777, а функция *NULLIF(col,777)* выполняет обратное преобразование.

Последнее решение дает нам требуемый результат, однако содержит один изъян - значение 777 может рано или поздно появиться в данных, что будет приводить к неверным результатам. Чтобы устранить этот огрех, можно использовать значение другого типа, которого заведомо не может присутствовать в целочисленном столбце, например, символ 'x'. Естественно, чтобы применить этот подход, для совместимости типов целочисленный тип столбцов следует конвертировать к символьному типу, выполнив при необходимости обратное преобразование конечного результата:

```
SELECT  CAST(NULLIF(col, 'x') AS INT)
FROM
    (SELECT COALESCE(CAST(col1 AS VARCHAR), 'x') col1,
        COALESCE(CAST(col2 AS VARCHAR), 'x') col2
    FROM T) p
UNPIVOT
    (col FOR xxx IN
        (col1, col2)
    )AS unpvt
```

Несколько слов об эффективности представленных решений. Согласно плану выполнения запроса, основные затраты обусловлены чтением данных (операция сканирования таблицы - Table scan). Для двух первых решений сканирование выполняется дважды, в то время как для последнего (UNPIVOT) - один раз, чем и обусловлено его двойное преимущество в производительности.

DROP TABLE T

Контрольные вопросы:

- 1) Назовите обязательные составляющие оператора SELECT.
- 2) Для всех ли данных в выражении ключевого слова WHERE обязательно нужно использовать кавычки?
- 3) К какому разделу SQL относится оператор SELECT?

- 4) Можно ли в выражении для ключевого слова **WHERE** задать несколько условий?
- 5) Допустимы ли кавычки для значений числовых полей?
- 6) Играет ли роль тип данных при использовании функции **COUNT**?
- 7) Чтобы группировать данные по столбцу, должен ли этот столбец быть указан в списке ключевого слова **SELECT**?
- 8) При использовании ключевого слова **ORDER BY** в операторе **SELECT** обязательно ли использовать ключевое слово **GROUP BY**?

5 Создание сложных запросов

5.1 Объединение таблиц в запросах

Чтобы понять как действует синтаксис объединения данных из таблиц *Table1 – Students* и *Table2 – Grup* с использованием **INNER JOIN** добавим в таблицу *Students* студентов, которые еще не распределены в группы (в поле *Grup_ID NULL*, а в таблицу *Grup* – группы, которых нет в таблице *Students*.

Псевдонимы назначаются таблицам с целью сокращения объема печатания, а также для использования при рекурсивном связывании таблиц.

Рассмотрим особенности применения аргумента **JOIN**:

- 1) **CROSS JOIN** указывает векторное произведение двух таблиц.
- 2) **LEFT [OUTER]** указывает, что все строки таблицы слева, которые не соответствуют указанному условию, включаются в результирующий набор в дополнение ко всем строкам, которые возвращаются внутренним объединением. Для выходных столбцов таблицы слева указано значение **NULL**.
- 3) **RIGHT [OUTER]** указывает, что все строки таблицы справа, которые не соответствуют указанному условию, включаются в результирующий набор в дополнение ко всем строкам, которые возвращаются внутренним объединением. Для выходных столбцов таблицы справа указано значение **NULL**.
- 4) **JOIN** показывает, что указанные таблицы следует объединить.
- 5) **ON < search_condition >** указание условия, на котором основывается объединение. Условие может указать любой допустимый предикат, однако часто используются столбцы и операторы сравнения.
- 6) **INNER** указывает, что все совпадающие пары строк возвращены. Отмена несовпадающих строк из обеих таблиц. Задается по умолчанию, если тип объединения не указан.

Использование **INNER JOIN** – это наиболее часто используемое в SQL соединение. Оно возвращает пересечение двух множеств. В терминах таблиц, оно возвращает только записи из обеих таблиц, отвечающие указанному критерию. На рисунке 5.1 показана диаграмма Венна, иллюстрирующая пересечение двух таблиц. Результат операции – закрашенная область.

Заметьте, что выдаются только данные из записей, имеющих одинаковые значения Chair_ID в таблицах.

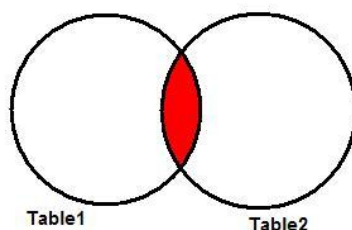


Рисунок 5.1 – INNER JOIN

Следующий пример возвращает имена только тех студентов, кто записан в группы:

```
SELECT S.Stud_FAM, S.Stud_IMYA, G.GRUP_NAME
FROM StudySchema.Students S
INNER JOIN StudySchema.Grup G
ON S.Grup_ID = G.Grup_ID
ORDER BY G.Grup_NAME
```

Аналог:

```
SELECT S.Stud_FAM, S.Stud_IMYA, G.GRUP_NAME
FROM StudySchema.Students S, StudySchema.Grup G
WHERE S.Grup_ID = G.Grup_ID
ORDER BY G. Grup_NAME
```

Использование *FULL JOIN* - Полностью это соединение называется *FULL OUTER JOIN* (зарезервированное слово *OUTER* необязательно). *FULL JOIN* работает как объединение двух множеств. На рисунке 5.2 показана диаграмма Венна для *FULL JOIN* двух таблиц. Результатом операции опять же является закрашенная область.

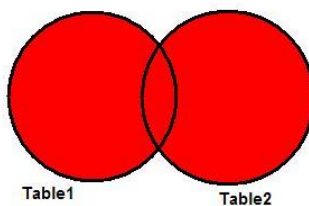


Рисунок 5.2 – FULL JOIN

Синтаксис почти такой же, как показанный выше:

```
SELECT A.Stud_ID, A.Stud_Fam, G. Grup_NAME
FROM StudySchema.Students A
FULL JOIN StudySchema.Grup G
ON A. Grup_ID= G.Grup_ID
```

Использование *CROSS JOIN* – это на самом деле декартово произведение. При использовании *CROSS JOIN* генерируется точно тот же результат, что и при вызове двух таблиц (разделенных запятой) без всякого *JOIN* вообще. Это значит, что мы получим огромный набор результатов, где каждая запись из *Table1(Students)* будет дублирована для каждой записи из *Table2 (Grup)*. Если в *Table1* содержится N1 записей, а в *Table2* – N2 записей, в результате будет N1 x N2 записей. В следующем примере возвращается перекрестное произведение двух таблиц, *Students* и *Group*. Возвращается список всех возможных сочетаний строк *Stud_ID* и *Group_ID*.

```
SELECT S.Stud_ID, G.Grup_Id
FROM StudySchema.Students S
CROSS JOIN StudySchema.Grup G
ORDER BY S.Stud_ID, G.Grup_ID
```

Использование *LEFT JOIN* - также известен как *LEFT OUTER JOIN*, и является частным случаем *FULL JOIN*. Дает все запрошенные данные из таблицы в левой части *JOIN* плюс данные из правой таблицы, пересекающиеся с первой таблицей. На рисунке 5.3 показана диаграмма Венна, иллюстрирующая *LEFT JOIN* для двух таблиц. Следующий пример соединяет две таблицы по столбцу *Group_ID* и сохраняет строки из левой таблицы, не имеющие соответствий. Таблица *Students* сравнивается с таблицей *Group* по столбцам *Group_ID*, которые имеются в обеих таблицах. В результирующем наборе отражаются все студенты (как входящие в группы, так и не входящие):

```
SELECT S.Stud_FAM,S.Stud_IMYA, G. GRUP_NAME
FROM StudySchema.Students S
LEFT JOIN StudySchema.Grup G ON S.Grup_ID = G. Grup_ID
ORDER BY G.Grup_NAME
```

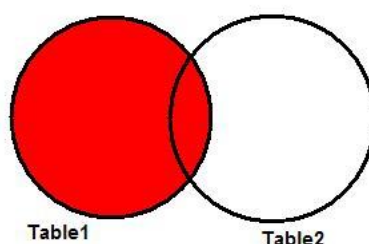


Рисунок 5.3 – LEFT JOIN

Использование *RIGHT OUTER JOIN* - также известен как *RIGHT OUTER JOIN*, и является еще одним частным случаем *FULL JOIN*. Он выдает все запрошенные данные из таблицы, стоящей в правой части оператора *JOIN*, плюс данные из левой таблицы, пересекающиеся с правой. Диаграмма Венна для *RIGHT JOIN* двух таблиц показана на рисунке 5.4.

Следующий пример соединяет две таблицы по столбцу Group_ID и сохраняет строки из левой таблицы, не имеющие соответствий. Таблица Group сравнивается с таблицей *Students* по столбцам Group_ID, которые имеются в обеих таблицах. В результирующем наборе отражаются все группы (как со студентами, так и без):

```
SELECT S.Stud_FAM, S.Stud_IMYA, G.GRUP_NAME
FROM StudySchema.Students S
RIGHT JOIN StudySchema.Grup G
      ON S.Grup_ID = G.Grup_ID
ORDER BY G.Grup_NAME
```

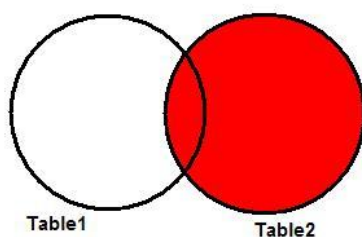


Рисунок 5.4 – RIGHT JOIN

SELF JOIN - Оператор JOIN можно использовать для комбинирования любой пары таблиц, включая комбинацию таблицы с самой собой. Это и есть «*SELF JOIN*». Рекурсивное связывание удобно использовать, когда все необходимые данные размещаются в одной таблице, но требуется каким-то образом сравнить одни записи таблицы с другими. Посмотрим на классический пример, возвращающий в таблице *Students* фамилиям студентов фамилии старост этих студентов. Получим список студентов с фамилиями их старост:

```
SELECT A.Stud_FAM, B.Stud_FAM
FROM StudySchema.Students A
LEFT JOIN StudySchema.Students B
      ON A.Stud_Star = B.Stud_ID
```

Связывание по нескольким ключам. Предположим вы хотите увидеть по каким видам работ и по каким предметам какие студенты какие получили оценки (для этого требуется связать четыре таблицы):

```
SELECT S.Stud_FAM, Sb.Subj_NAME , P.OCENKA
FROM StudySchema.Students S
JOIN StudySchema.Progress P
      ON S.Stud_ID = P.Stud_ID
JOIN StudySchema.Subjects Sb
      ON Sb.subj_id = P.subj_id
```

5.2 Использование подзапросов

5.2.1 Использование подзапросов, возвращающих единичное значение.

Запросы могут управлять другими запросами. Это особенно актуально, когда при использовании условия поиска в предложении *WHERE* значение, с которым надо сравнивать, заранее не определено и должно быть вычислено в момент выполнения оператора *SELECT*. В таком случае приходят на помощь законченные операторы *SELECT*, внедренные в тело другого оператора *SELECT*.

Внутренний подзапрос представляет собой также оператор *SELECT*, а кодирование его предложений подчиняется тем же правилам, что и основного оператора *SELECT*. Внешний оператор *SELECT* использует результат выполнения внутреннего оператора для определения содержания окончательного результата всей операции. Внутренние запросы могут быть помещены непосредственно после оператора сравнения (*=*, *<*, *>*, *<=*, *>=*, *<>*) в предложения *WHERE* и *HAVING* внешнего оператора *SELECT* – они получают название подзапросов или вложенных запросов. Кроме того, внутренние операторы *SELECT* могут применяться в операторах *INSERT*, *UPDATE* и *DELETE*.

Подзапрос – это инструмент создания временной таблицы, содержимое которой извлекается и обрабатывается внешним оператором. Текст подзапроса должен быть заключен в скобки. К подзапросам применяются следующие правила и ограничения:

- фраза *ORDER BY* не используется, хотя и может присутствовать во внешнем подзапросе;
- список в предложении *SELECT* состоит из имен отдельных столбцов или составленных из них выражений – за исключением случая, когда в подзапросе присутствует ключевое слово *EXISTS*;
- по умолчанию имена столбцов в подзапросе относятся к таблице, имя которой указано в предложении *FROM*. Однако допускается ссылка и на столбцы таблицы, указанной во фразе *FROM* внешнего запроса, для чего применяются квалифицированные имена столбцов (т.е. с указанием таблицы);
- если подзапрос является одним из двух операндов, участвующих в операции сравнения, то запрос должен указываться в правой части этой операции.

Существует два типа подзапросов:

Однострочный (скалярный) подзапрос возвращает единственное значение. В принципе, он может использоваться везде, где требуется указать единственное значение, оператора сравнения (*=*, *<*, *>*, *<=*, *>=*, *<>*).

Многострочный (табличный) подзапрос возвращает множество значений, т.е. значения одного или нескольких столбцов таблицы, размещенные в более чем одной строке. Он возможен везде, где допускается наличие таблицы.

Обычно внутренний запрос генерирует значение, которое проверяется в условии внешнего запроса, определяющего верно оно или, нет. Например, мы знаем фамилию студента - Сидоров, но не знаем его кода (*Stud_ID*), и хотим получить все его оценки из таблицы *Progress*:

```

SELECT *
FROM StudySchema.Progress
WHERE Stud_ID = (
    SELECT Stud_ID
    FROM StudySchema.Students
    WHERE Stud_FAM = 'Сидоров')

```

Чтобы выполнить внешний (основной запрос), сначала выполняется внутренний запрос (подзапрос) внутри предложения. При выполнении подзапроса просматривается таблица *Students*, в которой выбираются строки, где поле *Stud_FAM* равно 'Сидоров', затем извлекается значение поля *Stud_ID*. Пусть единственной строкой будет *Stud_ID* = 30104. Далее полученное значение помещается в условие основного запроса, вместо самого подзапроса, так что условие примет вид:

```

WHERE Stud_ID = 301004

```

При использовании подзапросов в условиях, основанных на операциях сравнения (больше, меньше, равно, не равно и т.д.), необходимо убедиться, что подзапрос будет возвращать одно и только одно значение. Если ваш подзапрос не вернет никакого значения, то основной запрос не выведет никаких значений.

На подзапросы накладываются следующие ограничения:

- подзапросы нельзя использовать в списках предложений *ORDER BY*, *GROUP BY* и *COMPUTE BY*;
- подзапрос не может содержать предложения *FOR BROWSE* или *UNION*;
- список выбора внутреннего подзапроса, которому предшествует операция сравнения, может содержать только одно выражение или название столбца, и подзапрос должен возвращать единственный результат. при этом тип данных столбца, указанного в конструкции *WHERE* внешнего оператора, должен быть совместим с типом данных в столбце, указанным в списке выбора подзапроса (правила здесь такие же, как и при соединении);
- в подзапросах не допускаются текстовые (*text*) и графические (*image*) данные;
- подзапросы не могут обрабатывать свои результаты внутренним образом, т.е. подзапрос не может содержать конструкций *ORDER BY*, *COMPUTE*, или ключевого слова *INTO*;
- коррелирующиеся (повторяющиеся) подзапросы не допускаются в конструкции *SELECT* обновляемого курсора, определенного с помощью *DECLARE CURSOR* (определить курсор);
- количество вложенных уровней для подзапросов не должно превышать 16.

– максимальное число подзапросов на каждой стороне объединения не больше 16.

5.2.2 Использование подзапросов, возвращающих множество значений.

Во многих случаях значение, подлежащее сравнению в предложениях *WHERE* или *HAVING*, представляет собой не одно, а несколько значений. Вложенные подзапросы генерируют непоименованное промежуточное отношение, временную таблицу. Оно может использоваться только в том месте, где появляется в подзапросе. К такому отношению невозможно обратиться по имени из какого-либо другого места запроса. Применяемые к подзапросу операции основаны на тех операциях, которые, в свою очередь, применяются к множеству, а именно:

- { *WHERE* | *HAVING* } выражение [*NOT*] *IN* (подзапрос) ;
- { *WHERE* | *HAVING* } выражение оператор_сравнения { *ALL* | *SOME* | *ANY* } (подзапрос) ;
- { *WHERE* | *HAVING* } [*NOT*] *EXISTS* (подзапрос) ;

Если Вы хотите использовать подзапрос, который возвращает несколько строк (многострочный), то необходимо использовать оператор *IN*. Например, если в базе несколько студентов с фамилией Сидоров и все имеют оценки:

```
SELECT *
FROM StudySchema.Progress
WHERE Stud_ID IN (
    SELECT Stud_ID
    FROM StudySchema.Students
    WHERE Stud_FAM = 'Сидоров' )
```

Найдем все оценки для предмета ТЭЦ:

```
SELECT P.OCENKA
FROM StudySchema.Progress P
JOIN StudySchema.Subjects Sb
ON P.subj_id= Sb.subj_id
WHERE P.Subj_ID IN (
    SELECT Subj_ID
    FROM StudySchema.Subjects
    WHERE Subj_NAME = 'ТЭЦ')
```

Этот результат можно получить и с помощью объединения:

```
SELECT P.OCENKA
FROM StudySchema.Subjects Sb
JOIN StudySchema.Progress P
ON Sb.subj_id= P.subj_id
```

```
WHERE Subj_NAME = 'ТЭЦ'
```

Хотя этот запрос эквивалентен предыдущему, SQL Server должен будет просмотреть каждую возможную комбинацию строк из двух таблиц и проверить их соответствие условию.

Все вышеприведенные подзапросы объединяет то, что все они выбирают один единственный столбец. Это обязательно, поскольку их результат сравнивается с одиночным значением. Команды типа *SELECT ** запрещены в подзапросах.

Подзапросы можно также использовать в предложении *HAVING*. Эти подзапросы могут использовать собственные предложения *GROUP BY* или *HAVING*. Следующий запрос является тому примером:

```
SELECT Ocenka, COUNT (DISTINCT Stud_ID )
FROM StudySchema.Progress
GROUP BY Ocenka
HAVING Ocenka > (
    SELECT AVG(ocenka )
    FROM StudySchema.Progress
    WHERE Pr_DATE >'01/12/16')
```

Эта команда подсчитывает студентов с оценкой выше средней, сдавших экзамен после 01.12.16.

5.3 Связанные подзапросы

При использовании подзапросов можно обратиться во вложенном подзапросе к таблицам из внешнего подзапроса. Например, как найти всех студентов, сдавших экзамены 1 марта:

```
SELECT *
FROM StudySchema.Students C
WHERE '2016-03-01' IN (
    SELECT Pr_DATE
    FROM StudySchema.Progress O
    WHERE O.Stud_ID = C.Stud_ID )
```

Связанные запросы можно использовать для сравнения таблицы с собой. Например, можно найти всех студентов с баллом выше среднего:

```
SELECT Stud_ID
FROM StudySchema.Progress O
WHERE ocenka > (
    SELECT AVG(ocenka )
    FROM StudySchema.Progress O1
    WHERE O1.Stud_ID = O.Stud_ID )
```

Использование оператора *EXISTS*. Иногда проще и удобнее использовать оператор *EXISTS*, который возвращает истинное значение, если подзапрос возвращает хотя бы одну строку (может и несколько), и не возвращает, если нет данных. Пусть необходимо найти все группы, в которых учится хотя бы один студент:

```
SELECT Grup_Name
FROM StudySchema.Grup G
WHERE EXISTS (
    SELECT *
    FROM StudySchema.Students Stud
    WHERE Stud.Grup_ID = G.Grup_ID)
```

Внутренний подзапрос выбирает всех студентов, которые учатся в группе. Оператор *EXISTS* во внешнем условии отмечает, что подзапрос вернул некоторые данные, следовательно, условие верное. Если подзапрос выдает хотя бы одного студента, то *EXISTS* возвращает истину и строка попадает в результат. Поэтому *EXISTS*, когда используется таким образом, делает условие верным или неверным для всех строк сразу.

Чтобы показать все группы, в которых нет студентов, достаточно перед *EXISTS* поставить оператор *NOT*. Благодаря оператору *NOT* происходит обратная операция – в результат попадают те строки, для которых подзапрос не вернул ни одного студента.

Использование операторов *ANY*, *SOME* и *ALL*. Рассмотрим новый способ поиска студента, сдавшего экзамен:

```
SELECT *
FROM StudySchema.Students
WHERE Stud_ID = ANY (
    SELECT Stud_ID
    FROM StudySchema.Progress)
```

Оператор *ANY* берет все значения, выведенные подзапросом, и оценивает их как верные, если любое из них равняется значению в текущей строке внешнего запроса. Это означает, что подзапрос должен выбирать значения такого же типа как и те, которые сравниваются в основном условии.

В приведенном выше запросе можно было бы использовать оператор *IN*. Однако отличие в том, что с оператором *ANY* можно применять все операторы сравнения (<>, >, >=, <, <=). *ANY* и *SOME* работают одинаково.

Если *ANY* и *SOME* требуют, чтобы условие подзапроса выполнилось хотя бы для одной строки, то оператор *ALL* считает условие верным, если каждое значение, выбранное подзапросом, удовлетворяет условию внешнего запроса. Выберем тех студентов, у которых стипендия выше 1500:

```
SELECT *
FROM StudySchema.Students
```

```
WHERE Stud_STIP > ALL(
    SELECT Stud_stip
    FROM StudySchema.Students
    WHERE Stud_STIP = 1500 )
```

5.4 Использование подзапроса в предложении FROM.

Рассмотрим запрос подсчета мальчиков и девочек по группам

```
SELECT g.Grup_name AS 'Группа',
    (SELECT COUNT(s.stud_id)
     FROM StudySchema.Students s
     WHERE s.Grup_id=g.Grup_id and s.Stud_pol = 'Жен') AS
'FEMALE',
    (SELECT COUNT(s.stud_id)
     FROM StudySchema.Students s
     WHERE s.Grup_id=g.Grup_id AND s.Stud_pol = 'муж')
AS 'MALE'
FROM StudySchema.Grup g;
```

Этот запрос можно выполнить, используя запрос в предложении *FROM*:

```
SELECT  A.Grup_ID, A.Male, B.Female, Grup.Grup_Name
FROM
    (SELECT COUNT(Stud_ID ) AS 'MALE', S.Grup_ID
     FROM StudySchema.Students S
     WHERE S.Stud_pol = 'муж'
     GROUP BY S.Grup_ID ) A
JOIN
    (SELECT COUNT (s.Stud_ID) AS 'FEMALE', S.GRUP_ID
     FROM StudySchema.Students s
     WHERE Stud_pol = 'жен'
     GROUP BY S.Grup_ID) B
ON B.Grup_ID = A.Grup_ID
JOIN
    StudySchema.Grup ON
    StudySchema.Grup.Grup_ID = B.Grup_ID
```

Использование подзапроса в *JOIN*. Можно использовать подзапрос в операторе *JOIN*. Узнаем разницу между стипендией студента и средней стипендией в группе.

```
SELECT Stud_fam, Stud_imya, qwe-Stud_stip Rasn_STIP
FROM StudySchema.Students S1
INNER JOIN
    (SELECT G.Grup_ID, COUNT(*) AS we, AVG(Stud_stip) AS qwe
     FROM StudySchema.Grup AS G, StudySchema.Students AS S
     WHERE S.Grup_ID=G.Grup_ID
     GROUP BY G.Grup_ID) AS B
ON B.Grup_ID=S1.Grup_ID
```

Использование команды *UNION*. Команда *UNION* объединяет вывод нескольких запросов с исключением повторяющихся строк. Например, приведенный ниже запрос выводит всех студентов и преподавателей, чьи фамилии размещены между буквами К и С:

```
SELECT Stud_FAM
FROM StudySchema.StudySchema.Students
WHERE Stud_FAM BETWEEN 'К' AND 'С'
UNION
SELECT Teach_FAM
FROM DekanatSchema.Teacher
WHERE Teach_FAM BETWEEN 'К' AND 'С'
```

Для применения команды *UNION* существует 2 правила:

- число и порядок следования колонок должны быть одинаковы во всех запросах;
- типы данных должны быть совместимы.

```
SELECT stud_id, Stud_fam, Stud_stip, 'student'
FROM students
UNION
SELECT teach_id, teach_fam, salary, 'teacher'
FROM teacher;
```

UNION автоматически исключает дубликаты строк из вывода. Если вы хотите, чтобы все строки из запросов попали в результат, используйте *UNION ALL*:

```
SELECT Stud_FAM
FROM StudySchema.Students
UNION ALL
SELECT Teach_FAM
FROM DekanatSchema.Teacher
```

Использование оператора *CASE*. Достаточно часто возникает задача сделать несколько сравнений за один запрос. Например, вывести лучшую, среднюю и худшую группы на курсе. В этом и подобных случаях удобно использовать оператор *CASE*. Чаще всего её используют для генерации вариантов значений поля в выборках, однако, у этой функции есть ещё много полезных применений.

Оператор (или более современное название «выражение») *CASE* имеет два формата:

- простое выражение *CASE* для определения результата сравнивает выражение с набором простых выражений:

```
CASE input_expression
WHEN when_expression THEN result_expression
```

```

        [ ...n ]
    [
        ELSE else_result_expression    ]
END ;

```

– поисковое выражение *CASE* для определения результата вычисляет набор логических выражений:

```

CASE
    WHEN Boolean_expression THEN result_expression
        [ ...n ]
    [
        ELSE else_result_expression
    ]
END

```

Оба формата поддерживают дополнительный аргумент *ELSE*.

Выражение *CASE* может использоваться в любой инструкции или предложении, которые допускают допустимые выражения. Например, выражение *CASE* можно использовать в таких инструкциях, как *SELECT*, *UPDATE*, *DELETE* и *SET*, а также в таких предложениях, как *select_list*, *IN*, *WHERE*, *ORDER BY* и *HAVING*.

Например, приведенный ниже запрос использует простое выражение и выводит всех студентов и их статус обучения:

```

SELECT Stud_FAM + ' ' + Stud_ImYA AS 'Фамилия Имя',
       CASE
           WHEN stud_stip is NULL then 'Платник'
           ELSE 'Грантник'
       END AS 'Статус'
FROM StudySchema.Students

```

Рассмотрим запрос, который ищет лучшую группу по средней оценке в диапазоне использует поисковое выражение:

```

SELECT G.Grup_name,AVG(P.Ocenka) AS 'srednii bal',
CASE
    WHEN AVG(P.Ocenka) between 7 and 9 THEN 'otlichnyi rezultat'
    WHEN AVG(P.Ocenka) between 4 and 6 THEN 'srednii rezultat'
    WHEN AVG(P.Ocenka) between 0 and 3 THEN 'nizkii rezultat'
    END AS 'rezultat gruppy'
FROM StudySchema.Grup G, StudySchema.Progress P
WHERE G.Grup_ID=P.Grup_ID
GROUP BY G.Grup_name

```

SQL Server допускает применение в выражениях *CASE* не более 10 уровней вложенности. Выражение *CASE* нельзя использовать для управления

потоком выполнения инструкций Transact-SQL, блоков инструкций, определяемых пользователем функций и хранимых процедур.

Использование оператора *COALESCE*. Возвращает первое выражение из списка аргументов, не равное *NULL*. Данный запрос выводит всех студентов, их стипендию. Если поле со стипендией будет пустым, т.е. иметь значение *NULL*, то оно заменится на другой аргумент :

```
SELECT Stud_FAM as 'Фамилия', Stud_Imya as 'Имя',  
       COALESCE (cast(stud_stip as char(10)), 'Платник')  
       as 'Стипендия'  
FROM StudySchema.Students
```

5.6 Оператор MERGE

Язык манипулирования данными (*Data Manipulation Language, DML*) — самый используемый модуль команд языка Transact-SQL. В девяносто случаях из ста применяется именно он. В последнее время в языке появились новые конструкции: *MERGE*, *WITH*, *CTE*-выражения, *PIVOT*, *UNPIVOT*, *TOP* и многие другие. Оператор *TOP* мы уже рассматривали.

MERGE — DML-оператор для вставки (*INSERT*) /обновления (*UPDATE*) /удаления (*DELETE*) данных при слиянии таблиц, на основе результатов соединения с исходной таблицей (или подзапросом) - эта команда является гибридом команд *INSERT*, *UPDATE* и *DELETE*. Она выполняет обновление результирующей таблицы на основе ее объединения с исходной таблицей.

Оператор *MERGE* иногда называют еще *UPSERT* = *UPdate* + *inSERT*, так как он как бы объединяет в себе два этих оператора. Данный оператор позволяет обновить основную таблицу с помощью вспомогательной «виртуальной» таблицы. При совпадении ключевых полей в строках обеих таблиц строки основной таблицы обновляются данными из совпавших строк вспомогательной таблицы; все не совпавшие строки из вспомогательной таблицы добавляются в основную таблицу.

Логика работы команды следующая:

```
/* Объединяются результирующая и исходная таблицы */  
MERGE <Результирующая таблица>  
      USING <Исходная таблица> ON <Условие объединения>  
/* Что делать со строками результирующей таблицы, для которых  
найлены соответствующие строки исходной таблицы  
*/  
WHEN MATCHED [AND <Дополнительное условие>]  
  THEN UPDATE SET ...  
/* или */
```

THEN DELETE

/* В конструкции MERGE может присутствовать до двух сценариев для "WHEN MATCHED". В этом случае первый сценарий обязательно должен включать "Дополнительное условие". При этом один сценарий должен выполнять UPDATE, а другой DELETE

*/

/* Что делать, если в исходной таблице есть строки, для которых нет соответствующих строк в результирующей таблице

*/

**WHEN TARGET NOT MATCHED [AND <Дополнительное условие>]
THEN INSERT (...) VALUES (...)**

/* Что делать со строками результирующей таблицы, для которых не найдены соответствующие строки в исходной таблице

*/

**WHEN SOURCE NOT MATCHED [AND <Дополнительное условие>]
THEN UPDATE SET ...
/* или */
THEN DELETE**

Примеры.

1

-- Текущие остатки

```
CREATE TABLE ProductInventory
(
    ProductId varchar(100) primary key,
    Quantity int not null
)
INSERT ProductInventory(ProductId, Quantity)
VALUES ('book', 10), ('magazine', 5)
-- Поступление и расход

CREATE TABLE ProductInventoryUpdate
(
    ProductId varchar(100) primary key,
    Quantity int not null
)
INSERT ProductInventoryUpdate (ProductId, Quantity)
VALUES('book', 1), ('magazine', -5), ('postcard', 2)
```

2

-- Изменяем остатки с учетом поступления и расхода

```
MERGE ProductInventory pi
USING ProductInventoryUpdate pu
ON pi.ProductId = pu.ProductId
WHEN MATCHED AND pi.Quantity + pu.Quantity = 0
    THEN DELETE
WHEN MATCHED
    THEN UPDATE SET Quantity += pu.Quantity
WHEN TARGET NOT MATCHED
```

```

        THEN INSERT(ProductId, Quantity) VALUES
(pu.ProductId, pu.Quantity)

```

3

```

SELECT * FROM ProductInventory

```

Результат:

	ProductId	Quantity
1	book	11
2	postcard	2

Пример. Расчет средней оценки студентов.
Реализуется через 2 шага.

1

```

CREATE TABLE Reating
(
    stud_id    int,
    Stud_F     char(20),
    Stud_Name  char(10),
    Stud_O     char(15),
    Grup_nam   char(15),
    sr_ball    char(10)
)

```

2

```

MERGE INTO Reating  r
    USING (
        SELECT DISTINCT
s.Stud_ID,stud_Fam,stud_imya,stud_Otch,grup_name,
avg(ocenka) AS 'sr_ball'
FROM students s
inner join grup g ON g.grup_id=s.grup_id
right join Progress p on p.Stud_ID=s.Stud_ID
WHERE p.Stud_ID IN (
                SELECT  Stud_ID
                FROM      Progress
                WHERE Stud_ID <> 0 )
                GROUP BY
                s.Stud_ID,stud_fam,stud_imya,stud_otch,grup_name)
s
    ON (r.stud_id = s.stud_id)
WHEN MATCHED THEN
    UPDATE
        SET r.sr_ball = s.sr_ball,
            r.stud_id=s.stud_id,
            r.stud_F=s.stud_Fam,
            r.stud_Name=s.stud_Imya,
            r.stud_O=s.stud_Otch,
            r.grup_nam=s.grup_name
    WHEN NOT MATCHED THEN

```

```

INSERT
(stud_id,stud_f,stud_name,stud_o,grup_nam,sr_ball)
VALUES
(stud_id,stud_fam,stud_imya,stud_otch,grup_name,sr_ball);

```

5.7 GROUPING SETS

В T-SQL уже есть две конструкции, позволяющие получать в одном запросе сводные строки по разным наборам столбцов. Это «*GROUP BY ... WITH CUBE*» и «*GROUP BY ... WITH ROLLUP*». Первая конструкция помимо стандартных строк, возвращаемых предложением *GROUP BY*, добавляет в результирующий набор сводные строки для всех возможных сочетаний всех или части столбцов, перечисленных в *GROUP BY*. При использовании конструкции «*GROUP BY ... WITH ROLLUP*» результирующий набор будет состоять из сводных строк для наборов, включающих первые *i* столбцов, перечисленных в предложении *GROUP BY* ($i = 0 \dots$ число столбцов, указанных в *GROUP BY*).

Давайте рассмотрим пример:

```

-- Создание и заполнение таблицы для примера
CREATE TABLE SampleData
(
    Id_1 int not null,
    Id_2 int not null,
    Id_3 int not null,
    [Value] int not null
)
INSERT SampleData (Id_1, Id_2, Id_3, [Value])
VALUES
(1, 1, 1, 10), (1, 1, 2, 10), (1, 2, 1, 10), (1, 2, 2, 10),
(2, 1, 1, 10), (2, 1, 2, 10), (2, 2, 1, 10), (2, 2, 2, 10)

-- Запрос с конструкцией GROUP BY ... WITH CUBE
SELECT Id_1, Id_2, Id_3, SUM(Value)
FROM SampleData
GROUP BY Id_1, Id_2, Id_3 WITH CUBE
ORDER BY 1, 2, 3

```

Результат:

	Id_1	Id_2	Id_3	(No column name)
1	NULL	NULL	NULL	80
2	NULL	NULL	1	40
3	NULL	NULL	2	40
4	NULL	1	NULL	40
5	NULL	1	1	20
6	NULL	1	2	20
7	NULL	2	NULL	40
8	NULL	2	1	20
9	NULL	2	2	20
10	1	NULL	NULL	40
11	1	NULL	1	20
12	1	NULL	2	20
13	1	1	NULL	20

Как видно, в результирующем наборе присутствуют итоговые строки по следующим сочетаниям столбцов:

(), (Id_1), (Id_2), (Id_3), (Id_1, Id_2), (Id_1, Id_3), (Id_2, Id_3), (Id_1, Id_2, Id_3)

```
-- Запрос с конструкцией GROUP BY ... WITH ROLLUP
SELECT Id_1, Id_2, Id_3, SUM(Value)
FROM SampleData
GROUP BY Id_1, Id_2, Id_3 WITH ROLLUP
ORDER BY 1, 2, 3
```

Результат:

	Id_1	Id_2	Id_3	(No column name)
1	NULL	NULL	NULL	80
2	1	NULL	NULL	40
3	1	1	NULL	20
4	1	1	1	10
5	1	1	2	10
6	1	2	NULL	20
7	1	2	1	10
8	1	2	2	10
9	2	NULL	NULL	40
10	2	1	NULL	20
11	2	1	1	10
12	2	1	2	10
13	2	2	NULL	20
14	2	2	1	10
15	2	2	2	10

В этом случае в результирующем наборе присутствуют итоговые строки по следующим сочетаниям столбцов:

(), (Id_1), (Id_1, Id_2), (Id_1, Id_2, Id_3)

Новая конструкция *GROUPING SETS*, добавленная в T-SQL в SQL Server 2008, позволяет произвольным образом указывать по каким наборам столбцов нужно считать сводные строки. Например, мы хотим получить в результирующем наборе только общий итог, итог сгруппированный по столбцу Id_1 и итог сгруппированный по набору столбцов (Id_1, Id_2, Id_3). С помощью новой конструкции *GROUPING SETS* это можно сделать:

```
SELECT Id_1, Id_2, Id_3, SUM(Value)
FROM SampleData
GROUP BY GROUPING SETS (( ), (Id_1), (Id_1, Id_2, Id_3))
ORDER BY 1, 2, 3
```

Результат:

	Id_1	Id_2	Id_3	(No column name)
1	NULL	NULL	NULL	80
2	1	NULL	NULL	40
3	1	1	1	10
4	1	1	2	10
5	1	2	1	10
6	1	2	2	10
7	2	NULL	NULL	40
8	2	1	1	10
9	2	1	2	10
10	2	2	1	10
11	2	2	2	10

5.8 Функции RANK() и DENSE_RANK()

Эти функции, как и функция *ROW_NUMBER()*, тоже нумеруют строки, но делают это несколько отличным способом. Это отличие проявляется в том, что строки, которые имеют одинаковые значения в столбцах, по которым выполняется упорядочивание, получают одинаковые номера (ранги). Например, значения (отсортированные по возрастанию)

1
5
6
6
6

получат такие номера:

1	1
5	2
6	3
6	3
6	3

Возникает вопрос, с какого номера продолжится нумерация, если, скажем, в последовательности чисел появится 7 и т.д.? Здесь есть два варианта:

- с номера 4, т.к. это следующий номер по порядку;
- с номера 6, т.к. следующая строка будет шестая по счету.

Такая "неоднозначность" и привела к появлению двух функций вместо одной - *RANK* и *DENSE_RANK*, первая из которых продолжит нумерацию с 6, а вторая (плотная) - с 4.

Рассмотрим несколько примеров. Начнем с демонстрации отличия в поведении функций *RANK* и *ROW_NUMBER*:

Контрольные вопросы:

- 1) Можно ли иметь несколько ключевых слов *AND* в выражении, заданном ключевым словом *WHERE*?
- 2) Что такое рекурсивное связывание?
- 3) При связывании таблиц должны ли они связываться в том же порядке, в каком они указаны в выражении ключевого слова *FROM*?
- 4) При использовании в операторе запроса таблицы-связки, обязательно ли выбирать в запросе ее столбцы?
- 5) Можно ли связывать в запросе не один, а несколько столбцов таблиц?
- 6) Какая часть оператора *SQL* задает условия связывания таблиц?
- 7) Что будет, если в запросе указать выборку из двух таблиц, но не связать их?
- 8) Для чего используются псевдонимы таблиц?
- 9) Что такое подзапрос? Как он работает?

6 Представления

6.1 Общие сведения о представлениях

Представление (*VIEW*) является своего рода виртуальной таблицей, обеспечивающей доступ пользователей к некоторому подмножеству данных, составленному из одной или более таблиц. Для пользователя представление как обычная таблица, но при этом само по себе оно не содержит данных. Физически представление реализовано в виде *SQL*-запроса, на основе которого производится выборка данных из одной или более таблиц, или представлений. Таблицы, на основе которых создается представление, называются базовыми (*BASE TABLE*). В самом простом случае можно создать представление на базе одной таблицы, которое будет содержать точно такой же набор данных, как и исходная таблица. На практике такие представления редко используются. Более сложные представления могут объединять столбцы из нескольких таблиц. При этом, задав условие для выборки, можно сделать доступным из представления только ограниченное множество строк

из этих таблиц. Когда из представления исключается один или более столбцов базовой таблицы, говорят, что на таблицу наложен вертикальный фильтр. Если в определении представления установлено одно или более условий для выборки строк, говорят, что на таблицу наложен горизонтальный фильтр.

Механизм представлений позволяет ограничить доступ пользователей к конфиденциальным данным в таблице. Каждый раз, когда пользователь обращается к представлению, сервер осуществляет выборку данных, в соответствии с запросом, содержащемся в определении этого представления. При этом проверяется актуальность всех ссылок в запросе на предмет существования объектов, требующихся для его выполнения. Если одна из таблиц, включенных в запрос, окажется недоступной, то станет невозможным использование представления в целом.

Представления создаются с помощью оператора *CREATE VIEW*. Аналогично оператору *CREATE TABLE*, оператор *CREATE VIEW* можно использовать только для создания представления, которого до сих пор не существовало.

```
CREATE VIEW [ schema_name . ] view_name [ (column [ ,...n ]
) ]
[ WITH <view_attribute> [ ,...n ] ]
AS select_statement
[ WITH [CASCADED | LOCAL]
CHECK OPTION ] [ ; ]
<view_attribute> ::=
{
    [ ENCRYPTION ]
    [ SCHEMABINDING ]
    [ VIEW_METADATA ] }
```

CHECK OPTION - Обеспечивает соответствие всех выполняемых для представления инструкций модификации данных критериям, заданным при помощи аргумента *select_statement*. Если строка изменяется посредством представления, предложение *WITH CHECK OPTION* гарантирует, что после фиксации изменений доступ к данным из представления сохранится.

Представление может быть создано только в текущей базе данных.

Инструкция *CREATE VIEW* должна быть первой в пакетном запросе.

6.2 Опции [WITH [CASCADED | LOCAL] CHECK OPTION

Для изменяемого представления можно указывать фразу *WITH CHECK OPTION*, позволяющую предотвращать "потерю строк" в представлениях. Так, если эта фраза указана, то при внесении изменений в таблицу будет проверен предикат, указанный в запросе, использованном для создания таблицы.

Если предикат не возвращает значение *TRUE*, то изменения не будут внесены.

Например, если запрос создан оператором

```
CREATE VIEW v_tb11 AS
  (SELECT f1,f2, f3 FROM tb11 WHERE f2>100)
  WITH CHECK OPTION;;
```

то вставка строки не будет произведена:

```
INSERT INTO v_tb11 (f1,f2,f3)
  VALUES (1,50,'abc');
```

Фраза *WITH CHECK OPTION* может быть расширена до:

- 1) *WITH CASCADED CHECK OPTION* - предикаты проверяются во всех вложенных запросах;
- 2) *WITH LOCAL CHECK OPTION* - предикаты проверяются только в запросе, использованном для создания данного представления.

Для представления, созданного операторами

```
CREATE VIEW v_1 AS
  (SELECT f1,f2, f3 FROM tb11 WHERE f2>100);,
CREATE VIEW v_2 AS
  (SELECT f1,f2, f3 FROM v_1 WHERE f2>50)
  WITH LOCAL CHECK OPTION;;
```

добавление строки будет выполнено:

```
INSERT INTO v_2 (f1,f2,f3)
  VALUES (1,30,'abc');
```

Эта строка будет добавлена в основную таблицу, но не будет видна в представлении, посредством которого она была добавлена.

По умолчанию предполагается, что для *WITH CHECK OPTION* используется фраза *CASCADED*.

Предложения *SELECT*, используемые в определении представления, не могут включать следующие элементы:

- предложение *ORDER BY*, если только в списке выбора инструкции *SELECT* нет также предложения *TOP*;
- ключевое слово *INTO*;
- предложение *OPTION*;
- ссылку на временную таблицу или табличную переменную.

Представление можно создать на базе данных одной или нескольких таблиц, а также других представлений.

6.3 Представление на базе одной таблицы

Создадим представление *Stud_Address* из таблицы *Students*:

```
CREATE VIEW StudAddress
AS
SELECT Stud_ID, Stud_FAM, Stud_IMYA, Stud_ADDRESS
FROM StudySchema.Students
```

Создадим представление *StudStip* из таблицы *StudySchema.Students*:

```
CREATE VIEW StudStip
AS
SELECT TOP (100) stud_fam, stud_stip
FROM StudySchema.Students
ORDER BY stud_fam
```

На первый взгляд, бессмысленно создавать представление на базе одной таблицы. Однако некоторые столбцы таблицы могут содержать данные, доступ к которым отдельным категориям пользователей может оказаться не желательным или наоборот (например, размер стипендии студента). Или попросту нет необходимости в определенных данных из той или иной таблицы (допустим, нас интересуют только фамилия, имя и адрес студента). В данном случае создается представление на базе некоторой таблицы, из которого исключаются конфиденциальные, либо не нужные данные.

Для удаления представлений из базы данных используется команда *DROP VIEW имя_представления*. У этой команды есть две опции – *RESTRICT* и *CASCADE*. Если используется *RESTRICT* и в условиях имеются зависимые представления, то оператор *DROP VIEW* возвращает ошибку. При использовании опции *CASCADE* и наличии зависимых представлений оператор *DROP VIEW* завершается успешно, и все зависимые представления тоже удаляются.

Представления удаляются с помощью оператора *DROP VIEW <имя_представления>*:

```
DROP VIEW StudAddress
```

6.4 Представление на базе нескольких таблиц

Чаще всего представления используются для упрощения работы с SQL, и нередко это относится к объединениям.

Создадим представление *PrepodChair* на базе двух таблиц *Teacher* и *Chair*, которое бы содержало фамилии преподавателей, работающих на кафедрах:

```
CREATE VIEW Prepod_Chair
AS
SELECT T.teach_fam, C.Chair_name
FROM DekanatSchema.Teacher T, DekanatSchema.Chair C
WHERE T.Chair_id=C.Chair_id
```

Создадим представление *Studocenka* из таблиц *Students*, *Subject* и *Progress*, в котором отражается список студентов с их оценками по предметам:

```
CREATE VIEW Studocenka
AS
SELECT stud.stud_fam, Sb.subj_name, P.ocenka
FROM StudySchema.Students stud, StudySchema.Subject
Sb, StudySchema.Progress P
WHERE (P.stud_id=stud.stud_id) and
(P.subj_id=Sb.subj_id)
```

Представления чрезвычайно полезны для упрощения использования вычисляемых полей. Создадим представление *Otli4n*, в котором отражается количество отличников по предметам в группе:

```
CREATE VIEW Otli4n
AS
SELECT g.Grup_name as 'Группа', s.subj_name as 'Предмет',
count(p.ocenka) as 'Количество отличников'
FROM StudySchema.Progress p
JOIN StudySchema.Grup g
ON g.Grup_id = p.Grup_id
JOIN StudySchema.Subjects s
ON s.subj_id = p.subj_id
WHERE p.ocenka in (8,9)
GROUP BY g.Grup_name, s.subj_name
```

Таким образом, представления могут значительно упростить сложные операторы SQL. Используя представления, можно один раз записать код SQL и затем повторно использовать его, если возникает такая необходимость.

Другим наиболее частым случаем использования представлений является переформатирование выбранных данных. Предположим, что результаты регулярно требуются в определенном формате. Вместо того, чтобы выполнять объединение каждый раз, когда в этом возникает необходимость, можно создать представление и использовать его вместо объединения.

Можно использовать представления для переформатирования данных. Создадим представление, отражающее преподавателей, работающих на кафедрах в определенном формате:

```
CREATE VIEW Prepod_Chair2
AS
SELECT left(T.Teach_IMYA,1) + '.' + left(T.Teach_OTCH,1)
+ '.' + left(T.Teach_FAM,15)
+ '(' + left(T.teach_position,10) + ')' AS
'Dekanat_Teacher' , C.Chair_name AS 'Dekanat_Chair'
```

```
FROM DekanatSchema.Teacher T, DekanatSchema.Chair C
WHERE C.Chair_id=T.Chair_id
```

6.5 Представление на базе представлений

Поскольку представления представляют собой виртуальные таблицы, то на их базе также можно создавать другие представления. Допустим, нам требуется создать представление отображающее количество грантников и платников на специальностях. Данное представление, конечно, можно сделать на основе сложных запросов, однако, сделаем его на базе двух представлений *Grantn* (Грантники) и *Platn* (Платники).

Сначала необходимо создать и заполнить таблицу *Speciality* (Специальности):

```
CREATE TABLE Speciality (
    Spec_ID      int not null PRIMARY KEY,
    Spec_NAME    char (20)  not null)
```

Далее необходимо добавить атрибут *spec_id* в таблицу *StudySchema.Students*, создать внешний ключ на *spec_id* в таблице *Spec*:

```
ALTER TABLE StudySchema.Students
ADD spec_id int foreign key references
Speciality (spec_id)
```

Создадим представление *Grantn*, отражающее специальность и количество грантов на этой специальности:

```
CREATE VIEW Grantn
AS
SELECT Spec.spec_name, count (distinct S.stud_id) as
'grant'
FROM StudySchema.Students S, Speciality Spec
WHERE S.spec_id=Spec.spec_id and S.Stud_stip is not null
GROUP BY Spec.spec_name
```

Создадим представление *Plat*, отражающее специальность и количество платников на этой специальности:

```
CREATE VIEW Plat
AS
SELECT spec.spec_name, count (distinct stud_id) as 'plat'
FROM stud, Spec
WHERE stud.spec_id=spec.spec_id and Stud_stip is null
GROUP BY spec_name
```

Теперь создадим собственно само представление *StudSpecGP*, отражающее специальность и количество платников, грантников на базе представлений *Grantn* и *Plat*:

```
CREATE VIEW StudSpecGP
AS
    SELECT  p.spec_name AS 'Speciality' ,
            (SELECT [grant]
             FROM Grantn
             WHERE grantn.spec_name= g.spec_name) AS 'Gratniki',
            (SELECT [plat]
             FROM plat
             WHERE plat.spec_name= p.spec_name) AS 'Platniki'
FROM grantn g, plat p
WHERE g.spec_name=p.spec_name
GROUP BY g.spec_name, p.spec_name
```

Контрольные вопросы:

- 1) Что такое представление?
- 2) Как называются таблицы, на основе которых создается представление?
- 3) Что означают понятия «Вертикальный», «Горизонтальный» фильтры?
- 4) Что случится, если таблица, на основе которой строится представление, будет удалена?
- 5) Как представление можно использовать для защиты данных?
- 6) Содержит ли данные представление?
- 7) Для достижения каких целей используют представления?

7 Хранимые процедуры и функции

7.1 Использование процедур

Команды SQL (*CREATE TABLE*, *INSERT*, *UPDATE*, *SELECT*) дают возможность сообщить базе данных, что делать, но не как делать. Сообщить ядру базы данных, каким образом следует обрабатывать данные, можно посредством составления процедур. Хранимые процедуры – это набор операторов SQL, созданный для удобства использования в программах. Сохраненную процедуру использовать проще, чем каждый раз записывать весь набор входящих в нее операторов SQL. Сохраненные процедуры можно вкладывать одну в другую (уровень вложенности не может превышать 16).

Сохраненная процедура может возвращать значения, выполнять сравнения вводимых пользователем значений с заранее установленными условиями, вычислить какие-либо результаты и т.п.

Некоторые из преимуществ использования сохраненных процедур:

- операторы процедуры уже сохранены в базе данных;
- операторы процедуры уже проверены и находятся в готовом для использования виде;
- при использовании процедур результат получается быстрее;
- возможность сохранения процедур позволяет использовать модульное программирование;
- сохраненные процедуры могут вызывать другие процедуры;
- сохраненные процедуры могут вызываться другими программами.

В SQL Server процедуры создаются с помощью оператора следующего вида:

```
CREATE PROCEDURE имя_процедуры
[ [ ( ] @имя_параметра
  ТИП_ДАННЫХ [ (ДЛИНА) | (ТОЧНОСТЬ) [ , МАСШТАБ ] )
  [=DEFAULT] [OUTPUT] ]
[ , @ИМЯ_ПАРАМЕТРА
  ТИП_ДАННЫХ [ (ДЛИНА) | (ТОЧНОСТЬ) [ , МАСШТАБ ] )
  [=DEFAULT] [OUTPUT] ]
[WITH RECOMPILE]
AS операторы SQL
```

Сохраненные процедуры используются следующим образом:

```
EXECUTE [ @ = ] имя_процедуры
[ [ @ имя_параметра = ] значение |
[ @ имя_параметра = ] @ переменная [ OUTPUT ] ]
[WITH RECOMPILE]
```

Идентификаторы - это имена объектов, на которые можно ссылаться в программе, написанной на языке Transact SQL. Первый символ может состоять из букв английского алфавита или "_", "@", "#". Остальные дополняются из цифр и «\$».

Имя идентификатора не должно совпадать с зарезервированным словом.

Для ограничителей идентификаторов при установленном параметре *SET QUOTED_IDENTIFIER ON* можно использовать как квадратные скобки, так и одинарные кавычки, а строковые значения только в одинарных кавычках (режим по умолчанию).

Если использовать установленный параметр в режиме *SET QUOTED_IDENTIFIER OFF*, то в качестве ограничителей идентификаторов можно использовать только квадратные скобки, а строковые значения указываются в одинарных или двойных кавычках.

Переменные используются для сохранения промежуточных данных в хранимых процедурах и функциях. Все переменные считаются локальными. Имя переменной должно начинаться с @.

Написать процедуру достаточно просто, зная, к примеру, оператор *SELECT*.

```
CREATE    PROCEDURE All_Stud
AS
    SELECT *
    FROM    StudySchema.Students;
```

После создания этой процедуры вызовем ее на выполнение:

```
EXEC All_Stud;
```

Таблица 7.1 - Специальные знаки и простейшие операторы в Transact SQL

Знак	Назначение	Знак	Назначение
*	Знак умножения	<code>''</code>	В них заключают строковые значения, если SET QUOTED_IDENTIFIER OFF
-	Знак вычитания	<code>''</code>	В них заключают строковые значения
%	Остаток от деления двух чисел	<>	Не равно
+	Знак сложения или конкатенации (объединение двух строк в одну)	[]	Аналог кавычек, в них можно заключать названия идентификаторов, если в их названиях встречаются пробелы
=	Знак равенства или сравнения	!<	Не менее чем
<=	Меньше или равно	!>	Не более чем
>=	Больше или равно	>	Больше
!=	Не равно	<	Меньше
@	Ставится перед именем переменной	•	Разделяет родительские и подчиненные объекты
@@	Указывает на системные функции	/	Знак деления
	Однострочный комментарий или комментарий с текущей позиции и до конца строки	/* */	Многострочный комментарий

Например, мы хотим увеличить на единицу значения номеров курсов (в поле Grup_COURSE) в таблице *Gruppa*:

```
CREATE    PROCEDURE new_course
AS
    UPDATE StudySchema.GRUPPA
    SET Grup_COURSE = Grup_COURSE +1 ;
```

Проверим работу процедуры:

```
EXEC new_course;

SELECT *
FROM    STUDYSCHEMA.GRUPPA;
```

От процедуры *All_Stud* не будет большой пользы, если не предусмотреть

передачу через нее каких либо данных (параметров) для выполнения некоторых действий. Параметры могут быть как входными, так и выходными, для получения определенной информации или результатов.

Пример процедуры, которую можно использовать для добавления новых групп:

```
CREATE PROCEDURE new_Gruppa
( @Grup_ID int ,
  @Grup_NAME char (9) ,
  @Grup_KOLSTUD int,
  @Grup_COURSE int )
AS
INSERT INTO StudySchema.Gruppa
VALUES (@Grup_NAME,
        @Grup_KOLSTUD,
        @ Grup_COURSE) ;
```

Столбец *StudySchema.Grup_ID* таблицы *StudySchema.Gruppa* имеет тип *identity*, поэтому определяется только при вводе.

Работа этой процедуры проверяется следующим образом:

```
EXEC new_Gruppa 18, 'ЕСТ-04-5', 25, 1;
```

Обратите внимание, что при вводе данных система автоматически присваивает полю *StudySchema.Grup_ID* очередное значение независимо от того, какое значение определил пользователь.

Простая процедура по использованию оператора *SELECT*:

```
CREATE PROCEDURE spisok_stud
AS
SELECT * from StudySchema.Students;
```

Процедура для добавления преподавателя:

```
CREATE PROCEDURE new_Teacher
( @Teach_ID bigint,
  @Teach_FAM char (20) ,
  @Teach_IMA char (10) ,
  @Teach_OTCH char (15) ,
  @Teach_POSITION char (18) ,
  @Teach_STEPEN char (12) ,
  @Chair_ID integer)
AS
INSERT INTO DekanatSchema.Teacher
VALUES (@Teach_ID,@Teach_FAM,@Teach_IMA,
        @Teach_OTCH,@Teach_POSITION,@Teach_STEPEN, @Chair_ID)
```

Работа этой процедуры проверяется следующим образом:

```
EXEC new_Teacher    <список параметров>
```

Процедура для добавления нового предмета:

```
CREATE PROCEDURE new_Subject
( @subj_name char(20) ,
  @lection_hours int ,
  @practice_hours int ,
  @labor_hours int )
AS
DECLARE @subj_id int
DECLARE @total_hours int
IF not exists (SELECT subj_name
               FROM   StudySchema.Subjects
               WHERE  subj_name=@subj_name)
BEGIN
SET @subj_id = (select max(subj_id)
  FROM   StudySchema.Subjects)+1
SET @total_hours = @lection_hours + @practice_hours +
@labor_hours
INSERT INTO StudySchema.Subjects
VALUES (@subj_id, @subj_name, @total_hours,
@lection_hours, @practice_hours, @labor_hours)
END;
```

Работа этой процедуры проверяется следующим образом:

```
EXEC new_Subject '3',20,20,30;
```

Данная процедура прежде чем добавить предмет проверяет существование такого предмета в таблице. Т.о., используя условные операторы, можно выполнять те или иные действия по проверке.

Процедура добавления нового студента, которая при необходимости также обновляет таблицу *StudySchema.Gruppa*:

```
CREATE PROCEDURE new_stud
(@stud_id int ,
@stud_fam char(20) ,
@stud_ima char(20) ,
@stud_otch char(20) ,
@stud_date datetime ,
@stud_addr char(20) ,
@Grup_name char(20) ,
@Grup_kurs int ,
@stip smallmoney ,
@spec char(20))
AS
```

```

declare @Grup_id int;
declare @spec_id int;
declare @kolvo int;
INSERT INTO StudySchema.Grup
VALUES (@Grup_name, 0, @Grup_kurs);
INSERT INTO Speciality
VALUES (@spec);
SELECT @Grup_id=(SELECT G.Grup_id FROM StudySchema.Grup G
WHERE G.Grup_name=@Grup_name);
SELECT @spec_id=(SELECT spec_id FROM Speciality WHERE
Spec_NAME=@spec);
INSERT INTO StudySchema.Students
VALUES (@stud_id, @stud_fam, @stud_ima, @stud_otch, @stud_date,
@stud_addr, @Grup_id, @stud_id, @stip, @spec_id);
SELECT @kolvo=(select count(stud_id) from
StudySchema.Students S where S.Grup_id=@Grup_id);
UPDATE StudySchema.Grup
SET StudySchema.Grup.Grup_kolstud=@kolvo WHERE
StudySchema.Grup.Grup_id=@Grup_id

```

Работа этой процедуры проверяется следующим образом:

```

EXEC new_stud 65785, 'Иванов', 'Иван', 'Иванович',
'08/05/89', 'Almaty', 'BVT-77-7', 2, 1000, 'ВЧИПО'

```

Рассмотрим процедуру по поиску текста в строке:

```

CREATE PROCEDURE poisk
@p_a varchar(20)
AS BEGIN
SELECT * FROM StudySchema.Students

WHERE stud_fam like (@p_a+'%');
END
GO

```

7.2 Функции

Функции — это подпрограммы, состоящие из одной или нескольких инструкций Transact-SQL, которые могут использоваться для сохранения кода для повторного использования. Пользовательская функция не может выполнять действия, изменяющие состояние базы данных.

Каждая функция имеет ряд свойств, определяющих возможность использования ее в вычисляемых столбцах.

Функция представляет собой именованный блок кода, возвращающий по результатам своего выполнения некоторое значение. Возвращаемое значение может быть, как величиной, так и набором строк. Функции могут участвовать в выражениях, вызываться в теле хранимых процедур, а также в теле других функций.

Помимо встроенных функций, присутствующих в системе по умолчанию, пользователи могут создавать собственные функции. Поскольку их можно включать в любое выражение, пользовательские функции представляют собой удобный и гибкий механизм обработки данных.

SQL Server поддерживает 4 типа функций:

1) *Встроенные функции* — большой набор предопределенных функций, которые не могут быть изменены, используются для расширения возможностей Transact-SQL по обработке данных в SQL Server.

Существующее многообразие встроенных функций SQL Server можно условно разделить на несколько категорий:

- математические функции;
- строковые функции;
- функции для работы с датами;
- статистические функции;
- криптографические функции;
- ранжирующие функции;
- функции приведения типов;
- системные функции.

2) *Скалярные функции* — возвращают единственное значение с типом, определенным в операторе *RETURNS*.

```
CREATE FUNCTION [ schema_name. ] function_name
( [ { @parameter_name [ AS ] [ type_schema_name. ]
parameter_data_type
    [ = default ] [ READONLY ] }
  [ ,...n ]
]
)
RETURNS return_data_type
    [ WITH <function_option> [ ,...n ] ]
    [ AS ]
BEGIN
    function_body
    RETURN scalar_expression
END
[ ; ]
```

3) *Табличные функции с одним запросом* — возвращают таблицу, которая является результатом одной инструкции *SELECT*.

```
CREATE FUNCTION [ schema_name. ] function_name
( [ { @parameter_name [ AS ] [ type_schema_name. ]
parameter_data_type
    [ = default ] [ READONLY ] }
  [ ,...n ]
]
)
RETURNS TABLE
```

```

    [ WITH <function_option> [ ,...n ] ]
    [ AS ]
    RETURN [ ( ] select_stmt [ ) ]
[ ; ]

```

4) *Многооператорные табличные функции* — возвращают таблицу, созданную одной или несколькими инструкциями Transact-SQL и сходны с хранимыми процедурами. В отличие от хранимой процедуры, на многооператорную табличную функцию можно сослаться в предложении *FROM* инструкции *SELECT*.

```

CREATE FUNCTION [ schema_name. ] function_name
( [ { @parameter_name [ AS ] [ type_schema_name. ]
parameter_data_type
    [ = default ] [READONLY] }
    [ ,...n ]
]
)
RETURNS @return_variable TABLE <table_type_definition>
[ WITH <function_option> [ ,...n ] ]
[ AS ]
BEGIN
    function_body
RETURN
END
[ ; ]

```

Изменение функции осуществляется командой Transact-SQL *ALTER FUNCTION*, удаление - *DROP FUNCTION*.

Пример. Функция определения возраста студента (например, мы ходим узнать возраст студента по id_stud):

```

CREATE FUNCTION GetStudentAge(@uid int)
RETURNS varchar(200)
AS
BEGIN
    declare @age datetime
    SELECT @age=Stud_DATE
    FROM StudySchema.Students WHERE Stud_ID=@uid
    SET @age=GETDATE()-@age;
    RETURN convert(varchar(20),YEAR(@age)-1900)+' лет,
'+convert(varchar(20),MONTH(@age)-1)+' месяцев и
'+convert(varchar(20),DAY(@age)-1)+' дней'
END

```

Данную пользовательскую функцию можно использовать в следующем запросе:

```

SELECT Stud_FAM as 'Фамилия', Stud_IMYA as 'Имя',

```

```

Stud_DATE as 'Дата рождения',
dbo.GetStudentAge(Stud_ID) as 'Возраст'
FROM StudySchema.Students
ORDER BY Stud_DATE ASC

```

Т.о., создавая пользовательские функции, можно значительно упростить сложность запросов.

Пример. Создать функцию, возвращающую количество студентов в конкретной группе.

```

Create Function GetKolStud(@uid int)
RETURNS int
AS
BEGIN
    declare @studid bigint
    SELECT @studid = Count(Stud_id)
    FROM StudySchema.Students WHERE Grup_id = @uid
    RETURN @studid
END;

go

SELECT G.Grup_Name AS [Название группы],
       dbo.GetKolStud(S.Grup_ID) AS [Количество студентов]
FROM StudySchema.Students S
JOIN StudySchema.Grup G
ON S.Grup_ID = S.Group_ID;

CREATE FUNCTION get_credits_for_group(
    @group_id int
) RETURNS int
AS
BEGIN
    DECLARE @res int
    SELECT @res=SUM(Kredit_cout)
    FROM Study
    WHERE Group_ID=@group_id
    RETURN @res
END;

```

Для проверки результата работы функции выполним команду:

```
PRINT dbo.get_credits_for_group(2)
```

7.3 Курсоры языка Transact-SQL

В СУБД ORACLE курсоры используются достаточно часто при написании процедур, триггеров и пакетов. Инструкции Microsoft SQL Server создают полный результирующий набор, но бывают случаи, когда результаты

удобнее обрабатывать построчно. В последней версии SQL Server 2008 время курсоры языка Transact-SQL используются в основном в хранимых процедурах, триггерах и скриптах Transact-SQL, делая содержимое результирующего набора доступным для других инструкций Transact-SQL.

Алгоритм использования курсора языка Transact-SQL в хранимой процедуре или триггере следующий:

1) Объявите переменные языка Transact-SQL, которые будут содержать данные, возвращенные курсором. Объявите по одной переменной для каждого столбца результирующего набора. Объявите переменные достаточно большого размера для хранения значений, возвращаемых в столбце и имеющих тип данных, к которому могут быть неявно преобразованы данные столбца.

2) Воспользуйтесь инструкцией *DECLARE CURSOR*, чтобы связать курсор языка Transact-SQL с инструкцией *SELECT*. Инструкция *DECLARE CURSOR* определяет также характеристики курсора, например, имя курсора и тип курсора (*read-only* или *forward-only*).

3) Воспользуйтесь инструкцией *OPEN* для выполнения инструкции *SELECT* и заполнения курсора.

4) Воспользуйтесь инструкцией *FETCH INTO* для выборки отдельных строк и перемещения данных для каждого столбца в указанную переменную. После этого другие инструкции языка Transact-SQL могут ссылаться на эти переменные для доступа к выбранным значениям данных. Курсоры языка Transact-SQL не поддерживают выборку группы строк.

5) После завершения работы с курсором примените инструкцию *CLOSE*. Заккрытие курсора освобождает некоторые ресурсы, например, результирующий набор курсора и его блокировки на текущей строке, однако структура курсора будет доступна для обработки, если снова выполнить инструкцию *OPEN*. Поскольку курсор все еще существует на этом этапе, повторно использовать его имя не удастся. Инструкция *DEALLOCATE* полностью освобождает все ресурсы, выделенные курсору, в том числе имя курсора. После освобождения курсора его необходимо перестроить заново с помощью инструкции *DECLARE*.

7.3.1 Управление курсором в среде MS SQL Server.

Управление курсором реализуется путем выполнения следующих команд:

- а) *DECLARE* – создание или объявление курсора;
- б) *OPEN* – открытие курсора, т.е. наполнение его данными;
- в) *FETCH* – выборка из курсора и изменение строк данных с помощью курсора;
- г) *CLOSE* – закрытие курсора;
- д) *DEALLOCATE* – освобождение курсора, т.е. удаление курсора как объекта.

Объявление курсора

В стандарте SQL для создания курсора предусмотрена следующая команда:

```
<создание_курсора>::=  
  DECLARE имя_курсора  
    [INSENSITIVE][SCROLL] CURSOR  
  FOR SELECT_оператор  
  [FOR { READ_ONLY | UPDATE  
    [OF имя_столбца[, ...n]]}]
```

7.3.2 Курсорные переменные.

Transact-SQL позволяет объявлять переменные типа *CURSOR*. В этом случае стандартный синтаксис *DECLARE* не создает курсор; вы должны явно установить переменную для курсора с помощью ключевого слова *SET*.

```
DECLARE myCursor CURSOR  
  LOCAL  
  FAST_FORWARD  
  FOR SELECT OilName FROM Oils  
DECLARE @myCursorVariable CURSOR  
SET @myCursorVariable = myCursor
```

Этот синтаксис полезен, если вы хотите создать переменные, которые могут быть назначены различным курсорам. Это может потребоваться, если вы создаете обобщенную процедуру для работы с различными результирующими множествами.

Вы можете объявить курсорную переменную, а затем использовать ее для непосредственного создания курсора.

```
DECLARE @myCursorVariable CURSOR  
SET @myCursorVariable = CURSOR  
  LOCAL FAST_FORWARD FOR SELECT OilName FROM Oils
```

При использовании такого синтаксиса курсор не имеет идентификатора, и ссылка на него может быть осуществлена только через переменную. Подобный синтаксис больше подходит для хранимых процедур.

7.3.3 Открытие курсора.

Объявление курсора создает объект курсора, но не создает набор записей, которыми курсор будет манипулировать (множество курсора). Множество курсора не создается, пока вы не откроете курсор. После достаточно сложного синтаксиса оператора *DECLARE CURSOR*, синтаксис оператора кажется вполне прозрачным:

```
OPEN [GLOBAL] курсор_или_переменная
```

Ключевое слово *GLOBAL* помогает избежать конфликтов: если курсор, объявленный с ключевым словом *LOCAL*, и курсор, объявленный с ключевым словом *GLOBAL*, имеют одинаковый идентификатор, ссылки на курсор будут по умолчанию отнесены к локальному курсору, если вы не использовали ключевое слово *GLOBAL*. Как и в других подобных случаях, лучше явно указывать ключевое слово, если вы открываете глобальный курсор.

7.3.4 Закрытие курсора.

Закончив использование курсора, вы должны его закрыть. Оператор *CLOSE* освобождает ресурсы, используемые для обслуживания множества курсора, а также освобождает все блокировки, наложенные на строки, если вы использовали параметр *SCROLLLOCKS* в операторе *DECLARE*. Синтаксис команды *CLOSE* почти идентичен синтаксису оператора *OPEN* – меняется только ключевое слово:

***CLOSE* [*GLOBAL*] курсор_или_переменная**

7.3.5 Манипулирование строками с помощью курсора.

Курсоры сами по себе не вызвали бы никакого интереса, если бы вы не могли осуществлять с их помощью определенные действия. *Transact-SQL* поддерживает три различные команды для работы с курсорами: *FETCH*, *UPDATE* и *DELETE*.

Команда *FETCH* извлекает указанную строку из множества строк курсора. В своем простейшем варианте команда *FETCH* имеет следующий синтаксис:

***FETCH* курсор_или_переменная**

В этом формате записи возвращается строка в позиции курсора (текущая строка).

Команда *FETCH* может не только возвращать строку непосредственно, но и позволяет сохранять значения из возвращенного столбца в переменных. Чтобы сохранить результаты выполнения команды *FETCH* в переменной, используйте следующий синтаксис:

***FETCH* курсор_или_переменная
INTO список_переменных**

Список_переменных - перечень разделяемых запятыми идентификаторов переменных. Перед выполнением команды *FETCH* вы должны объявить переменные. *Список_переменных* должен содержать переменную для каждого столбца, фигурирующего в операторе *SELECT*, который определяет курсор. Тип данных переменной должен либо совпадать, либо быть совместимым с *типом данных столбца*.

```

DECLARE Stud_Cursor CURSOR --объявляем курсор
FOR
    SELECT Stud_ID, Stud_Stip
    FROM StudySchema.Students
    ORDER BY Stud_ID DESC
OPEN MyCursor --вызываем отработку курсора
--внутри работаем, как с массивом
FETCH NEXT FROM MyCursor
FETCH NEXT FROM MyCursor
FETCH NEXT FROM MyCursor
CLOSE MyCursor --закрываем курсор
DEALLOCATE MyCursor --уничтожаем курсор

```

Создадим процедуру для вывода списка преподавателей по признаку «пенсионер – не пенсионер», использующую курсор.

```

CREATE PROCEDURE Pensionery
AS
DECLARE @teach    VARCHAR(20) ,
        @age      int,
        @pol      int,
        @status   VARCHAR(15) ,
        @message  VARCHAR(80)
PRINT '  Список преподавателей'
DECLARE teach_cursor CURSOR GLOBAL SCROLL
KEYSET FOR
    SELECT Teach_Fam, dbo.Age_year(Teach_ID) , Pol_ID
    FROM DekanatSchema.Teacher
    FOR UPDATE
OPEN teach_cursor
FETCH NEXT FROM teach_cursor
    INTO @teach, @age, @pol
WHILE @@FETCH_STATUS=0
BEGIN
    IF (@age>=63) and (@pol=1)
        begin
            set @status=' - Пенсионер'
        end
    Else IF (@age<63) and (@pol=1)
        begin
            set @status=' - Не достиг '
        end
    ELSE IF (@age>=58) and (@pol=2)
        begin
            set @status=' - Пенсионерка'
        end
    ELSE IF (@age<58) and (@pol=2)
        begin
            set @status=' - Не достигла'
        end

    SELECT @message='Преподаватель: '+@teach+
        '  Возраст: '+cast(@age as varchar)+@status

```

```

Print @message
FETCH NEXT FROM teach_cursor
    INTO @teach, @age, @pol
END

FETCH ABSOLUTE 1 FROM teach_cursor
    INTO @teach, @age, @pol

CLOSE teach_cursor
DEALLOCATE teach_cursor
EXEC Pensionery

```

Контрольные вопросы:

- 1) Может ли сохраненная процедура вызывать другую сохраненную процедуру?
- 2) В чем преимущества использования процедур?
- 3) В чем преимущества использования функций?
- 4) Каковы различия между процедурами и функциями?
- 5) Принцип работы курсора.

8 Триггеры

8.1 Понимание триггера

Триггер — это особая разновидность хранимой процедуры, выполняемая автоматически при возникновении события на сервере базы данных. Триггеры языка обработки данных выполняются по событиям, вызванным попыткой пользователя изменить данные с помощью языка обработки данных. Событиями DML являются процедуры *INSERT*, *UPDATE* или *DELETE*, применяемые к таблице или представлению. Эти триггеры срабатывают при запуске любого допустимого события независимо от того, влияет ли оно на какие-либо строки таблицы. Наиболее распространенное применение триггеров — это проверка сложных критериев в базе данных. Триггер — особый инструмент SQL-сервера, поддерживающий целостность данных в БД.

Триггер представляет собой специальный тип хранимых процедур, запускаемых сервером автоматически при попытке изменения данных в таблицах, с которыми триггеры связаны. Каждый триггер привязывается к конкретной таблице. Все производимые им модификации данных рассматриваются как одна транзакция. В случае обнаружения ошибки или нарушения целостности данных происходит откат этой транзакции. Тем самым внесение изменений запрещается. Отменяются также все изменения, уже сделанные триггером.

Создает триггер только владелец базы данных. Это ограничение позволяет избежать случайного изменения структуры таблиц, способов связи с ними других объектов и т.п.

Триггер представляет собой весьма полезное и в то же время опасное средство. Так, при неправильной логике его работы можно легко уничтожить целую базу данных, поэтому *триггеры* необходимо очень тщательно отлаживать. Применение триггеров большей частью весьма удобно для пользователей базы данных. И все же их использование часто связано с дополнительными затратами ресурсов на операции ввода/вывода. В том случае, когда тех же результатов (с гораздо меньшими непроизводительными затратами ресурсов) можно добиться с помощью хранимых процедур или прикладных программ, применение триггеров нецелесообразно.

В отличие от обычной подпрограммы, триггер выполняется неявно в каждом случае возникновения триггерного события, к тому же он не имеет аргументов. Приведение его в действие иногда называют запуском триггера. С помощью триггеров достигаются следующие цели:

- проверка корректности введенных данных и выполнение сложных ограничений целостности данных, которые трудно, если вообще возможно, поддерживать с помощью ограничений целостности, установленных для таблицы;
- выдача предупреждений, напоминающих о необходимости выполнения некоторых действий при обновлении таблицы, реализованном определенным образом;
- накопление аудиторской информации посредством фиксации сведений о внесенных изменениях и тех лицах, которые их выполнили;
- поддержка репликации.

Триггер состоит из 4-х частей:

- область определения имени функции (*имя_триггера*);
- область, показывающая для какой таблицы создается триггер (*имя_таблицы* | *имя_просмотра*);
- область, показывающая, когда выполнять триггер (*INSERT*- при создании записи в таблице, *DELETE*- при удалении и *UPDATE*- при изменении) и как его выполнять (*AFTER*- после выполнения операции, *INSTEAD OF*- вместо выполнения операции);
- тело триггера, содержит команды языка программирования запросов T-SQL.

```
<определение_триггера> ::=  
    {CREATE | ALTER} TRIGGER имя_триггера  
    ON {имя_таблицы | имя_представления }  
    [WITH ENCRYPTION ]  
    {  
        { { FOR | AFTER | INSTEAD OF }  
        { [ DELETE] [,] [ INSERT] [,] [ UPDATE] }  
        [ WITH APPEND ]  
        [ NOT FOR REPLICATION ]  
    }  
    AS  
        sql_оператор[...n]
```

```

} |
{ {FOR | AFTER | INSTEAD OF } { [INSERT] [,]
  [UPDATE] }
[ WITH APPEND]
[ NOT FOR REPLICATION]
AS
{ IF UPDATE(имя_столбца)
  [ {AND | OR} UPDATE(имя_столбца)] [...n]
  |
  IF (COLUMNS_UPDATES() {оператор_бит_обработки}
    бит_маска_изменения)
    {оператор_бит_сравнения }бит_маска [...n]}
  sql_оператор [...n]
  }}

```

Триггер может быть создан только в текущей базе данных, но допускается обращение внутри триггера к другим базам данных, в том числе и расположенным на удаленном сервере.

Имя триггера должно быть уникальным в пределах базы данных. Дополнительно можно указать имя владельца.

При указании аргумента *WITH ENCRYPTION* сервер выполняет шифрование кода триггера, чтобы никто, включая администратора, не мог получить к нему доступ и прочитать его. Шифрование часто используется для скрывания авторских алгоритмов обработки данных, являющихся интеллектуальной собственностью программиста или коммерческой тайной.

8.2 Типы триггеров

Существует два *вида* триггеров:

1) DML - запускаются с помощью инструкций *INSERT*, *UPDATE* или *DELETE*.

2) DDL - запускаются с помощью инструкций *CREATE*, *ALTER*, *DROP*.

В SQL Server существует два *параметра*, определяющих поведение триггеров:

FOR | AFTER - Тип *AFTER* указывает, что триггер DML срабатывает только после успешного выполнения всех операций в инструкции SQL, запускаемой триггером. Все каскадные действия и проверки ограничений, на которые имеется ссылка, должны быть успешно завершены, прежде чем триггер сработает. Если единственным заданным ключевым словом является *FOR*, аргумент *AFTER* используется по умолчанию. Триггеры *AFTER* не могут быть определены на представлениях.

INSTEAD OF - Указывает, что триггер DML срабатывает *вместо* инструкции SQL, используемой триггером, переопределяя таким образом действия выполняемой инструкции триггера. Аргумент *INSTEAD OF* не может быть указан для триггеров DDL или триггеров входа. На каждую инструкцию *INSERT*, *UPDATE* или *DELETE* в таблице или представлении может быть

определено не более одного триггера *INSTEAD OF*. Однако можно определить представления на представлениях, где у каждого представления есть собственный триггер *INSTEAD OF*. Использование триггеров *INSTEAD OF* не допускается в поддерживающих обновление представлениях, которые используют параметр *WITH CHECK OPTION*. SQL Server вызывает ошибку, если триггер *INSTEAD OF* добавляется к поддерживающему обновлению представлению с параметром *WITH CHECK OPTION*. Пользователь должен удалить этот параметр при помощи инструкции *ALTER VIEW* перед определением триггера *INSTEAD OF*.

Также триггеры различают по типу команд, на которые они реагируют (три типа триггеров):

1) *INSERT TRIGGER* – запускаются при попытке вставки данных с помощью команды *INSERT*.

2) *UPDATE TRIGGER* – запускаются при попытке изменения данных с помощью команды *UPDATE*.

3) *DELETE TRIGGER* – запускаются при попытке удаления данных с помощью команды *DELETE*.

Конструкции *[DELETE] [,] [INSERT] [,] [UPDATE]* и *FOR | AFTER | INSTEAD OF* { *[INSERT] [,] [UPDATE]* определяют, на какую команду будет реагировать триггер. При его создании должна быть указана хотя бы одна команда. Допускается создание триггера, реагирующего на две или на все три команды. Аргумент *WITH APPEND* позволяет создавать несколько триггеров каждого типа.

1) Команда *INSERT*. В таблице *inserted* будут содержаться *все строки*, которые пользователь пытается вставить в таблицу. Таблица *deleted* не будет содержать ни одной строки. После завершения триггера все строки из таблицы *inserted* будут вставлены в таблицу.

2) Команда *DELETE*. В таблице *deleted* будет приведен *список строк*, которые пользователь пытается удалить. Триггер может проверить каждую строку и решить, разрешено ли ее удаление. Таблица *inserted* не будет содержать ни одной строки.

3) Команда *UPDATE*. При выполнении этой команды таблица *deleted* будет содержать старые значения строк, которые будут удалены при успешном завершении триггера. Новые значения строк содержатся в таблице *inserted*. Эти строки будут добавлены в исходную таблицу после успешного завершения триггера.

При вызове триггеров используются две специальные таблицы: таблица удаления (*deleted table*) и таблица добавления (*inserted table*).

Для отслеживания изменений, которые выполнил пользователь в таблице необходимо получить доступ к данным, которые были изменены пользовательским запросом. Эту информацию можно получить из триггерных таблиц *inserted* и *deleted*, которые автоматически создаются сервером при вызове триггера. В этих таблицах содержатся списки строк, которые будут соответственно вставлены и удалены по завершении

транзакции. Структура таблиц *inserted* и *deleted* идентична структуре таблицы, для которой определен триггер. Пользователь не может непосредственно изменять данные в этих таблицах, но может использовать находящуюся в них информацию для проверки последствий выполнения операторов *insert*, *update* или *delete*.

В таблице *deleted* сохраняются копии строк, которые удаляются операторами *UPDATE* или *DELETE*. В процессе выполнения этих операторов строки удаляются из триггерной таблицы и помещаются в таблицу удаления. Обычно триггерная таблица и таблица удаления не имеют общих строк.

В таблице *inserted* сохраняются копии строк, которые вставляются операторами *INSERT* или *UPDATE*. В процессе выполнения этих операторов новые строки вставляются в таблицу добавления и триггерную таблицу одновременно. Таким образом, таблица добавления всегда содержит копии новых строк, которые были добавлены в триггерную таблицу.

Триггеры могут быть привязаны не к таблице, а к представлению (*VIEW*). В этом случае с их помощью реализуется механизм «обновляемого представления».

Для примера рассмотрим создание для таблицы *Teacher* триггера *Teach_tr1*, который будет запрещать вставку новых строк в таблицу, выдавая при этом сообщение 'Вставка строк запрещена':

```
CREATE TRIGGER Teach_tr1
ON DekanatSchema.Teacher
FOR INSERT AS
PRINT 'Вставка строк запрещена'
ROLLBACK TRAN
```

Рассмотрим простой пример. Стипендия студента не может превышать 15000. Определим триггер, который не позволял бы вводить сумму стипендии больше 15000. Для создания триггера с помощью утилиты SQL Server Enterprise Manager необходимо выбрать таблицу *Progress* в списке объектов базы данных, после чего выполнить команду контекстного меню All tasks (Все задачи) - Manage Triggers. Эти действия приведут к открытию диалогового окна свойств триггера.

```
CREATE TRIGGER tri_stip
ON StudySchema.Students
FOR INSERT, UPDATE
AS
/*описываются локальные переменные*/
DECLARE @stip smallmoney
/* определяется информация о вставляемых записях*/
SELECT @stip = U.stud_stip
FROM Inserted U
IF @stip >15000
BEGIN
```

```

/*Команда ROLLBACK используется для того, чтобы отменить
модификацию данных в случае, если база заблокирована*/
ROLLBACK TRAN
RAISERROR ('Стипендия студента не может превышать 15000',
16,10)
END;

```

Определим триггер *Tri_Ins_Progress* для таблицы *Progress*, который будет запускаться каждый раз, когда запись вставляется в таблицу *Progress* или модифицируется. Если экзамен или зачет сданы не в срок (например, после 15-го числа месяца), то запись не принимается:

```

CREATE TRIGGER Tri_Ins_Progress
ON StudySchema.Progress
FOR INSERT, UPDATE
AS
/*описываются локальные переменные*/
DECLARE @nDayOfMonth TINYINT
/* определяется информация о вставляемых записях*/
SELECT @nDayOfMonth = DATEPART (Day, I.Pr_DATE)
FROM Progress P, Inserted I
WHERE P.Stud_ID = I.Stud_ID AND P.Oценка = I.Oценка
/*проверяется условие вставки записи*/
/*и при необходимости сообщается об ошибке*/
IF @nDayOfMonth >15
BEGIN
/*Команда ROLLBACK используется для того, чтобы отменить
модификацию данных в случае, если база заблокирована*/
ROLLBACK TRAN
RAISERROR ('Вводить оценки, полученные до 15-го
числа', 16,10)
END;

```

Попробуйте ввести в таблицу *Progress* данные об оценках, полученных студентами позже 15-го числа.

Триггер можно удалить следующим образом:

```

DROP TRIGGER имя_триггера
,

```

или из контекстного меню All tasks (Все задачи) - Manage Triggers выбором имени триггера из списка.

Триггер на добавление студента:

```

CREATE TRIGGER tr_ins_stud
ON StudySchema.Students
FOR INSERT
AS

```

```

DECLARE @grup integer
SELECT @grup=I.Grup_ID
FROM Inserted I
    UPDATE StudySchema.Grup
    SET StudySchema.Grup.Grup_KOLSTUD =
StudySchema.Grup.Grup_KOLSTUD+1
    WHERE StudySchema.Grup.Grup_ID=@grup

```

Триггер: преподаватель, у которого стаж меньше 5 лет, не может вести больше 200 часов занятий

```

CREATE TRIGGER mol_spec
ON StudySchema.Study
FOR insert,update
AS
DECLARE @hour int
SELECT @hour=(select sum(s.lesson_hours)
               from StudySchema.Study s, inserted i
               where s.teach_id=i.teach_id)
BEGIN
DECLARE @staj datetime
SET @staj=(select floor(datediff(day, t.rab_date,
                                GetDate()))/365)
           from DekanatSchema.Teacher t, inserted i
           where t.teach_id=i.teach_id)
IF (@hour>200 and @staj<=5)
BEGIN
ROLLBACK tran
RAISERROR ('Молодой специалист не может взять более 200
часов занятий',16,10)
END
END;

```

Создадим триггер *tr_ins_stud*, который обновляет старост в таблице *Students*, если в таблице *Grup* меняется староста.

```

CREATE TRIGGER tr_ins_stud
ON StudySchema.Grup
FOR INSERT, update
AS
DECLARE @Stud_Star int, @grup int
SELECT @Stud_Star=I.Stud_Star, @grup=I.Grup_ID
FROM StudySchema.Grup I
    UPDATE StudySchema.Students
    SET StudySchema.Students.Stud_Star = @Stud_Star
    WHERE StudySchema.Students.Grup_id=@grup

```

Пусть нам необходимо знать, кто (пользователь) и когда производил изменения оценок студентов. Для этого необходимо создать таблицу журнала и собственно триггер.

Пример: создадим таблицу для журналирования изменений в базе данных:

```
CREATE TABLE journ (
  mod_oper CHAR(20),      /* Тип производимой операции */
  mod_datetime DATETIME, /*Дата изменения */
  mod_user VARCHAR(30),  /*Пользователь БД */
  mod_id INTEGER,        /* id студента чья оценка была изменена*/
  mod_ocen integer,      /* Измененная оценка*/
  old_ocen integer
)
```

Триггер, регистрирующий изменения оценки:

```
CREATE TRIGGER reg_oc
  ON StudySchema.Progress
  FOR Update
  AS
  DECLARE @id int, @ocen int, @old_ocen int
  SELECT @old_ocen=P.ocenka,@id = P.stud_id , @ocen =
U.ocenka
  FROM StudySchema.Progress P, Inserted U
  INSERT INTO journ
  VALUES ('Обновлена',Current_timestamp,Current_USER,@id,
@ocen,@old_ocen) ;
```

Данный триггер будет вносить информацию о изменениях оценок студентов.

Контрольные вопросы:

- 1) Когда выполняются триггеры – до или после выполнения команд INSERT, UPDATE и DELETE?
- 2) Можно ли изменить триггер?

9 Права пользователей. Разработка системы пользователей базы данных

9.1 Система безопасности СУБД

Система безопасности СУБД основывается на таких понятиях как *учетная запись, роль, группа, пользователь* [9].

Каждый пользователь проходит два этапа проверки системы безопасности при попытке доступа к данным [10]:

- этап 1 – аутентификация;

- этап 2 – получение доступа к данным.

Первый этап относится к уровню работы всего сервера СУБД. На первом этапе пользователь идентифицирует себя с помощью логического имени (*login*) и пароля (*password*).

Логическое имя и пароль хранятся на сервере СУБД в виде учетной записи (*account*).

Если данные были введены правильно, то считается, что процедура аутентификации пройдена, и данный сервер СУБД разрешает попытку доступа к конкретной базе данных.

Второй этап. Однако сама по себе аутентификация не дает пользователю прав доступа к каким бы то ни было данным. Для получения доступа к данным текущей БД необходимо, чтобы учетной записи пользователя соответствовал некоторый пользователь базы данных (*database user*). *Пользователь базы данных* – совокупность разрешений и запретов на работу с данными в конкретной базе данных.

На втором этапе учетная запись пользователя отображается в пользователя базы данных и получает все привилегии, соответствующие этому пользователю базы данных. Второй этап задействует систему безопасности конкретной базы данных, а не всего сервера СУБД.

В разных базах данных одной и той же учетной записи могут соответствовать одинаковые или разные имена пользователя базы данных с разными правами доступа.

В том случае, когда учетная запись пользователя не отображается в пользователя базы данных, клиент все же может получить доступ к базе данных под гостевым именем *guest* (если оно не запрещено администратором БД). Гостевой вход позволяет минимальный доступ к данным только в режиме чтения.

Третий этап. Обычно после создания логина и пользователя базы данных следующее, что нужно сделать, — предоставить пользователю разрешения в базе данных. Один из способов сделать это — воспользоваться ролями базы данных.

Роли базы данных — это специальные объекты, которые используются для упрощения предоставления разрешений в базах данных. В отличие от серверных ролей, которые могут быть только встроенными, роли баз данных могут быть как встроенными, так и пользовательскими. Встроенные роли баз данных обладают предопределенным набором разрешений, а пользовательские роли можно использовать для группировки пользователей при предоставлении разрешений. Обычно пользовательские роли используются только для логинов SQL Server (хотя им можно назначать любых пользователей баз данных, в том числе созданных на основе логинов Windows). Для группировки логинов Windows обычно удобнее и проще использовать группы Windows.

Т.о. пользователи баз данных могут объединяться в *роли* (иногда – *группы*) для упрощения управления системой безопасности.

| 9.2 Роли базы данных

Роли базы данных объединяют нескольких пользователей в административную единицу и позволяют назначать права доступа к объектам базы данных для роли, наделяя этими правами всех участников этой роли. Различают *пользовательские* и *встроенные* роли. *Встроенные* роли создаются автоматически при установке сервера СУБД (и не могут меняться). Различают *встроенные роли уровня СУБД* и *уровня конкретной базы данных*. Так, например, в SQL Server есть такие роли сервера:

Встроенная роль сервера	Назначение
Sysadmin	Может выполнять любые действия в SQL Server
Serveradmin	Выполняет конфигурирование и выключение сервера
Setupadmin	Управляет связанными серверами и процедурами, автоматически запускающимися при запуске SQL Server
Securityadmin	Управляет учетными записями и правами на создание базы данных, также может читать журнал ошибок.
Processadmin	Управляет процессами, запущенными в SQL Server
Dbcreator	Может создавать и модифицировать базы данных
Diskadmin	Управляет файлами SQL Server
Bukladmin	Может вставлять данные с использованием средств массового копирования, не имея непосредственного доступа в таблицам.

SQL Server поддерживает такие *встроенные роли уровня базы данных*:

Встроенная роль базы данных	Назначение
Db_owner	Имеет все права в базе данных
Db_accessadmin	Может добавлять или удалять пользователей
Db_securityadmin	Управляет всеми разрешениями, объектами, ролями и членами ролей
Db_ddladmin	Может выполнять любые команды DDL, кроме GRANT, DENY, REVOKE
Db_backupoperator	Может выполнять команды DBCC, CHECKPOINT, BACKUP
Db_datareader	Может просматривать любые данные в любой таблице
Db_datawriter	Может модифицировать любые данные в любой

	таблице
Db_denydatareader	Запрещается просматривать данные в любой таблице
Db_denydatawriter	Запрещается модифицировать данные в любой таблице

Кроме указанных ролей, существует еще одна, не удаляемая роль — *public*. Участниками этой роли являются все пользователи, имеющие доступ к базе данных. Нельзя явно указать участников этой роли, т.к. все пользователи уже включены в нее.

Еще одна специальная разновидность ролей баз данных — роли приложений (*application roles*).

9.3 Манипулирование элементами системы безопасности СУБД

9.3.1 Создание учетной записи.

Для создания учетной записи в MS SQL Server используется системная хранимая процедура *sp_addlogin*:

Формат процедуры:

```
sp_addlogin [@loginname=] login
[, [@passwd=] password]
[, [@defdb=] database]
[, [@deflanguage=] language]
[, [@sid=] SID]
[, [@encryptopt=] encryption_option]
```

Пример. Добавить учетную запись *teacher* к базе данных *education*.

```
EXEC sp_addlogin teacher, 123, education;
EXEC sp_adduser 'teacher', 'teacher';
```

Для создания учетной записи из скрипта — при помощи команды *CREATE LOGIN*. Хранимая процедура *sp_addlogin*, которая использовалась для этой цели в предыдущих версиях SQL Server, оставлена только для обеспечения обратной совместимости и к использованию не рекомендуется. Например, команда на создание логина SQL Server с именем *Student* и паролем *P@ssw0rd* и доступом к базе *education* (для всех остальных параметров будут приняты значения по умолчанию) может выглядеть так:

```
CREATE LOGIN Student WITH PASSWORD = 'P@ssw0rd',
DEFAULT_DATABASE= education;
```

Если вы создаете логин Windows, вам потребуется выбрать соответствующую учетную запись Windows (доменную или локальную). Не забудьте о возможности использовать группы Windows.

9.3.2 Создание пользователя базы данных.

После завершения аутентификации и получения *идентификатора учетной записи (login ID)* пользователь считается зарегистрированным, и ему предоставляется доступ к серверу. Для каждой базы данных, к объектам которой он намерен получить доступ, *учетная запись пользователя (login)* ассоциируется с *пользователем (user)* конкретной базы данных, что осуществляется посредством процедуры.

Следовательно, после создания логинов следующая задача администратора — спуститься на уровень базы данных и создать объекты пользователей базы данных. Пользователи баз данных — это специальные объекты, которые создаются на уровне базы данных и используются для предоставления разрешений в базе данных (на таблицы, представления, хранимые процедуры). Для пользователей используется термин *database users* (или просто *users*), в отличие от логинов (*logins*) — учетных записей для подключения к SQL Server. Логины и пользователи баз данных — это совершенно разные объекты.

На практике разделение логинов и пользователей баз данных обеспечивает большую гибкость. Например, пользователь, который входит от имени одного и того же логина, сможет работать в разных базах данных от имени разных пользователей.

В SQL Server 2008 для предоставления разрешений в базе данных используется объект пользователя, а для именования объектов в базе данных используется специальный объект схемы. Запрос с использованием полного формата имени в SQL Server 2008 выглядит так:

```
SELECT * FROM MyServer.MyDatabase.Schema1.Table1;
```

Схема формально определяется как набор объектов в базе данных, объединенных общим пространством имен. Проще всего представить себе схему, как некий логический контейнер в базе данных, которому могут принадлежать таблицы, представления, хранимые процедуры, пользовательские функции, ограничения целостности, пользовательские типы данных и другие объекты базы данных. Этот контейнер удобно использовать как для именования объектов и их логической группировки, так и для предоставления разрешений. Например, если в базе данных есть набор таблиц с финансовой информацией, удобно поместить их в одну схему и предоставлять пользователям разрешения на эту схему (т. е. на этот набор таблиц).

Пользователю можно назначить схему по умолчанию. В эту схему SQL Server будет по умолчанию помещать объекты, которые создает этот пользователь. Кроме того, искать объекты, к которым обращается пользователь (например, в случае запроса вида *SELECT * FROM Table1*), SQL Server также будет в первую очередь в его схеме по умолчанию.

Применение схемы дает ряд дополнительных преимуществ по сравнению со старым подходом:

- нескольким пользователям можно назначить одну и ту же схему по умолчанию, что может быть удобно при разработке приложений;
- несколько пользователей (через группы Windows или роли баз данных) могут владеть одной и той же схемой. При этом один пользователь может являться владельцем сразу нескольких схем;
- при удалении пользователя из базы данных не придется переименовывать его объекты;
- как уже говорилось, упрощается предоставление разрешений для наборов объектов в базе данных.

Создание схемы производится из контейнера *Имя_базы_данных | Security | Schemas* в Management Studio или при помощи команды *CREATE SCHEMA*.

Пример. Создадим схему *Accounting* с владельцем *Student*.

```
CREATE SCHEMA Accounting AUTHORIZATION Student;  
GO
```

Для создания пользователя базы данных в MS SQL Server 2012 используется системная хранимая процедура *sp_grantdbaccess* (ранее - *sp_adduser*).

Ее формат таков:

```
sp_grantdbaccess [@loginame=] login  
[,[@name_in_db=] user [OUTPUT]]
```

Пример. Добавить нового пользователя *teacher_adb* к базе данных *Lab_Study*.

```
EXEC sp_adduser teacher, teacher_adb, db_owner
```

Но в основном создают пользователя базы данных следующими способами:

- на графическом экране из контейнера *Имя_базы_данных | Security | Users* в Management Studio;
- при помощи команды *CREATE USER* (хранимая процедура *sp_adduser*, которая использовалась для этой цели в предыдущих версиях SQL Server, оставлена только для обеспечения обратной совместимости).

Например, команда на создание пользователя *User1*, которому будет соответствовать логин SQL Server *Login1* со схемой по умолчанию *dbo*, может выглядеть так:

```
CREATE USER User1 FOR LOGIN Login1  
WITH DEFAULT_SCHEMA = dbo;
```

При создании пользователя вам нужно будет указать:

- *имя пользователя (User name)*, к которому применяются те же правила, что и для других объектов SQL Server;
- *логин (SQL Server или Windows)*, которой будет назначен пользователю этой базы данных. После создания пользователя назначенный ему логин изменять будет нельзя. Можно создать пользователя, которому не будет назначен никакой логин (при помощи переключателя *Without login*). Такому пользователю уже не получится назначить логин. Пользователи этого типа — без логинов — используются только для дополнительной настройки безопасности в Service Broker. Отметим также, что если какой-то логин уже был назначен пользователю, то другому пользователю одновременно назначить его нельзя;
- *сертификат (Certificate name) или асимметричный ключ (Key name)*;
- *схему по умолчанию (Default schema)*;
- для каких схем этот пользователь будет являться *владельцем (Owned schemas)*;
- *какие роли базы данных (Database roles)* будут ему назначены.

Обязательных параметров всего два — имя пользователя и логин.

Изменение свойств пользователя и его удаление производится из того же контейнера в Management Studio, что и создание пользователя, а также при помощи команд *ALTER USER/DROP USER*. Удалить пользователя, владеющего какими-либо объектами в базе данных, нельзя.

9.3.3 Создание роли базы данных.

Для создания роли базы данных в MS SQL Server 2008 используется системная хранимая процедура *sp_addrole* [11]:

Ее формат таков:

```
sp_addrole [@rolename=] role  
[, [@ownername=] owner]
```

Эта процедура не может быть запущена как часть транзакции, определенной пользователем. Исполнять эту процедуру могут *только* участники роли сервера *sysadmin*, либо участники ролей *db_securityadmin*, *db_owner*.

Пример. Добавление роли Managers.

```
EXEC sp_addrole Managers;
```

В отличие от серверных ролей роли баз данных вы можете создавать самостоятельно. Это можно сделать из контекстного меню для контейнера *Имя базы данных | Security | Roles | Database Roles* или при помощи команды *CREATE ROLE*, например:

```
CREATE ROLE DBRole1;
```

Кроме имени роли, при создании можно также указать ее владельца (если это не указано, владельцем роли автоматически станет создавший ее пользователь базы данных). Если вы создаете роль при помощи графического интерфейса, вы можете сразу назначить роль владельцем схемы в базе данных и назначить пользователей, которые будут обладать правами этой роли.

Встроенным ролям назначены предопределенные права, изменить которые невозможно. Предоставление прав пользовательским ролям производится точно так же, как и обычным пользователям базы данных.

9.3.4 Добавление пользователя в роль.

Используется процедура *sp_addrolemember* вида:

```
sp_addrolemember [@rolename=] role,  
[@membername=] security_account
```

где

role - имя роли, в которую добавляется пользователь;

security_account – пользователь базы данных.

Пример. Добавление пользователя *teacher_abd* в роль *Managers*.

```
EXEC sp_addrolemember Managers, teacher_abd
```

9.3.5 Удаление ролей, учетных записей.

Рассмотрим в той очередности, которая разрешается в SQL Server, хранимые системные (всегда начинаются с букв *sp*) процедуры удаления пользователей и ролей из базы данных.

Системная хранимая процедура *sp_droprolemember* вычеркивает участника из роли:

```
sp_droprolemember [@rolename = ] role ,  
[@membername = ] security_account
```

Системная процедура *sp_droprole* удаляет роль (в том случае, если предварительно из роли были удалены все участники):

```
sp_droprole [@rolename=] role
```

Системная процедура *sp_revokedbaccess* (и ее устаревший аналог *sp_dropuser*) удаляет пользователя базы данных:

```
sp_revokedbaccess [@name_in_db =] name
```

Системная процедура *sp_droplogin* удаляет учетную запись из реестра сервера СУБД:

```
sp_droplogin [@loginame=] login
```

9.3.6 Просмотр информации об учетных записях, ролях, привилегиях.

Для просмотра информации о текущих назначениях пользователей, ролей используются хранимые процедуры *sp_helpuser*, *sp_helplogins*, *sp_helprole*, *sp_helprolemember*.

Параметры их вызовов таковы:

```
sp_helpuser [[@name_in_db=] security_account]  
sp_helplogins [[@LoginNamePattern=] login]  
sp_helprole [[@rolename=] role]  
sp_helprolemember [[@rolename=] role]
```

9.4 Предоставление прав на объекты в базе данных

Главное принципиальное изменение, которое произошло в SQL Server 2005 и соответственно в SQL Server 2008, — это появление возможности предоставлять разрешения на уровне схемы. К схеме в SQL Server 2005 могут относиться таблицы, представления, хранимые процедуры, пользовательские функции, ограничения целостности, пользовательские типы данных и другие объекты, на которые приходится предоставлять разрешения чаще всего. Если вы назначите пользователю разрешения на схему, то он получит разрешения на все объекты этой схемы.

9.4.1 Изменение прав доступа к таблице.

Таблица и столбцы хранят данные, которые извлекают и создают приложения. Управление доступом к этим данным осуществляется через иерархию разрешений SQL Server 2008. Управлять этой иерархией разрешений можно при помощи инструкций *GRANT*, *DENY* и *REVOKE*.

GRANT - разрешает роли или пользователю выполнять операции, определенные в момент предоставления разрешения.

DENY - запрещает пользователю или роли определенные разрешения и предотвращает наследование этих разрешений от других ролей..

REVOKE - отзывает ранее запрещенные или предоставленные разрешения.

WITH GRANT — разрешение не только предоставлено данному пользователю, но он также получил право предоставлять это разрешение другим пользователям. Можно предоставлять, конечно, только вместе с *GRANT*.

Работа с разрешениями производится одинаково для всех объектов базы данных:

- на вкладке *Permissions* свойств этого объекта (эта вкладка, правда, предусмотрена не для всех объектов, для которых можно предоставить разрешения);

- при помощи команд *GRANT* (предоставить разрешение), *DENY* (явно запретить что-то делать) и *REVOKE* (отменить явно предоставленное разрешение или запрет).

Далее перечислены разрешения, которые можно предоставить на уровне схемы. Мы приведем только разрешения для этого объекта, поскольку, во-первых, разрешения в большинстве случаев придется предоставлять именно на уровне схемы, а во-вторых, разрешения схемы включают в себя разрешения, которые можно предоставить другим объектам, например, разрешения *SELECT* для таблиц и представлений и *EXECUTE* для хранимых процедур.

ALTER — возможность вносить любые изменения в свойства объекта (за исключением смены владельца). На уровне базы данных можно настроить права *ALTER ANY ...* — возможность вносить изменения в любые объекты данного типа в базе данных, например, *ALTER ANY TABLE*.

CONTROL — аналогично разрешению *FULL CONTROL* в операционной системе. Тот, кому предоставлено такое разрешение, получает полные права как на сам объект, так и на информацию в нем.

DELETE — возможность удалять существующую информацию в таблицах. Применяется к таблицам, представлениям и столбцам.

EXECUTE — право запускать на выполнение. Применяется к хранимым процедурам и функциям.

INSERT — право на вставку новых данных в таблицы. Применяется к таблицам, представлениям и столбцам.

REFERENCES — разрешение, которое можно предоставить для проверки ограничений целостности. Например, пользователь может добавлять данные в таблицу с внешним ключом, а на таблицу с первичным ключом ему нельзя предоставлять права на просмотр. В этом случае на таблицу с первичным ключом ему можно предоставить право *REFERENCES* — и он сможет производить вставку данных в таблицу с внешним ключом, не получая лишних прав на главную таблицу. Это разрешение можно предоставлять на таблицы, представления, столбцы и функции.

SELECT — право на чтение информации. Предоставляется для таблиц, представлений, столбцов и некоторых типов функций (которые возвращают табличные наборы записей).

TAKE OWNERSHIP — право на принятие на себя владения данным объектом. Владелец автоматически обладает полными правами на свой объект. Такое право можно назначить для любых объектов.

UPDATE — возможность вносить изменения в существующие записи в таблице. Предоставляется на таблицы, представления и столбцы.

VIEW DEFINITION — право на просмотр определения для данного объекта. Предусмотрено для таблиц, представлений, процедур и функций.

Из кода Transact-SQL разрешения можно предоставлять командами **GRANT**, **DENY** и **REVOKE**.

Например, пользователю *Student* можно дать разрешения на создание баз данных, таблиц:

```
GRANT CREATE DATABASE, CREATE TABLE TO Student;
```

А чтобы предоставить пользователю *Student* возможность просматривать данные в таблице *Progress* в схеме *dbo*, можно воспользоваться командой:

```
GRANT SELECT ON dbo.Progress TO Student;  
GRANT SELECT ON Teachers TO Teacher, Student;
```

Лишить его ранее предоставленного права можно при помощи команды:

```
REVOKE CREATE DATABASE FROM Student;  
REVOKE SELECT ON dbo.Progress FROM Student;
```

Запретить пользователю *Student* разрешение **DELETE** на таблицу *Progress* независимо от того, какие разрешения этот пользователь мог унаследовать от роли:

```
DENY CREATE DATABASE TO Student;  
DENY DELETE ON Progress TO Student;
```

9.4.2 Предоставление доступа к отдельным столбцам.

В SQL Server есть возможность предоставить или отклонить доступ к отдельным столбцам, вместо того, чтобы работать со всей таблицей. Эта возможность предоставляет гибкость в отклонении доступа, например, к конфиденциальным данным в некоторых столбцах. Разрешения, которыми можно управлять для столбцов таблицы:

- 1) **SELECT** - Разрешает выполнить выборку из столбца
- 2) **UPDATE** - Разрешает изменять данные в столбце

Доступ к отдельным столбцам можно предоставить с помощью инструкции **GRANT**. В следующем примере разрешения **SELECT** и **UPDATE**

предоставляются пользователю *Student* столбцы *Stud_id* и *Ocenka* таблицы *Progress*.

```
GRANT SELECT, UPDATE (Stud_id, Ocenka) ON Progress TO
Student;
GRANT SELECT ON Teacher (Teach_Fam) TO teacher;
```

9.4.3 Отзыв доступ к столбцам.

Аналогично, можно отозвать доступ к отдельным столбцам при помощи инструкции *REVOKE*. Не забывайте, что, если нужно не допустить получения пользователем разрешения, необходимо использовать инструкцию *DENY*.

Отзываем ранее предоставленные или запрещенные разрешения на столбец *Ocenka* для пользователя *Student*.

```
REVOKE UPDATE (Ocenka) ON Progress TO Student;
```

Рассмотрим еще несколько моментов, связанных с предоставлением разрешений:

- в большинстве реальных задач используются десятки и даже сотни таблиц и других объектов базы данных. Предоставлять каждому пользователю разрешения на каждый из этих объектов очень неудобно. Если есть возможность, постарайтесь использовать разрешения на уровне схемы или всей базы данных. Часто упростить предоставление разрешений могут и встроенные роли баз данных (*db_datareader* и *db_datawriter*);

- существует общий принцип: не стоит обращаться из клиентского приложения к таблицам базы данных напрямую. Для изменения данных лучше использовать хранимые процедуры, а для запросов на чтение — хранимые процедуры или представления (при этом хранимые процедуры предпочтительнее). Причина проста: вы избежите внесения изменений в клиентское приложение, если потребовалось поменять структуру вашей базы данных (например, какая-то таблица поделилась на текущую и архивную или добавился новый столбец). Это следует помнить и при предоставлении разрешений. Отметим также, что при помощи хранимых процедур можно очень просто реализовать дополнительные проверки в добавление к обычным разрешениям;

- SQL Server позволяет настраивать разрешения на уровне отдельных столбцов. На практике лучше не пользоваться такими разрешениями из-за падения производительности и усложнения системы разрешений. Если пользователю можно видеть не все столбцы в таблице (например, ему не нужны домашние телефоны сотрудников), то правильнее будет создать представление или хранимую процедуру, которые будут отфильтровывать ненужные столбцы;

– в SQL Server 2005 появилась кнопка *Effective Permissions* (она расположена на вкладке *Permissions*). Эта кнопка позволяет посмотреть итоговые разрешения для пользователя или роли базы данных (поскольку разрешения от разных ролей базы данных, назначенных этому пользователю, суммируются).

9.5 Роли приложений

Альтернативный метод предоставления разрешений в базе данных SQL Server 2008 — воспользоваться ролью приложения (*application role*). Отличием этого метода является то, что разрешения предоставляются не пользователю, а приложению, из которого пользователь подключается к базе данных.

Применение этого способа выглядит так:

- пользователь от имени любого логина (которому должен соответствовать какой-нибудь объект пользователя в базе данных с правом *CONNECT*) подключается к серверу и делает нужную базу данных текущей;
- затем пользователь выполняет хранимую процедуру *sp_setapprole*, чтобы активизировать указанную им роль приложения. При этом в базе данных он получает права этой роли приложения (и теряет свои текущие права);
- после того, как необходимость в правах роли приложения закончилась, пользователь может опять переключиться на свою исходную учетную запись (и получить соответствующие ей права). Эта новая возможность SQL Server 2005 обеспечивается при помощи хранимой процедуры *sp_unsetapprole*.

Чаще всего такие роли используются в тиражируемых приложениях, когда задача вполне может обслуживаться не очень квалифицированным администратором. Преимущества такого подхода очевидны:

- гарантируется, что приложение будет обладать необходимыми правами в базе данных;
- администратору достаточно создать любой логин и любую учетную запись в базе данных, без необходимости настраивать требуемые разрешения.

Однако у ролей приложений имеются также и недостатки. В первую очередь, они связаны с тем, что пароль роли приложения передается по сети открытым текстом (предусмотрена возможность использования функции *ODBC Encrypt*, но реальной защиты она не обеспечивает). Как правило, этот пароль можно извлечь и из двоичного кода клиентского приложения.

Стоит ли использовать роли приложений? Все зависит от ситуации. Если приложение будет работать только на одном предприятии и обслуживаться квалифицированным администратором, то проще использовать обычные средства предоставления разрешений. Если же приложение будет устанавливаться на десятках и сотнях предприятий и за квалификацию администратора вы поручиться не можете, то применение роли

приложения может быть очень удачным решением. Часто в качестве альтернативы разработчики требуют предоставлять всем пользователям права владельца базы данных, а иногда и системного администратора сервера, что, конечно, является намного большим злом.

Создание роли приложения производится точно так же, как и создание обычных ролей — из контейнера *Имя_базы_данных | Security | Roles | Application Roles*. Главное отличие заключается в том, что назначить пользователей этой роли невозможно, зато ей нужно будет указать пароль, который будет использоваться при активизации. Команда Transact-SQL на создание роли приложения может выглядеть так:

```
CREATE APPLICATION ROLE AppRole1  
WITH PASSWORD = 'password';
```

Предоставление прав этой роли ничем не отличается от предоставления прав обычным пользователям или ролям базы данных.

При подключении к базе данных клиентская программа, которая использует роль приложения, должна выполнить код, аналогичный следующему:

1) Объявляем переменную *appCookie*. Она потребуется для возвращения к исходным правам

```
DECLARE @appCookie varbinary(8000);
```

2) Активизируем роль приложения

```
EXEC sp_setapprole 'AppRole1', 'password', 'none', TRUE,  
@appCookie OUTPUT;
```

3) Проверяем

```
SELECT USER_NAME();
```

4) Возвращаемся к исходным правам

```
EXEC sp_unsetapprole @appCookie;
```

5) Проверяем еще раз

```
SELECT USER_NAME();
```

При желании, конечно, можно и не возвращаться к исходным правам. Но, например, если потребуется во время работы имени роли приложения обратиться к другой базе данных, вы не сможете получить в ней других прав, кроме прав, предоставленных для специального пользователя *guest*.

9.6 Изменение контекста выполнения. Выражение *EXECUTE AS*

Новая возможность SQL Server 2005/2008 — изменение контекста выполнения команд Transact-SQL. Например, вы подключились от имени пользователя, у которого нет прав на определенную таблицу. Но в то же время вы обладаете правом выступать от имени другого пользователя, у которого права на эту таблицу есть (право действовать от имени другого пользователя называется *IMPERSONATE*). Вместо того чтобы разрывать соединение с SQL Server и входить заново от имени второго пользователя, вы можете просто "превратиться" во второго пользователя при помощи выражения *EXECUTE AS* и выполнить необходимые запросы к таблице от имени этого пользователя и с его правами, а потом, при желании, вернуться обратно.

В принципе, возможность менять контекст выполнения была и в предыдущих версиях SQL Server. Она представлялась командой *SET USER* (для обратной совместимости эта команда сохранена и в SQL Server 2005/2008). Однако возможностей у *EXECUTE AS* намного больше:

- команда *SET USER* позволяла менять контекст выполнения только пользователям с правами системного администратора (на уровне сервера) или *dbo* (на уровне базы данных). Поскольку прав у этих пользователей и так всегда достаточно, то обычно команда *SET USER* применялась только для проверки, как та или иная команда будет выполняться от имени определенного пользователя. Выражение *EXECUTE AS* могут использовать любые пользователи, поэтому основное назначение этого выражения — "подключение" пользователю дополнительных прав на время выполнения определенных команд;

- выражение *EXECUTE AS*, в отличие от команды *SET USER*, позволяет возвращаться «обратно» — к исходной учетной записи, которая использовалась до смены контекста выполнения. Для этого используется команда *REVERT*. При этом с помощью *EXECUTE AS* вы вполне можете поменять контекст выполнения несколько раз. Каждая выполненная команда *REVERT* вернет контекст выполнения на один «уровень» назад.

Работа с выражением *EXECUTE AS* очень проста. Предположим, что у вас есть пользователь базы данных *User1*, у которого нет прав на таблицу *Table2*, и пользователь *User2*, у которого такие права есть. До начала применения команды *EXECUTE AS* необходимо предоставить пользователю *User1* право *IMPERSONATE* на объект пользователя *User2*, чтобы он получил возможность выполнять команды от имени этого пользователя. На графическом экране это можно сделать так:

- откройте свойства пользователя *User1* в Management Studio и перейдите на вкладку *Securables*;
- нажмите кнопку *Add* под списком *Securables*, в открывшемся окне установите переключатель в положение *All objects of the types (Все объекты типа)* и нажмите кнопку *OK*;
- в списке типов объектов выберите *Users* и нажмите кнопку *OK*;

– в загруженном списке пользователей выберите пользователя *User2* и установите для него флажок в столбце *Grant* напротив разрешения *IMPERSONATE*.

Соответствующая команда Transact-SQL может выглядеть так:

```
GRANT IMPERSONATE ON USER::User2 TO User1;
```

После этого можно использовать возможности выражения *EXECUTE AS*, например, так:

- переключаемся в контекст выполнения *User2*

```
EXECUTE AS USER = 'User2';
```

- обращаемся к таблице *Table2*

```
SELECT * FROM dbo.Table2;
```

- возвращаемся в контекст выполнения *User1*

```
REVERT;
```

Чаще всего выражение *EXECUTE AS* используется в определениях программных модулей Transact-SQL (процедур или функций). Это позволяет менять контекст выполнения только на время работы данной хранимой процедуры или функции, например:

```
CREATE PROCEDURE SelectFromTable2  
WITH EXECUTE AS 'User2'  
AS SELECT * FROM dbo.Table2;
```

Если у пользователя есть право *EXECUTE* на хранимую процедуру *SelectFromTable2*, он может выполнять ее от имени пользователя *User2*.

В такой ситуации проще бы было сделать так, чтобы владельцем хранимой процедуры и таблицы был один и тот же пользователь, тогда вам не нужно будет прибегать к таким приемам, но это возможно не всегда.

Если вы используете выражение *EXECUTE AS* в хранимых процедурах и функциях, ему можно передать несколько зарезервированных значений, которые подставляются вместо имени пользователя:

1) *EXECUTE AS CALLER* — все команды в хранимой процедуре или функции будут выполнены с правами текущего пользователя (при этом неважно, кто является владельцем этой хранимой процедуры или функции);

2) *EXECUTE AS SELF* — все команды в хранимой процедуре или функции будут выполняться от имени пользователя, который создает или изменяет данную процедуру или функцию. Это значение используется в

редких случаях, когда клиентскому приложению требуется создавать хранимые процедуры или функции и запускать их от имени разных пользователей;

3) *EXECUTE AS OWNER* — все команды хранимой процедуры или функции будут выполняться от имени ее текущего владельца. Это значение тоже используется редко: обычно только в тех ситуациях, когда владелец хранимой процедуры или функции может часто меняться.

Выражение *EXECUTE AS* можно использовать не только для изменения контекста выполнения на уровне базы данных, но и на уровне сервера (т. е. логинов). Для этого ключевое слово *USER* меняется на *LOGIN*, например:

```
EXECUTE AS LOGIN = 'Login2';
```

Контрольные вопросы:

- 1) Как создать пользователя базы данных?
- 2) В чем преимущества учетной записи?
- 3) В чем преимущества использования системы безопасности СУБД?
- 4) Каковы понятия системы безопасности СУБД?
- 5) Какие бывают встроенные роли сервера СУБД?
- 6) Какие бывают встроенные роли базы данных?

Список литературы

- 1 Хансен Г., Хансен Д. Базы данных: разработка и управление. –М.: ЗАО «Издательство БИНОМ», 1999.
- 2 Питер Роб, Карлос Коронел. Системы баз данных: проектирование, реализация и управление, 5-е издание. – BHV Санкт-Петербург, 2004. – 1040 с.
- 3 Кренке Д. Теория и практика построения баз данных. Изд.9 – Питер, 2005.
- 4 Петкович Д. Microsoft® SQL Server™ 2012. Руководство для начинающих: пер. с англ. — СПб.: БХВ-Петербург, 2013. — 816 с.
- 5 Бондарь А. Г. Microsoft SQL Server 2012. — СПб.: БХВ-Петербург, 2013. — 608 с.
- 6 Клайн К. SQL. Справочник. 2-е изд./пер. с англ. – М.: Кудиц-Образ. 2006. – 832 с.
- 7 Форта Б. Освой самостоятельно SQL. 10 минут на урок, 3-издание.: пер. с англ. — М. : Издательский дом «Вильямс», 2005. —288 с.
- 8 Плю Р., Стивенс Р., Райан К. Освой самостоятельно SQL за 24 часа. – М.: Издательский дом «Вильямс», 2000.
- 9 Кригель А., Трухнов Б. SQL. Библия пользователя. — М.: «Диалектика», 2009. – 752 с.
- 10 Найт Б., Пэтел К., Снайдер В., Лофорт Р., Уорт С. Microsoft SQL Server 2008: руководство администратора для профессионалов.
- 11 [https://msdn.microsoft.com/ru-ru/library/bb510741\(v=sql.120\).aspx](https://msdn.microsoft.com/ru-ru/library/bb510741(v=sql.120).aspx)

Содержание

Введение.....	2
1 Концептуальное проектирование базы данных.....	4
2 Реализация модели базы данных в среде MS SQL Server.....	19
3 Манипуляция над данными.....	33
4 Простые запросы к базе данных.....	40
5 Создание сложных запросов.....	64
6 Представления.....	82
7 Хранимые процедуры и функции.....	88
8 Триггеры.....	100
9 Права пользователей. Разработка системы пользователей базы данных. ..	108
Список литературы.....	124

Сатимова Елена Григорьевна

ПРОЕКТИРОВАНИЕ БАЗ ДАННЫХ

Учебное пособие

Редактор

Н.М. Голева

Подписано в печать ____ .12.2016
Тираж 50 экз. Формат 60х84 1/16

Бумага типографская №2
Уч. изд.л. 8,0
Цена тг.

Некоммерческое АО «АУЭС»
Алматы, ул. Байтурсынова, 126

Копировально-множительное бюро
некоммерческого акционерного общества
«Алматинский университет энергетики и связи»
050013 Алматы, ул. Байтурсынова, 126